KingbaseES 数据库性能调优指南

金仓数据库管理系统 KingbaseES

文档版本：V9（V009R001C010）

发布日期：2025 年 12 月 3 日

中电科金仓（北京）科技股份有限公司

目　录

第 1 章　性能概述 1
1.1 什么是性能 1
1.2 理解性能指标 2
1.2.1 性能指标模型 2
1.2.2 数据库时间模型 2
1.3 理解可扩展性 3
1.3.1 什么是可扩展性 3
1.3.2 系统可扩展性 4
1.3.3 影响可扩展性的因素 5
1.4 性能问题产生模型 6
1.5 如何管理性能 7
1.5.1 被动性能管理 7
1.5.2 主动性能管理 7

第 2 章　新系统性能规划 8
2.1 整体过程 8
2.2 系统体系架构 9
2.2.1 硬件组件和软件组件 9
2.2.1.1 硬件组件 9
2.2.1.2 软件组件 10
2.2.2 根据需要合理配置系统结构 11
2.3 应用程序设计原则 14
2.3.1 应用程序设计的简单性 14
2.3.2 数据建模 14
2.3.3 表和索引的设计 14
2.3.3.1 将列追加到索引 15
2.3.3.2 使用其他索引类型 15
2.3.3.3 查找索引的成本 16
2.3.3.4 对索引中的列排序 16
2.3.4 使用视图 16
2.3.5 SQL 执行效率 16

2.3.6 实现应用程序 17
2.3.7 应用开发趋势 19
2.4 工作负载测试，建模及实施 20
2.4.1 确定数据大小 20
2.4.2 估计负载量 20
2.4.3 应用建模 21
2.4.4 测试，调试，验证设计 21

第 3 章 性能调优工具 23
3.1 性能调优工具概述 23
3.2 kbbadger 日志分析工具 24
3.2.1 工具概述 24
3.2.2 基本能力 25
3.2.3 使用示例 26
3.2.4 适用场景 27
3.3 动态性能视图 27
3.3.1 动态性能视图概述 28
3.3.2 动态性能视图分类 29
3.3.3 数据库实时状态相关视图 31
3.3.4 数据库实例统计相关视图 33
3.3.5 数据库对象统计相关视图 35
3.3.6 数据库时间相关视图 37
3.3.7 TOP SQL 相关视图 40
3.3.8 主机信息相关视图 42
3.3.9 管理动态性能视图 43
3.3.9.1 开启动态性能视图统计 43
3.3.9.2 关闭动态性能视图统计 44
3.3.9.3 清理动态性能视图统计 44
3.4 KWR 报告 45
3.4.1 KWR 快照概述 45
3.4.2 生成 KWR 快照 46
3.4.3 生成 KWR 报告 48
3.4.4 KWR 按库展示报告 50
3.4.5 KWR 数据库时间模型解析 51
3.4.5.1 数据库时间模型概述 52
3.4.5.2 按 SQL 报文分解 52
3.4.5.3 按 SQL 语句类型分解 53
3.4.5.4 按等待事件分解 54
3.4.5.5 按关键活动分解 55
3.4.5.6 按 SQL 执行阶段分解 55
3.4.5.7 按 TOP SQL 分解 56
3.4.6 KWR 其它性能指标解析 58

3.4.6.1 KWR 快照时间 58
3.4.6.1.1 快照时间概述 58
3.4.6.1.2 快照时间的作用 58
3.4.6.1.3 快照时间、TPS 和 QPS 60
3.4.6.2 实例效率百分比 60
3.4.6.2.1 缓存命中率 60
3.4.6.2.2 解析和计划重用率 61
3.4.6.2.3 执行计划缓存率 61
3.4.6.3 IO 性能指标解析 61
3.4.6.3.1 主机 IO 62
3.4.6.3.2 实例 IO 63
3.4.6.3.3 SQL IO 63
3.4.6.3.4 数据库对象 IO 64
3.4.6.4 内存统计指标解析 65
3.4.6.4.1 KWR 中的内存统计 65
3.4.6.5 数据库对象指标解析 65
3.5 KDDM 报告 66
3.5.1 KDDM 概述 66
3.5.2 KDDM 报告生成 67
3.5.3 KDDM 报告解析 67
3.5.4 KDDM 建议应用 71
3.6 KWR DIFF 报告 72
3.6.1 KWR DIFF 概述 72
3.6.2 KWR DIFF 报告生成 72
3.6.3 KWR DIFF 报告解析 73
3.7 KSH 报告 74
3.7.1 KSH 概述 74
3.7.2 KSH 报告生成 75
3.7.2.1 配置 GUC 参数 75
3.7.2.2 KSH 内部视图和关系表 75
3.7.2.3 生成 KSH 报告 76
3.7.3 KSH 报告解析 76
3.7.3.1 KSH 报告头解析 77
3.7.3.2 KSH 报告主体解析 77

第 4 章 性能问题调优 79
4.1 整体过程 79
4.2 定义性能目标 80
4.3 性能诊断，定位资源瓶颈 81
4.3.1 资源利用率瓶颈定位法 82
4.3.2 响应时间测量瓶颈定位法 82
4.4 性能诊断，定位优化点 83

4.5 性能调优，解决性能问题 83
4.6 性能诊断与优化经验 84

第 5 章 操作系统 VS 数据库诊断 86

第 6 章 操作系统性能诊断 87
6.1 操作系统诊断概述 87
6.2 操作系统资源监控工具 89
6.3 操作系统资源分析 90
6.3.1 CPU 资源分析 90
6.3.1.1 CPU 信息简介 90
6.3.1.2 CPU 瓶颈分析 91
6.3.1.3 CPU 瓶颈示例 - 单进程 91
6.3.1.4 CPU 瓶颈示例 - 多进程 92
6.3.2 内存资源分析 93
6.3.2.1 内存信息简介 93
6.3.2.2 内存资源分析 94
6.3.2.3 内存瓶颈示例 94
6.3.3 磁盘资源分析 95
6.3.3.1 IO 信息简介 95
6.3.3.2 IO 资源分析 96
6.3.3.3 I/O 瓶颈示例 97
6.3.4 网络资源分析 98
6.3.4.1 网络信息简介 98
6.3.4.2 网络资源分析 100
6.3.4.3 网络瓶颈示例 100

第 7 章 数据库系统性能分析与优化 101
7.1 时间模型基础 102
7.1.1 数据库时间分析 102
7.1.1.1 数据库时间模型概述 102
7.1.1.2 数据库时间模型组成 102
7.2 性能分析概述 103
7.3 等待事件瓶颈分析与优化 104
7.3.1 I/O 类等待事件分析与优化 104
7.3.1.1 DataFileRead 和 DataFileWrite 104
7.3.1.2 BufFileRead 和 BufFileWrite 107
7.3.2 LightLock 类等待事件分析与优化 110
7.3.2.1 buffer_content 110
7.3.2.2 lock_manager 111
7.3.2.3 wal_insert 112
7.3.2.4 WALWriteLock 113
7.3.3 Lock 类等待事件分析与优化 116

7.3.3.1 extend 116
7.3.3.2 Relation 116
7.3.3.3 transactionid 117
7.3.3.4 tuple 118
7.3.4 Client 类等待事件分析与优化 120
7.3.4.1 ClientRead 120
7.3.4.2 ClientWrite 121
7.4 CPU 瓶颈分析与优化 123
7.4.1 实例效率分析与优化 123
7.4.2 SQL 分析与优化 124
7.4.3 架构与硬件分析与优化 124

版权声明 125

服务周期承诺 126

第1章　性能概述

随着互联网技术在商业应用中的广泛应用，数据库系统变得日益庞大和复杂。在这种背景下，数据库性能表现的重要性也愈发显著。基于丰富的性能调优经验，KingbaseES 总结出了一套系统的数据库性能方法论，供读者参考和使用。

本章节包含以下内容：

- 什么是性能
- 理解性能指标
- 理解可扩展性
- 性能问题产生模型
- 如何管理性能

1.1　什么是性能

性能是衡量软件系统是否满足其时间要求的一种指标。时间要求主要通过响应时间和吞吐量来评估。响应时间是指对请求做出回应所需的时间。对于单个事务，它表示完成该事务所需的时间；对于用户任务，则指从发起请求到系统返回结果的端到端时延。例如，在线系统要求在用户按下回车键后的 0.5 秒内显示结果。系统吞吐量是指在特定时间段内，系统能够处理的请求数量。以电话交换机为例，其吞吐量可以达到每秒钟处理 1,000,000 次呼叫。

软件性能的时间特性主要体现在两个方面：响应性和可扩展性。

- **响应性**指系统实现既定响应时间或吞吐量目标的能力。
- **可扩展性**指当系统的功能需求增加时，仍能保持满足响应时间和吞吐量目标的特性。

1.2　理解性能指标

1.2.1　性能指标模型

- **数据库请求响应时间**：指从发出一个数据库请求到最终接收到所有结果数据所需的时间间隔。其计算公式为 CPU 处理时间 + 非空闲等待时间（如 IO 时间、数据库锁时间）。
- **并发吞吐量**：指单位时间内，数据库能够处理的请求数量。该指标与前一指标的关系是：假设系统中只有一个会话，则只要该会话持续发起请求，请求之间没有任何时间间隔，系统即可达到其最大并发吞吐量 = 1 / 数据库请求响应时间。相应的，假设系统中有 N 个会话，则系统的理论上最大并发吞吐量 = N / 数据库请求响应时间——这里达到最大吞吐量的前提是：多个会话之间没有任何相互等待；系统的各种硬件资源均能够支撑这 N 个会话的资源需求，即最大吞吐量 = 各类资源的处理能力 / 单个负载请求对本资源的处理能力需求。

注意:

这里的资源包括：CPU、内存、磁盘等存储设备、网络设备等硬件资源，数据库锁、应用锁等软件资源。

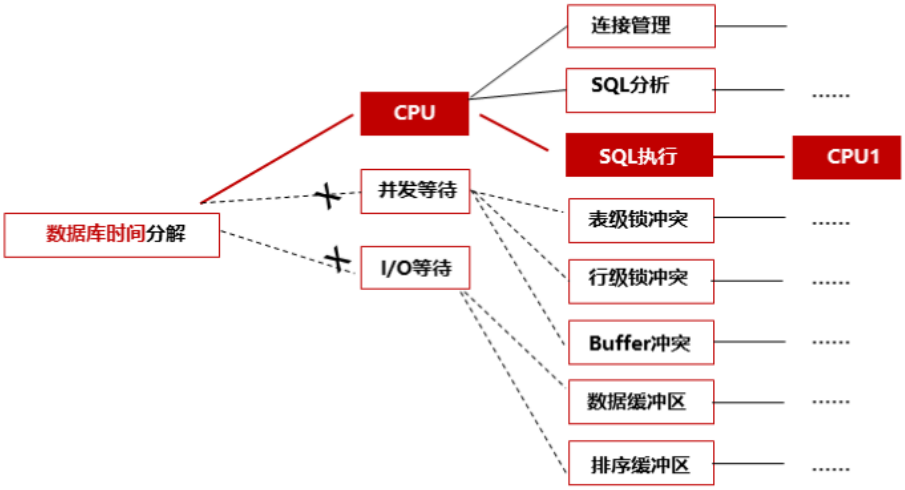
1.2.2　数据库时间模型

确定性能问题的根本原因并提出有效的调优建议是实现性能优化的首要任务。

显而易见，在分析用户请求中的性能问题时，采用自上而下的分析方法无疑是最直观的方式。数据库请求响应时间可作为基础度量依据，进而用于推导出吞吐量等相关数据。

基于此，我们引入了时间模型以解决该问题。时间模型通过量化分析，揭示了数据库中各种操作对性能的影响机制。在时间模型中，最重要的概念是“数据库时间”（DB Time）。DB Time 定义为数据库处理用户请求所消耗的总时间。这里，我们提供一个量化公式来描述数据库时间：

```
数据库时间 = CPU + 非空闲等待时间
```



数据库调优的核心目标在于缩短数据库时间：即减少 CPU 占用或等待时间，进而提升吞吐量。

注意:

需要注意的是，数据库时间并不等同于用户感知的响应时间。它仅包含数据库处理请求所花费的时间，不包括中间层（如应用、网络、中间件等）所消耗的时间。

1.3　理解可扩展性

在开发环境中，“可扩展性”一词被广泛使用。本节内容为应用设计者及性能调优专家提供一种新的“可扩展性”定义。本节包含以下内容：

- 什么是可扩展性
- 系统可扩展性
- 影响可扩展性的因素

1.3.1　什么是可扩展性

可扩展性是指当系统资源使用量成倍增加时，系统仍能有效处理更多负载的能力。理想情况下，若负载翻倍，所需的系统资源也应相应倍增。然而，由于系统内部的各种竞争和冲突，实际的资源消耗往往会超过预期。

以下是一些因系统资源冲突而导致可扩展性受限的例子：

- 用户增长引发的高并发处理需求
- 锁机制的开销增大
- 数据一致性维护带来的额外负担
- 操作系统资源占用上升
- 数据量增长导致的访问压力增加
- SQL 和索引设计不当引发的 I/O 开销激增

若一个应用程序过度消耗系统资源，导致在负载增加时无法进一步提高吞吐量，则该应用程序属于不可扩展的类别。这种设计会导致明确的性能瓶颈，具体表现为吞吐量无法按需增长以及响应时间显著恶化。

以下是一些系统资源耗尽导致性能下降的例子：

- 硬件资源达到极限
- 大规模表扫描引发磁盘 I/O 瓶颈
- 过度网络请求造成网络拥堵
- 内存不足触发分页与交换机制
- 过多进程/线程引发操作系统稳定性问题

这要求应用程序设计者制定通用资源管理策略，确保在各种用户规模和数据量下，系统资源使用不会超载。

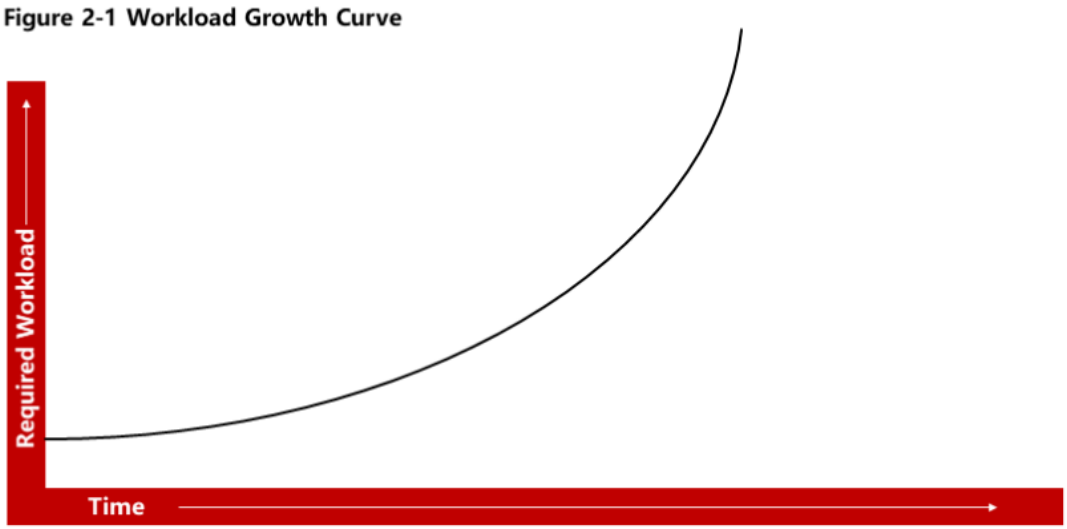
1.3.2　系统可扩展性

通过互联网访问的应用程序需要满足更为复杂的性能和可扩展性要求。尽管某些应用程序是为互联网环境专门设计和编写的，但即使是传统的后台系统（如总账应用程序），也可能需要部分或全部数据来支持在线操作。

互联网时代应用的特点包括：

- 需要全天候运行
- 不可预测且不精确的并发用户数
- 容量规划的困难
- 提供任何类型查询的可用性
- 多层体系结构
- 无状态中间件
- 快速增长的时间规模
- 较短的测试时间

下图展示了典型的工作负载增长曲线，需求呈指数级增长。应用程序必须随着工作负载的增加而相应扩展，并在添加额外硬件资源时提供充分支持。无论现有硬件资源多么充足或系统架构经过何种优化，设计缺陷仍可能导致系统性能达到瓶颈。



Description of “Figure 2-1 Workload Growth Curve”

互联网应用的开发时间通常非常短，留给测试、验证和优化的时间也非常有限。然而，糟糕的设计往往意味着团队在后期可能需要重新设计并重构系统。如果一个应用程序存在架构缺陷或功能瓶颈，并且实际的工作负载超过了预期需求，则很可能导致服务失败。从商业角度来看，这不仅会对用户体验造成负面影响，还可能导致客户流失。研究表明：如果互联网用户在等待响应的时间超过 7 秒——用户的注意力很可能永远消失。这一关键阈值凸显了系统性能和可用性对于用户留存的重要性。

通常情况下，重新设计系统以及迁移到新实现所产生的成本（包括相关停机时间）往往高于从一开始就构建一个稳定、可靠的系统。由此可见，从项目的早期阶段就将可扩展性和可维护性纳入考量，对于控制长期成本并确保系统的可持续发展具有重要意义。

1.3.3　影响可扩展性的因素

在构建应用程序时，设计师和架构师应当尽可能追求接近完美的可扩展性。这种理想状态被称为线性可扩展性，即系统吞吐量与 CPU 数量成正比。

然而，在实际操作中，完全的线性可扩展性是难以实现的，因为存在许多不可控的因素。尽管如此，仍应通过优化应用程序的设计和实现，尽可能提升系统的可扩展性，以确保当前及未来的性能目标能够通过硬件扩展和技术升级来实现。

影响线性可扩展性的因素包括：

- **糟糕的应用程序设计、实现和配置**

应用程序对可扩展性的影响最为显著。以下是一些典型问题：

- 模式设计不当：关键 SQL 语句未经过优化，导致无法充分利用系统资源。
- 事务设计不合理：锁竞争和序列化问题会严重限制系统的扩展能力。
- 连接管理不善：可能导致响应时间增加，甚至引发系统可靠性问题。

- **设计不是唯一问题，应用的物理实现同样可能是薄弱环节**

  例如：

  - 生产环境中采用了不恰当的输入输出（I/O）策略。
  - 生产环境和测试环境中生成的执行计划存在差异。
  - 内存密集型应用程序未考虑运行时内存释放机制，可能导致内存资源过度占用。
  - 内存利用率低下和内存泄漏问题对运行中的虚拟内存子系统造成显著压力，影响系统的整体性能和可用性。
  - **不正确的硬件组件大小**

    尽管硬件价格有所下降，但不当的容量规划仍可能成为扩展性的瓶颈。当系统负载增加时，过多或不足的硬件资源都会影响可扩展性。因此，选择合适的硬件配置至关重要。
  - **软件组件的限制**

    所有软件组件（如应用服务器、数据库等）均存在性能和资源使用上的局限性。应用程序设计者需充分了解这些限制，并避免超出其处理能力范围。
  - **硬件组件的限制**

    硬件并非完全可扩展，特别是在多处理器环境下。虽然有限数量的 CPU 能够接近线性扩展，但当 CPU 数量超过一定阈值时，额外增加的 CPU 可能无法按比例提升性能，甚至可能导致性能下降。这种现象与工作负载和操作系统设置密切相关。

1.4　性能问题产生模型

性能问题是由于某些系统资源竞争或耗尽所导致。当系统资源达到其处理能力的极限时，系统将无法进一步扩展性能。

而资源竞争或不足的根本原因在于：资源使用效率低下，或者资源的处理能力已无法满足系统的负载需求。

数据库需要依赖的软硬件资源包括：CPU、I/O、内存、网络和锁等。当这些资源接近或达到其处理能力上限时，通常会导致系统性能问题。常见的资源瓶颈现象可能表现为：

- CPU 利用率接近 100%
- I/O 利用率接近 100%
- 内存不足，开始使用 swap 空间

- 网络流量达到带宽限制、网络传输出现大量错误
- 数据库内部存在长时间等待状态

注意:

在进行性能调优时，DBA 的主要目标是识别并解决资源的不当使用问题；如果不存在不当使用，则说明资源的处理能力已无法满足系统的负载需求，此时应提供资源需求评估。

1.5　如何管理性能

1.5.1　被动性能管理

以下的说法是否听起来很耳熟？

- “把它构建起来，看它做的怎么样”
- “我们以后会调整，现在没时间考虑性能”
- “在没有能测量的东西之前，做不了任何有关性能的事”
- “性能？那是发布版本 V2 时要做的事情”

如果你觉得这些说法很熟悉，那么你很可能正在采用被动式软件性能管理。被动式的性能管理模式是在问题出现后才做出反应，这可能会导致预算超支、交付延迟以及项目失败等风险。

关于如何解决性能问题，请参见性能问题调优。

1.5.2　主动性能管理

主动性能管理是一种能够预见潜在性能问题并将其纳入开发早期的技术方法。与被动式管理不同，主动式的性能管理贯穿于整个软件开发生命周期。

主动性能管理涉及的环节较多，本文仅针对新系统的主动性能管理进行简要介绍。如需深入了解相关内容，请参考新系统性能规划（仅供参考）。

第2章　新系统性能规划

最优的系统性能从设计开始，并贯穿于系统的整个生命周期。在初始设计阶段就充分考虑性能问题，可以为后续的生产调优奠定坚实的基础。本章将涵盖以下小节：

- 整体过程
- 系统体系架构
- 应用程序设计原则
- 工作负载测试，建模及实施

2.1　整体过程

KingbaseES 新系统性能规划的一般过程如下：

1. 定义应用负载特征与性能目标

   明确数据规模、用户并发数量、应用操作特性（如事务类型）、响应时间及吞吐量等关键指标。

2. 根据启发式原则确定软件部署架构

   基于类似项目的经验，选择适合的架构方案，可能包括但不限于：

   - 应用和数据库资源部署在同一服务器
   - 应用服务与数据服务分离
   - 利用缓存技术优化性能
   - 构建应用服务器集群
   - 实现数据库读写分离
   - 采用分布式数据库架构
   - 使用 NoSQL 数据库和搜索引擎
   - 进行业务逻辑拆分

3. 根据负载特征与系统架构，估算硬件资源需求

   参考系统体系架构

4. 设计并开发应用程序

   遵循应用程序设计原则

5. 通过模拟或实际负载测试测量资源消耗

   参见工作负载测试，建模及实施

6. 验证是否达成初始设定的目标，如果未达标，则需重复步骤 3、4、5，持续优化直至满足性能要求。

2.2　系统体系架构

系统体系架构主要包含以下两个部分：

- 硬件和软件
- 根据需要合理配置系统架构

2.2.1　硬件组件和软件组件

系统结构主要包含硬件组件和软件组件：

- 硬件组件
- 软件组件

2.2.1.1　硬件组件

当今的设计师和架构师需要负责多层环境中每一层硬件的规模与容量规划。架构师的职责是实现一个平衡性设计。这类似于桥梁设计师的任务——必须全面考虑桥梁的各种有效载荷和结构要求。一座桥的坚固程度取决于其最薄弱环节。因此，桥梁的设计需保持整体平衡，以确保所有组件能够同时达到其设计极限。

以下是系统的主要硬件组件：

- CPU 是计算机系统的运算和控制核心。主要关注的因素包括：
  - 架构：常见的架构类型有 X86（如英特尔、AMD、海光、兆芯等）、MIPS（如龙芯）、ARM（如飞腾、麒麟等）以及 Alpha（如神威）。不同架构在效率上存在差异。
  - 核心数：在同等架构和工艺条件下，核心数越多，系统的扩展性越好。然而，过多的核心可能导致由于访存性能限制而无法实现线性性能提升。
  - 主频：在同等架构和工艺条件下，主频越高，性能表现越优异。

注意:

架构会带来效率上的差异。通常情况下，在相同核心数量下，X86 架构的性能优于鲲鹏等其他架构。此外，核心数越多，系统能够支持的并发连接和吞吐量也就越大。

- 内存：数据库和应用服务器需要大量内存来缓存数据，以减少耗时的磁盘访问。
  - 在同等主频条件下，内存容量越大越好。
- I/O 子系统：I/O 子系统的性能受多种硬件因素影响：
  - 硬盘类型：常见硬盘可分为两大类：机械硬盘（HDD）和固态硬盘（SSD）。相比机械硬盘，固态硬盘具有更快的访问速度，能够提供更高的输入输出操作次数（IOPS）和更大的存储带宽。
  - 硬盘转速：机械硬盘通过旋转磁头进行数据寻址，因此转速越快，理论上的 IOPS 值越高。常见转速包括 5400 转/分钟、7200 转/分钟和 15000 转/分钟。
  - 硬盘个数：通过使用磁盘阵列或人为控制多个硬盘资源，可以实现 IO 性能的线性扩展。硬盘数量越多，能够提供的 IOPS 越大。
- 网络：系统内的所有设备均通过调制解调器线路与外部网络相连，并借助高速内部局域网实现设备间的通信。网络性能主要关注两个关键指标：带宽（即网络的传输能力或容量）以及延迟（即数据传输所需的时间，影响整体响应速度）。

2.2.1.2　软件组件

就像计算机拥有相同的硬件组件一样，应用程序也有共同的功能模块。通过将软件开发划分为功能组件，可以更好地理解应用程序的设计与架构。某些系统组件由现有软件执行，以加速应用的实现，或避免重复开发通用组件。

软件组件和硬件组件的区别在于：硬件组件通常只执行单一任务（例如，磁盘驱动器仅用于存储和检索数据）。软件组件则可以承担多种角色。例如，客户端程序既可以管理用户界面，又可以执行业务逻辑。

大多数应用程序包含以下主要软件组件：

- 用户界面：

  该组件是用户最直接看到的部分，主要功能包括：

  - 向用户展示界面
  - 收集用户信息并传递给业务逻辑部分
  - 允许数据入口
  - 控制应用程序的状态或级别
- 业务逻辑：

  该组件实现对应用程序功能至关重要的核心规则。修复此组件中的错误通常代价较高，因为它直接影响应用程序的核心功能。主要包含：

  - 将数据模型转换为关系表结构。
  - 在关系表中定义约束（如主键和外键）。
  - 编写过程逻辑以实现业务规则（例如折扣策略）。

- 管理用户请求的资源：

  该功能在所有软件中普遍存在，但其实现方式可能会受到应用程序设计的影响。例如，在多用户应用程序中，大多数由用户请求分配的资源通常由数据库服务器或操作系统处理。然而，在用户数量动态变化或快速增长的应用程序中，系统架构师需要主动设计和优化，以避免单个组件过载或不稳定。主要包含：

  - 数据库的连接管理
  - 高效执行 SQL（游标和 SQL 共享）
  - 管理客户端状态信息
  - 在硬件资源之间平衡用户请求的负载
  - 为硬件和软件组件设置操作目标
  - 异步执行任务的持久队列

- 数据和事务：

  该组件对于数据库服务器和操作系统至关重要，因为它们负责确保数据的一致性和完整性。以下是其核心功能：

  - 使用锁和事务语义提供对数据的并发访问
  - 使用索引和内存缓存提供对数据的优化访问
  - 确保在发生硬件故障时记录数据更改
  - 强制执行为数据定义的任何规则

2.2.2　根据需要合理配置系统结构

配置初始系统架构是一个迭代和细致的工程过程。在这一过程中，系统架构师需要在预算和进度的限制范围内，确保满足系统的各项需求。如果系统的功能需求涉及交互用户根据数据库内容处理业务决策，那么该体系结构将由用户需求主导设计。相反，若系统中交互用户的数量较少，则应采用进程驱动的架构模式进行设计。

用户交互式应用举例：

- 会计和簿记应用程序
- 订单输入系统
- 电子邮件服务器
- 基于 Web 的零售应用程序
- 交易系统

进程驱动式应用举例：

- 公用事业计费系统
- 欺诈检测系统
- 直邮系统

在许多方面，进程驱动的应用程序比多用户应用程序更易于设计，因为这些应用程序通常不需要复杂的用户界面元素。然而，由于目标是面向过程的，系统架构师可能会面临处理大数据量和不同继承因素的挑战，这可能引起一定的混乱。进程驱动的应用程序从基于用户的应用程序和数据仓库中汲取方法论的支持进行构建。因此，本手册的重点将放在为交互式用户应用程序设计系统架构上。

注意:

生成系统架构并非一个确定的过程。在这一过程中，必须仔细考虑多种因素，包括业务需求、技术选择、现有基础设施和系统的状况，以及实际的物理资源（如预算和人力资源）。

以下问题将引发对系统架构的思考，尽管它们并非系统架构设计的具体指南。这些问题说明了业务需求如何影响系统的体系结构、易实现性以及整体性能和可用性。例如：

- **系统必须支持的用户数量为多少？**

  大部分应用设计时会归类为以下某一种类型：

  – 很少的用户或专用计算机：

    对于这种类型的应用程序，通常只有一个用户。应用程序的设计重点是通过提供良好的响应时间使单个用户尽可能高效地工作，同时减少管理需求。这些应用程序的用户间干扰较少，资源冲突也较为罕见。

  – 中等规模或大规模用户共享应用：

    对于这种类型的应用程序，用户的数量受到公司员工人数的实际业务处理需求限制。因此，用户数量是可以预测的。然而，提供可靠的服务对业务至关重要。由于用户需要共享资源，设计工作必须解决系统负载较重时的响应时间问题、每次会话的资源升级问题以及未来的增长空间。

  – 互联网上大量分布式用户：

    对于这类应用，需要额外的工作来确保系统组件不会超出其设计限制。这形成了性能瓶颈，可能阻碍或破坏系统的稳定性。这些应用程序要求复杂的负载均衡策略、无状态的应用服务器以及高效的数据库连接管理机制。此外，在数据库因系统过载而无法及时响应用户请求的情况下，应利用统计信息和调控器（governor）来确保用户仍能收到反馈。

- **用户交互的方法是什么？**

  用户界面的选择范围广泛，涵盖从简单的基于 Web 的浏览器到定制化的客户端程序。

- **用户的分布情况是怎样的？**

  用户之间的距离会影响应用程序如何处理网络延迟。当无法执行批处理或系统维护功能时，用户分布还会影响一天中高峰期的出现时间。

- **网络速度如何？**

  网络速度直接影响数据传输量和用户界面与应用程序及数据库服务器之间交互的方式。高度互动的用户界面可以在每次键盘输入或字段验证时与后端服务器通信，而较少互动的界面则采用屏幕发送和接收模型。在网络环境较慢的情况下，高度互动的用户界面将无法实现高效的数据输入。

- **用户需要获取的数据量是多少？其中有多少数据是只读的？**

  在线查询的数据量影响设计的各个方面，包括表、索引的设计以及表示层的开发。设计时必须确保用户的响应时间不受数据库规模的影响。如果应用程序主要为只读操作，则可以考虑将数据复制并分发到应用服务器的本地缓存中，从而减轻核心事务服务器的工作负载。

- **用户对响应时间的要求是什么？**

  用户类型决定了响应时间的需求。例如，需要快速决策的执行者需要高精度和高性能，而其他类型的用户（如数据录入人员）可能对性能要求相对较低。

- **用户是否需要全天候服务？**

  对于如今的互联网应用程序而言，全天候可用性是一项强制性的基本要求。这些应用程序通常需要支持实时交易和全球范围内的用户交互，因此必须确保 24 小时无间断服务。然而，在单一时区运行的企业系统可能能够容忍下班后的停机时间。例如，许多企业在深夜或周末期间安排系统维护，以减少对日常业务的影响。在这种情况下，可以利用非工作时间段执行批处理进程或进行系统管理。通过这种方式，虽然无法实现全天候的完全可用性，但在特定场景下，这种折中的解决方案可以在成本和效率之间取得平衡，使其更具经济性和实用性。

- **变更操作是否需要实时生效？**

  确定事务的实时性要求至关重要：其变更操作是否必须在用户响应时间内完成，还是可以排队等待以异步方式处理。

以下为一些设计相关的次要考量因素，尽管这些因素会对整体架构产生一定影响，但对预算规划及实施便利性的影响更为显著。

- **数据库需要设计的规模是多少？**

  这会影响数据库服务器的大小。在处理非常大的数据库时，可能需要选择比工作负载要求更大的计算机。这是因为大型数据库的管理开销与数据库规模密切相关。随着表和索引的增长，为了在可接受的时间内完成表重组和索引构建，通常需要增加 CPU 资源。

  – 业务事务所需的吞吐量是多少？
  – 可用性的具体要求是什么？
  – 是否存在建立和管理应用程序的技术能力？
  – 由于预算限制，需要在哪些方面做出妥协？

2.3　应用程序设计原则

该部分描述了构建应用时可考虑的设计决策：

- 应用程序设计的简单性
- 数据建模
- 表和索引的设计
- 使用视图
- SQL 执行效率
- 实现应用程序
- 应用开发趋势

2.3.1　应用程序设计的简单性

应用程序与任何其他设计和工程产品并无本质区别。一般来说，如果一个设计看似合理，那么它很可能也是合理的。在构建应用程序时，始终应牢记这一原则。在设计过程中，需重点关注以下几点：

- 如果表的设计过于复杂，以至于无人能够完全理解其结构，那么该表可能设计得不够良好。
- 如果 SQL 语句异常冗长且涉及范围过大，导致任何优化器都无法实时有效地对其进行优化，则可能存在错误的语句、底层事务或表设计问题。
- 如果表上存在索引，但相同列被重复索引，那么可能是索引的设计存在问题。
- 如果提交的查询未能为在线用户提供快速响应，则可能反映出较差的用户界面设计或事务设计。
- 如果对数据库的调用被许多软件层从应用程序逻辑中过度抽象出来，则可能存在一种糟糕的软件开发方法。

2.3.2　数据建模

数据建模对于成功的关系型应用程序设计至关重要。必须以快速反映业务实践的方式执行此模型。关于正确的数据模型可能会引发激烈的争论。关键在于对那些受到最频繁的业务事务影响的实体进行建模。在建模阶段，一个常见的误区是过度专注于非核心数据元素的建模，这会导致开发周期被不必要的延长。使用建模工具可以快速生成模式定义，并且在需要快速原型设计时非常实用。

2.3.3　表和索引的设计

表设计在很大程度上是核心事务灵活性与性能之间的权衡。为了保持数据库的灵活性并能够适应未知的工作负载，表设计应尽可能贴近数据模型，并规范化至第三范式。然而，某些用户的核心事务可能需要出于性能考虑而选择性地进行非规范化。对于业务关键表，应优先投入资源以实现最佳性能表现；而对于非关键表，则可以采用更为简便

的设计方式，以加快应用程序的开发速度。但如果发现非核心表的原型设计和测试对性能造成了瓶颈，则需立即着手进行补救性设计工作。

索引设计同样是一个迭代的过程。可以从添加主键约束和在已知访问对象（如人名）上构建基础索引开始。随着应用程序的发展以及在实际数据量上的测试，可能需要通过优化索引来提升特定查询的性能表现。当为新需求构建索引时，请考虑以下索引设计原则：

- 将列追加到索引
- 使用其他索引类型
- 查找索引的成本
- 对索引中的列排序

2.3.3.1　将列追加到索引

加快查询速度的最简单方法之一是通过从执行计划中消除全表扫描来减少逻辑 I/O 的数量。这可以通过将查询中引用的所有列追加到索引来完成。这些列包括“选择列表”中的所有字段以及任何必需的联接或排序列。当减少了耗时的 I/O 操作时，此技术在加快在线应用程序的响应时间方面特别有用。因此，在第一次使用适当大小的数据测试应用程序时，最好采用这种方法以确保优化效果。

2.3.3.2　使用其他索引类型

有多种索引类型可供选择，每种类型的适用场景各不相同。以下是对常见索引类型的描述及其适用情况：

B-树索引：B-树索引是数据库中最常见的标准索引类型，适用于主键列以及需要频繁查询的字段。它特别适合用于连接操作，因为数据库可以利用这些索引快速检索按索引列排序的数据。

位图索引：位图索引通过将相同数据以元组 ID 位图的形式存储在磁盘上，具有占用空间小和查找速度快的特点。这种索引特别适合于那些没有更新操作且基数较低（例如 1 到 10,000）的数据列。

基于函数的索引：这种索引允许通过 B-树访问从原始数据中通过函数派生的值。尽管基于函数的索引在使用空值方面有一些限制，并且需要启用查询优化器，但它在某些情况下非常有用。

在查询组合列或克服由于数据在数据库中的存储方式带来的限制时，基于函数的索引特别有用。一个例子是查询订单中超过某个值的行，计算方式为（销售价格 - 折扣）x 数量。另一个例子是将 upper 函数应用于数据，以允许不区分大小写的搜索。

分区索引：分区全局索引允许在索引访问中进行分区修剪，从而减少 I/O 操作并提高查询性能。通过根据良好的范围定义或列表分区策略对索引进行分区，可以实现快速的索引扫描，进而缩短查询时间。

反向索引：反向索引旨在消除插入型应用程序中的索引热点问题。这种索引对于插入操作的性能表现非常出色，但由于其设计限制，无法用于范围扫描操作。

2.3.3.3　查找索引的成本

构建和维护索引结构可能会非常耗费资源，包括占用磁盘空间、CPU 以及 I/O 容量等。设计人员需要确保索引带来的性能提升能够抵消其维护所消耗的开销。

使用以下简单方法估算索引维护的成本：与在表上实际执行 DML 操作相比，插入、删除或更新索引所维护的键所需的资源大约是其三倍。因此，如果向包含三个索引的表中插入数据，则插入速度约为无索引表插入速度的十分之一。对于涉及大量插入操作的 DML，设计人员应当仔细评估索引设计方案，这可能需要在查询性能和插入性能之间做出权衡。

2.3.3.4　对索引中的列排序

设计者应灵活定义索引构建规则。根据具体情况，采用以下两种方法之一对索引中的键进行排序：

- 按列被选中的频率顺序排列

  这是最常用的方法，因为它能实现对所需元组的快速访问和最小 I/O 消耗。该技术主要用于主键和选择频率较高的范围扫描。

- 通过数据分组或排序以减少 I/O

  在大范围扫描中，通常可通过以下方式降低 I/O：利用低访问度列进行排序，或按检索顺序对列进行排序。

2.3.4　使用视图

视图能够加速并简化应用程序的设计流程。一个简单的视图定义可以屏蔽数据模型的复杂性，对于编程人员而言，重要的是实现数据的检索、显示、收集和存储功能。

然而，尽管视图提供了一个干净的编程接口，但它们可能会导致资源消耗较高的查询执行。最不理想的使用场景是当视图引用其他视图，或者在查询中进行联接操作时。在许多情况下，开发者可以选择直接从表中获取数据以满足查询需求，而不必依赖视图。通常，由于视图的固有特性，优化器难以生成最优的执行计划。

2.3.5　SQL 执行效率

在系统开发的设计和体系结构设计阶段，必须确保应用程序开发人员了解 SQL 的执行效率。为此，开发环境应具备以下关键特性：

- 数据库连接的有效管理

  建立数据库连接是一项资源消耗高且扩展性差的操作。因此，应尽可能减少并发连接数量。对于简单系统，理想情况下应在应用程序初始化时建立连接。然而，在基于 Web 或多层架构的应用中，应用服务器需处理多个用户的数据库连接，这增加了复杂性。针对此类应用程序，设计工作应确保数据库连接能够被共享，并避免为每个用户请求重新建立连接。

- 游标的有效使用与管理

维护用户连接对最小化系统上的解析活动至关重要。解析是指解释 SQL 语句并为其生成执行计划的过程，涉及多个阶段：语法检查、安全验证、执行计划生成以及将共享结构加载到共享池中。存在两种类型的解析操作：

- 硬解析

  当首次提交 SQL 语句且在共享池中未找到匹配项时，会使用硬解析。硬解析是最耗资源且不可扩展的操作，因为它需要执行所有解析相关步骤。

- 软解析

  当首次提交 SQL 语句且在共享池中找到匹配项时，会使用软解析。这种情况下，SQL 语句可被共享以提高性能。然而，软解析仍需进行语法和安全检查，这会占用系统资源。

为尽可能减少解析操作，应用程序开发人员应设计应用程序仅解析一次 SQL 语句并重复执行它们。这可通过游标实现。经验丰富的 SQL 开发者应熟悉打开和重新执行游标的概念。

此外，应用程序开发人员还必须确保 SQL 语句在共享池中被共享。为此，可采用绑定变量的方法来执行 SQL 语句，并避免使用字符串文本形式的 SQL 语句。

例如：

```
-- 构造数据环境:
CREATE TABLE employees (first_name VARCHAR,last_name VARCHAR);
-- 插入数据:
INSERT INTO employees VALUES('zhang','san'),('wang','wu'),('KING','base');
-- 使用字符串文本语句:
SELECT * FROM employees WHERE first_name LIKE 'KING';
-- 使用 bind 变量的语句:
-- 构造预备语句:
PREPARE em_plan(VARCHAR) AS SELECT * FROM employees WHERE first_name LIKE :1;
-- 执行:
EXECUTE em_plan('KING');
```

下面展示了一个简单的 OLTP 应用的测试结果：

```
Test                                #Users Supported
No Parsing all statements            270
Soft Parsing all statements          150
Hard Parsing all statements          60
Re-Connecting for each Transaction   30
```

该测试在 4 个 CPU 的机器上进行。随着系统上 CPU 数量的增加，差异也随之增加。

2.3.6　实现应用程序

开发环境和编程语言的选择在很大程度上受到开发团队技能和架构决策的影响。然而，有一些简单的性能管理规则可以帮助构建可扩展且高性能的应用程序。

1. 选择一个能够支持软件组件性能设计的开发环境，避免因环境限制而导致性能决策失误。如果选择了不合适的语言或环境，则可能需要重新评估选择。

   - 用户界面

     编程模型在 HTML 生成和直接调用窗口系统之间存在差异，因此开发方法应重点关注用户界面代码的响应时间。如果使用网络传输 HTML 或 Java，请尽量减少网络交互的开销。

   - 业务逻辑

     解释型语言（如 Java 和 PL/SQL）非常适合编码业务逻辑，因为它们具有良好的可移植性和易于升级的特点。这些语言的语法丰富，使得代码易于阅读和解释。如果业务逻辑涉及复杂的数学运算，则可能需要使用编译型二进制语言。业务逻辑可以分布在客户机、应用服务器或数据库服务器上，而应用服务器通常是实现业务逻辑的主要场所。

   - 用户请求和资源分配

     这部分通常不受编程语言的影响，但选择能够抽象数据库连接和游标管理的工具或第四代语言时，需注意其可能采用低效机制。在评估这些工具和环境时，请重点关注它们的数据库连接模型以及对游标和绑定变量的处理方式。

   - 数据管理及事务

     这部分通常不受编程语言的影响。

2. 当实现一个软件组件时，专注于其实现的功能，而不是与其他组件相关联的功能。实现另一个组件的功能会导致设计和实现上的不足。这适用于所有组件。

3. 不应忽视任何功能的实现，也不应在设计、实现或测试阶段对软件组件进行研究。许多情况下，只有在应用程序实际发布或经过测试后，才能发现这些疏漏。这通常是由于体系结构设计或初始系统规范不完善所导致的。值得注意的是，在最初的系统设计、构建和实现过程中，数据存档和清除模块是最常被忽略的部分。

4. 在实现过程逻辑时，应使用过程语言（如 C、Java 或 PL/SQL）进行开发。对于数据访问（查询）或数据更改（DML）操作，请使用 SQL 完成。此规则适用于业务逻辑模块的代码实现，其中过程代码与数据访问（非过程化 SQL）代码混合在一起的情况。需要注意的是，在 SQL 访问中嵌入过程逻辑虽然看似理所当然，但这种做法往往会生成资源密集型的 SQL 语句，进而影响系统性能。因此，带有 decode 函数的 SQL 语句通常被认为是优化的重点对象，同时那些包含大量 OR 谓词或集合运算符（如 union 和 minus）的语句也常常需要进行优化处理。

5. 对于那些经常被访问但很少更改的数据，反复检索的开销往往非常高昂。因此，在设计和实现缓存机制时，需要同时满足两个关键要求：首先，该机制必须便于开发人员使用；其次，其性能表现必须优于直接从原始数据源进行访问。这一原则适用于系统中的所有模块。对于那些频繁使用的数据值，应优先选择在本地进行缓存或存储，避免因多次从远程或高成本数据源中检索而带来额外开销。

   本地缓存候选的最常见示例包括：

   - 当日的日期。SELECT SYSDATE FROM DUAL 可以占数据库工作负载的 60% 以上
   - 当前用户名
   - 重复的应用程序变量和常量，如税率、折扣率或位置信息

- 本地缓存数据可以进一步扩展到将本地数据缓存构建到应用服务器中间层。这有助于减轻中央数据库服务器的负载。但是，在构建本地缓存时应小心，以免它们变得更加复杂，以至于不再提供性能增益
- 本地序列的生成，应该考虑使用缓存的设计影响。例如，如果用户在午夜连接并且缓存了日期，则用户的日期值将变为无效。

6. 优化组件之间的接口，并确保所有组件在高可扩展性的环境下能够高效运行和交互；此规则要求最小化解析时间，适用于所有模块及其接口。
7. 利用数据库的外键约束功能。通过应用程序强制执行引用完整性会带来较高的开发和维护成本。可以通过在父表中选择子列的值并确保其存在的机制来维护外键引用。
8. 建议在应用程序中配置操作名称和模块标识符，以便更好地支持端到端应用程序的跟踪功能。这为更高效地排查和分析潜在问题提供了更大的灵活性。

2.3.7　应用开发趋势

当今应用程序开发面临两大主要挑战：一是用 Java 替代 C 或 C++ 等编译型语言的应用程序，二是日益普及的面向对象技术对模式设计的影响。

尽管 Java 为程序员提供了更好的代码可移植性和可用性，但其性能方面存在显著挑战。由于 Java 是一种解释型语言，在执行相同逻辑时通常比编译型语言（如 C）慢。这导致客户端计算机资源消耗增加，需要更强大的 CPU 支持，并要求开发者更加注重编写高效的代码。

作为面向对象语言，Java 鼓励将数据访问逻辑与业务逻辑分离到不同的类中。然而，这种做法使得程序员在调用数据访问方法时，往往不了解其效率和性能影响。结果是过度依赖数据库查询，使用简单而低效的接口进行数据操作。

在当前的设计实践中，所设计的查询通常未包含所有有效的 WHERE 条件，导致行筛选在 Java 代码层面执行，这严重影响了系统效率。此外，在处理 DML 操作时，尤其是 INSERT 语句，程序往往采用单条记录插入方式，无法利用数据库提供的数组接口，降低了批量处理效率。在某些情况下，过程调用甚至会进一步降低性能。

一般来说，将数据访问逻辑与业务逻辑紧密结合，能够实现更高效的事务设计。

在编程层面，面向对象技术已被用于在 KingbaseES 服务器中创建面向对象的数据库系统。这体现在多个方面：例如，BLOB 字段存储复杂对象结构，仅利用数据库作为索引卡文件，或是通过 KingbaseES 的对象关系特性进行开发。

然而，在选择面向对象方式进行模式设计时，必须确保保留关系型存储模型的灵活性和优势。许多情况下，面向对象的设计方法最终会导致高度非规范化数据结构，增加维护成本，并依赖大量与对象关联的引用指针（如 REF）。这种做法往往会导致模式设计回归到层次化或网状数据库的理念，而这已被现代的关系型数据库方法所取代。

总的来说，在需要长期数据存储以及预期模式具备一定自我适应能力的应用场景下，关系型存储方法通常能提供最优的性能和灵活性。

2.4　工作负载测试，建模及实施

本节描述工作负载估计、建模、实现和测试。本节包括以下主题：

- 确定数据大小
- 估计负载量
- 应用建模
- 测试，调试，验证设计

2.4.1　确定数据大小

在使用较差的样本集时，在处理可变长度数据的过程中可能会出现大小估计错误的问题。随着数据量的增长，键的长度可能会显著增加，从而改变对列大小的假设。

当系统开始运行时，预测数据库增长变得更加具有挑战性，特别是在处理索引时。表随着时间的推移而扩展，索引的变化在密钥生成、插入模式和行删除方面会受到应用程序行为的影响。最糟糕的情况是使用升序键进行插入操作，然后从左侧删除大多数行，但并非所有行。这种操作会导致空白区域和空间浪费。如果有此类索引的使用场景，建议确保熟悉并能够利用联机索引重建工具。

数据库管理人员应持续监控每个对象的空间分配情况，并识别可能失控的对象。通过深入理解应用程序的行为，可以识别出那些可能快速增长或难以预测的对象类型。这是任何系统性能和可用性规划中的关键部分。在设计生产数据库时，应尽量确保在交互用户使用应用程序的过程中实现最小化空间管理目标。这一原则适用于所有数据、临时对象和回滚段。

2.4.2　估计负载量

由于涉及的变量数量庞大，用于容量规划和测试的工作负载估计非常具有挑战性。然而，设计人员必须明确指定计算机配置（包括 CPU、内存和磁盘驱动器），并最终推出应用程序。在这里，我们提出了几种方法，每种方法都有其独特的优势。在确定规模时，建议使用以下方法来验证决策过程，并提供相应的支持文档以确保实施的准确性。

- **从类似系统推断**

  这是一种完全基于经验的方法，其中现有的类似特征和已知性能的系统被用作基础系统的参考。随后，相关专家会根据已知的差异对基础系统的规格进行调整。这种方法的优点在于其与现有系统的关联性较强，但在处理差异时却难以提供具体帮助。在准备大型工程项目的成本估算（例如大型建筑、船舶、桥梁或石油钻机等）时，几乎所有大型工程学科都采用这种方法。然而，如果参考系统的规模与预期系统存在一个数量级的差距，则某些部件可能会超出其设计极限，从而导致潜在的问题。

- **基准测试**

  基准测试过程既耗费资源又耗时，而且未必能得出正确的结果。通过在应用程序的早期开发阶段或以原型形式模拟应用程序，有可能会发现与实际生产系统不相符的情况，这看起来似乎有些奇怪。然而，在 KingbaseES 多年来为数据库开发组织进行客户应用程序基准测试的过程中，尚未观察到基准

测试应用程序与实际生产系统的可靠关联性。这主要是因为在开发过程中引入了大量应用程序特性，导致效率低下。

然而，目前基准测试已成功应用于确定系统的规模，可达到令人满意的精度水平。特别是，基准测试非常擅长确定实际的 I/O 需求，并在系统满负荷运行时验证恢复过程的能力。

基准测试从其本质出发，强调所有系统组件的极限。当基准测试涉及所有组件时，由于应用程序在设计和实现过程中会存在一定的缺陷，因此在基准测试时必然会暴露出这些问题。此外，基准测试还可以对数据库、操作系统以及硬件组件进行全面评估。然而，由于大多数基准测试都以快速执行的方式进行，所以在某个系统组件出现问题时，预计会遇到瓶颈和问题。需要注意的是，基准测试是一项具有高强度负载压力的任务，想要从基准测试中获得最大收益，需要具备相当丰富的经验。

2.4.3　应用建模

无论是复杂的数学建模练习，还是在信封背面进行的经典简单计算，对应用的建模都是适用的。不同的建模方法都有其优点，有的方法试图非常精确地建模，而有的方法则进行粗略估计。然而，这些建模方法的一个共同缺点是它们都不允许实现错误和低效率。

估计和确定数据大小的过程是一门不精确的科学。然而，通过研究这个过程，可以进行一些智能的估算。整个估计过程中，我们通常不会考虑到由糟糕的 SQL、索引设计或游标管理导致的应用程序效率低下问题。一个规模测定工程师应该为应用程序可能存在的低效性留出空间，而一个性能调优工程师则应发现这些低效之处，并使评估更加切合实际。KingbaseES 的性能方法详细描述了如何识别和处理应用程序中的低效性。

2.4.4　测试，调试，验证设计

测试过程主要包括功能性测试和稳定性测试。在流程的某个阶段应执行性能测试。

以下是用于应用程序性能测试的一些简单规则列表。如果正确记录，此列表将为生产应用程序和容量规划过程提供重要信息。

1. **使用真实的数据量和分布进行测试**

   所有测试必须在完全填充的数据表中完成。测试数据库应包含代表生产系统的数据，包括数据量和表基数。所有生产索引应构建，并正确填充模式统计信息。

2. **测试单个用户的性能**

   在空闲或轻载的数据库上测试单用户性能以实现可接受的性能。如果一个用户在理想条件下无法达到可接受的性能，则多个用户在实际条件下也将无法达到可接受的性能。

3. **获取并记录所有 SQL 语句的计划**

   为每个 SQL 语句获取其执行计划。此过程可以验证优化器是否获得了最佳执行计划，并通过 CPU 时间和物理 I/O 了解 SQL 语句的相对成本。这有助于识别将来需要进行最多调整和性能工作的事务。

4. **尝试多用户测试**

精确执行此过程具有挑战性，因为用户负载和配置文件可能无法完全量化。然而，应在交易中执行 DML 语句以测试是否存在锁定冲突或序列化问题。

5. **使用正确的硬件配置进行测试**

尽可能使用接近生产的配置进行测试。对于网络延迟、I/O 子系统带宽、处理器类型和速度，特别重要的是使用真实系统的设置。如果不采用这种方法，可能会导致对潜在性能问题的错误分析。

6. **测量稳态性能**

在基准测试期间，测量稳定状态下的性能至关重要。每个基准测试运行都应包括一个预热阶段，在此期间用户连接到应用程序并逐渐开始执行工作。这使频繁访问的数据能够初始化到缓存中，并完成解析等一次性执行操作在达到稳态之前。类似地，在基准测试运行结束时，应该有一个缓冲期，在这段时间内系统释放资源，用户停止工作并断开连接。

第3章　性能调优工具

本章节包含以下内容：

- 性能调优工具概述
- kbbadger 日志分析工具
- 动态性能视图
- KWR 报告
- KDDM 报告
- KWR DIFF 报告
- KSH 报告

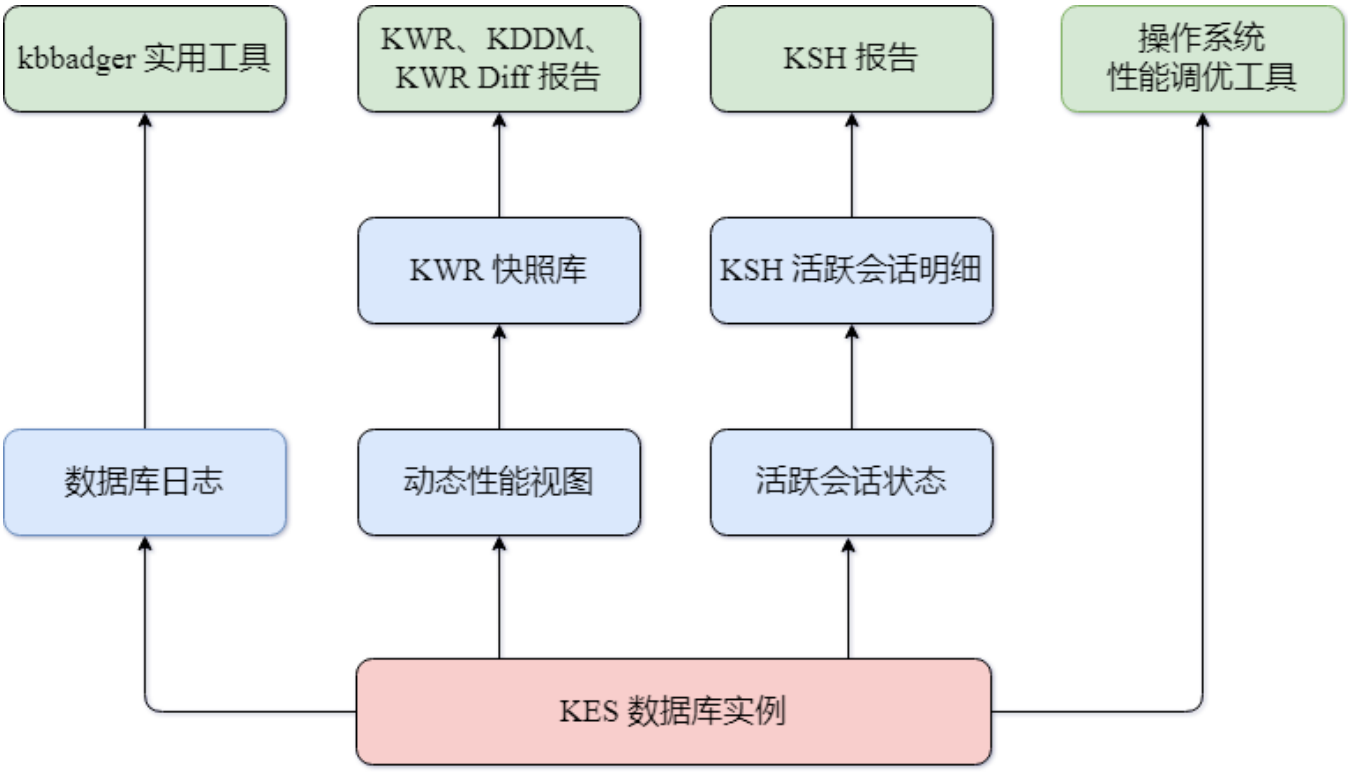
3.1　性能调优工具概述

KingbaseES 数据库在运行期间会产生大量日志文件，kbbadger 实用工具基于这些日志文件进行系统性能指标的深入分析。能够生成多种可视化性能报告，其中包括总体统计信息、占用时间最多的查询详情以及查询时间直方图。

当 KingbaseES 数据库的统计功能被启用时，系统会生成大量的统计信息。这些统计信息以动态性能视图的形式呈现，为数据库性能分析提供了有力支持。通过这些动态性能视图，可以有效分析、评估数据库的整体性能。例如，通过 sys_stat_statements 视图可以查看消耗资源最多的 TOP SQL，通过 sys_stat_sqltime 和 sys_stat_sqlwait 这两个视图可以查看 SQL 报文时间和 SQL 等待时间的相关数据，通过 sys_stat_dbtime、sys_stat_wait 及 sys_stat_sql 这些数据库时间模型动态性能视图可以查看与数据库时间模式相关的性能信息。

当 KWR 自动快照功能被启用时，数据库后台进程 kwr collector 将按照设定的时间间隔（默认为每小时）自动采集动态性能视图的相关数据，并将采集到的性能数据以 KWR 快照表的形式存储到 KWR 快照数据库中。DBA 可基于 KWR 快照数据生成 KWR 报告、KDDM 报告、KWR DIFF 报告。同时，KSH 使用采样式统计方法实时收集数据库的活跃会话状态信息，主要是由后台进程 ksh collector 和 ksh writer 完成，并基于采样的活跃会话历史明细生成 KSH 报告。

此外，KingbaseES 的性能调优还可借助多种操作系统级别的工具来实现，其中包括火焰图、TOP、Free 和 IO-Stat 等常用工具。关于这些工具的详细使用方法，请参阅手册中的《操作系统资源分析》章节。



本章节旨在全面介绍 KingbaseES 的性能调优工具及其使用方法，以及一些关键概念（例如数据库时间模型等）。这些性能调优工具包括：

1. kbbadger 实用工具：基于数据库日志的性能分析工具
2. 动态性能视图：反映数据库实例内部状态，比如 SQL 执行、等待事件、IO 等性能指标相关的视图
3. KWR 快照：基于动态性能视图生成的自动负载信息库
4. KWR 报告：基于 KWR 快照和数据库时间模型生成的性能报告
5. KDDM 报告：基于 KWR 快照和数据库时间模型生成的性能诊断和建议报告
6. KWR DIFF 报告：基于两组 KWR 快照的生成的性能指标对比报告
7. KSH 报告：基于活跃会话历史明细生成的性能报告

3.2　kbbadger 日志分析工具

3.2.1　工具概述

kbbadger 是一个命令行工具，可以分析 KingbaseES 在运行过程中产生的大型日志文件，并以图形化报告的形式展示数据库性能指标。

生成日志的内容可以通过 GUC 参数比如 log_min_duration_statement 等进行控制。主要分析的统计有：连接、会话、检查点、临时文件、回收（VACUUM）、锁、查询、事件等。

支持多种日志文件格式，如 syslog，stderr，csv log 和 json log 等。生成的所有图表都是可缩放的并且可单独下载为 PNG 文件。另外，在生成的报告中 SQL 查询将突出显示。

3.2.2　基本能力

kbbadger 生成的报告中包含的信息有：

- 总体统计
- 占用时间最多的查询
- 最常见的查询
- 最常见的错误
- 查询时间直方图
- 会话时间直方图
- 顶级查询的用户
- 顶级查询的应用程序
- 最多取消的查询
- 最耗时的准备/绑定查询

以下是主要耗时的查询的示例报告：

□ 主要耗时查询

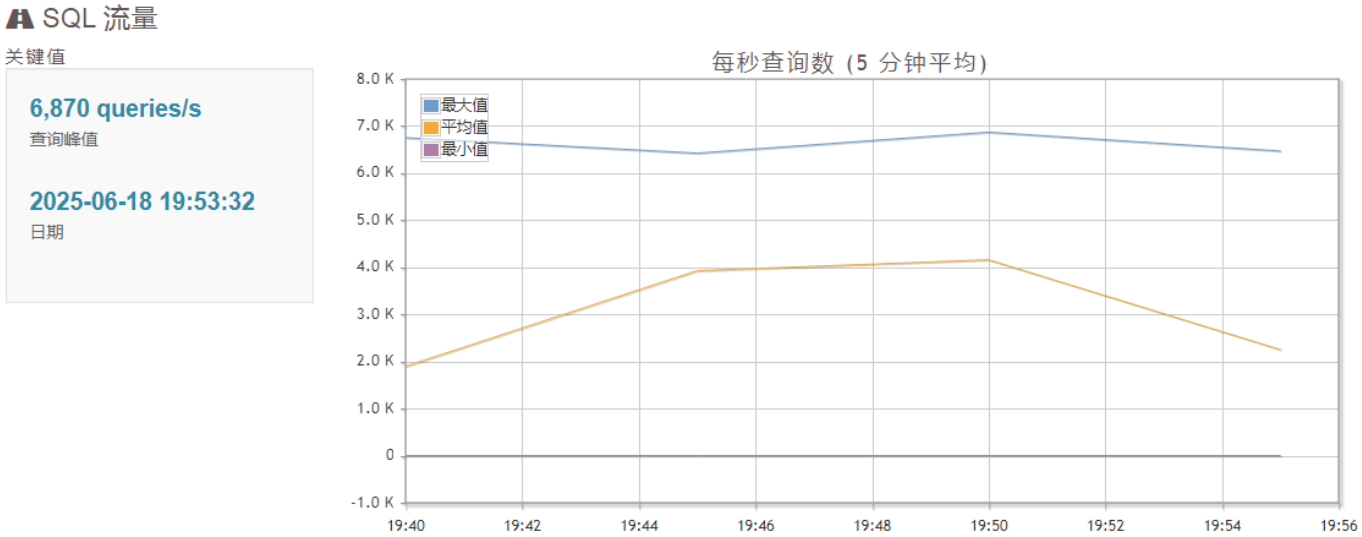
排名	总持续时间	执行次数	最小持续时间	最大持续时间	平均持续时间	查询
1	1h39m19s	69,553 细节	0ms	1s830ms	85ms	□ SELECT d_tax, d_next_o_id, ctid, xmin FROM bmsql_district WHERE d_w_id = '' AND d_id = '' FOR UPDATE; 示例 执行用户
2	1h36m20s	66,439 细节	0ms	2s459ms	87ms	□ UPDATE bmsql_district SET d_ytd = d_ytd + '' WHERE d_w_id = '' AND d_id = ''; 示例 执行用户
3	27m29s	66,439 细节	0ms	929ms	24ms	□ UPDATE bmsql_warehouse SET w_ytd = w_ytd + '' WHERE w_id = ''; 示例 执行用户
4	11m47s	64,963 细节	0ms	788ms	10ms	□ DELETE FROM bmsql_new_order WHERE no_w_id = '' AND no_d_id = '' AND no_o_id = ''; 示例 执行用户
5	5m34s	695,356 细节	0ms	886ms	0ms	□ SELECT s_quantity, s_data, s_dist_01, s_dist_02, s_dist_03, s_dist_04, s_dist_05, s_dist_06, s_dist_07, s_dist_08, s_dist_09, s_dist_10, ctid, xmin FROM bmsql_stock WHERE s_w_id = '' AND s_i_id = '' FOR UPDATE; 示例 执行用户

报告也会提供每小时的统计图表，其内容包括：

- SQL 查询统计信息

- 临时文件统计
- 检查点统计
- 自动 VACUUM 和自动分析统计
- 已取消查询
- 错误事件（死机、致命、错误和警告）
- 错误的类分布

以下是按 5 分钟粒度分割的 SQL 流量统计的示例：



3.2.3　使用示例

kbbadger 依赖数据库日志系统，需要在 kingbase.conf 中配置好相关参数：

```
# 启用日志记录
log_min_duration_statement = 0
# 用于 kbbadger 的默认解析格式
log_line_prefix = '%t [%p]: [%l-1] user=%u,db=%d,remote=%h '
# 日志记录的信息
log_checkpoints = on
log_connections = on
log_disconnections = on
log_lock_waits = on
log_temp_files = 0
log_autovacuum_min_duration = 0
log_error_verbosity = default
```

运行 TPCC（100 数据仓库 100 并发）大约 23 分钟，产生了 6.3 GB 日志：

```
$ du -sh sys_log
6.3G    sys_log
```

生成 kbbadger 报告：

```
$ kbbadger ./sys_log/kingbase* -f stderr -J 12 -j 32
[=========================>] Parsed 6704882201 bytes of 6704882201 (100.00%), queries: 7722936, events: 803
LOG: Ok, generating html report...
```

kbbadger 会在当前目录下产生 out.html 报告。

3.2.4　适用场景

kbbadger 是一款基于数据库日志系统的专业工具，在多种环境下均可直接运行。生成的报告以网页形式呈现，内容丰富且易于阅读。

当其他性能调优工具无法使用时，比如因为较低 KingbaseES 版本不支持 KWR 的场景，kbbadger 可作为有效的替代方案来分析大部分性能问题。

由于日志系统捕获的信息量较大，sys_log 目录的磁盘空间消耗也会显著增加。

此外，大量日志文件的生成会对数据库服务器的性能造成较大影响，同时分析日志的时间也可能相对较长。在完成性能分析后，需要及时关闭相关的日志跟踪功能。

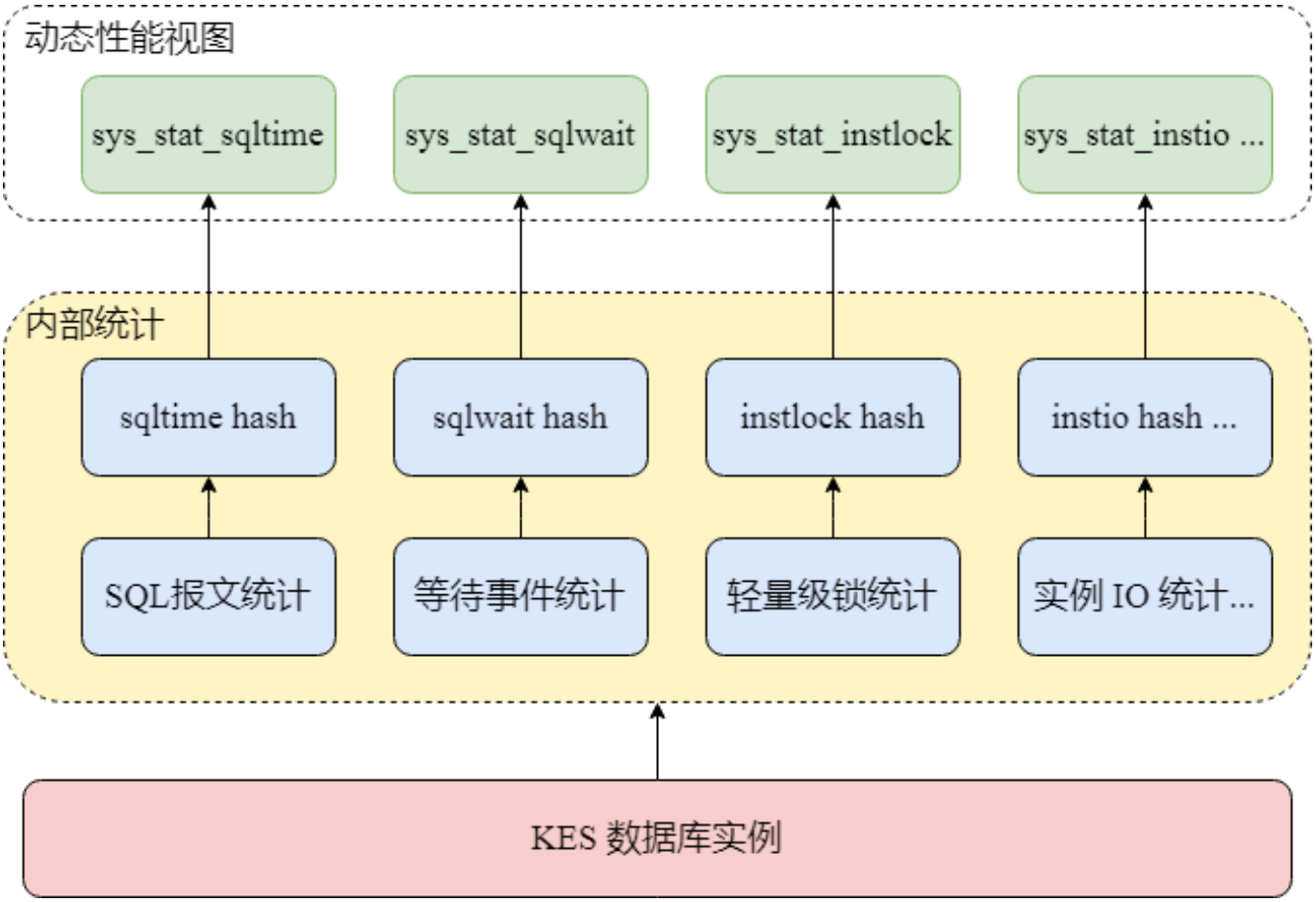
3.3　动态性能视图

本章节包含以下内容：

- 动态性能视图概述
- 动态性能视图分类
- 数据库实时状态相关视图
- 数据库实例统计相关视图
- 数据库对象统计相关视图
- 数据库时间相关视图
- TOP SQL 相关视图
- 主机信息相关视图
- 管理动态性能视图

3.3.1　动态性能视图概述

数据库运行过程中生成的大量统计信息，包括等待事件、IO 操作和 SQL 语句执行等数据，这些信息以动态性能视图的形式提供查询能力。其特点是它们并不以传统的关系表形式存储在数据库内部，而是采用哈希表或动态数组等高效的数据结构存在于数据库共享内存或者本地内存。通过设置相关 GUC 参数，可以控制要采集的动态性能视图内容，并通过 SQL 函数去管理这些动态性能视图。



例如，sys_stat_dmlcount 动态性能视图展示 DML 语句执行次数和时间：

```
kingbase=# SELECT * FROM sys_stat_dmlcount;
 datid | sql_type | background |  calls   |    times
-------+----------+------------+----------+-------------
 16384 | Select   | f          | 26824599 | 23767043376
 14731 | Select   | f          |       43 |       28707
 16384 | Insert   | t          |      405 |      766144
 16384 | Select   | t          |      397 |     1262721
 16384 | Update   | f          | 15485156 |  8386113705
 16384 | Insert   | f          | 62133982 | 16855029570
 16384 | Delete   | f          |   837204 |   524083952
(7 行记录)
```

动态性能视图不支持像普通关系表可以直接修改数据内容，其数据只能通过提供的 SQL 函数进行重置或清理操作。

```
kingbase=# DELETE sys_stat_statements_all;
ERROR:  cannot delete from view "sys_stat_statements_all" ...
kingbase=# SELECT sys_stat_statements_reset();
 sys_stat_statements_reset
---------------------------


(1 row)
```

3.3.2　动态性能视图分类

动态性能视图从统计范围上可以分：

1、实例级统计

实例级统计是指这些统计结果覆盖整个数据库实例，例如事务数统计（sys_stat_transaction）和实例 IO 统计（sys_stat_instio）等。在所有数据库上查询这些动态性能视图，显示的都是同一组统计信息。

大多数统计都是实例级统计。

实例级统计的清理可以执行命令：`SELECT sys_stat_reset_shared('all')`。

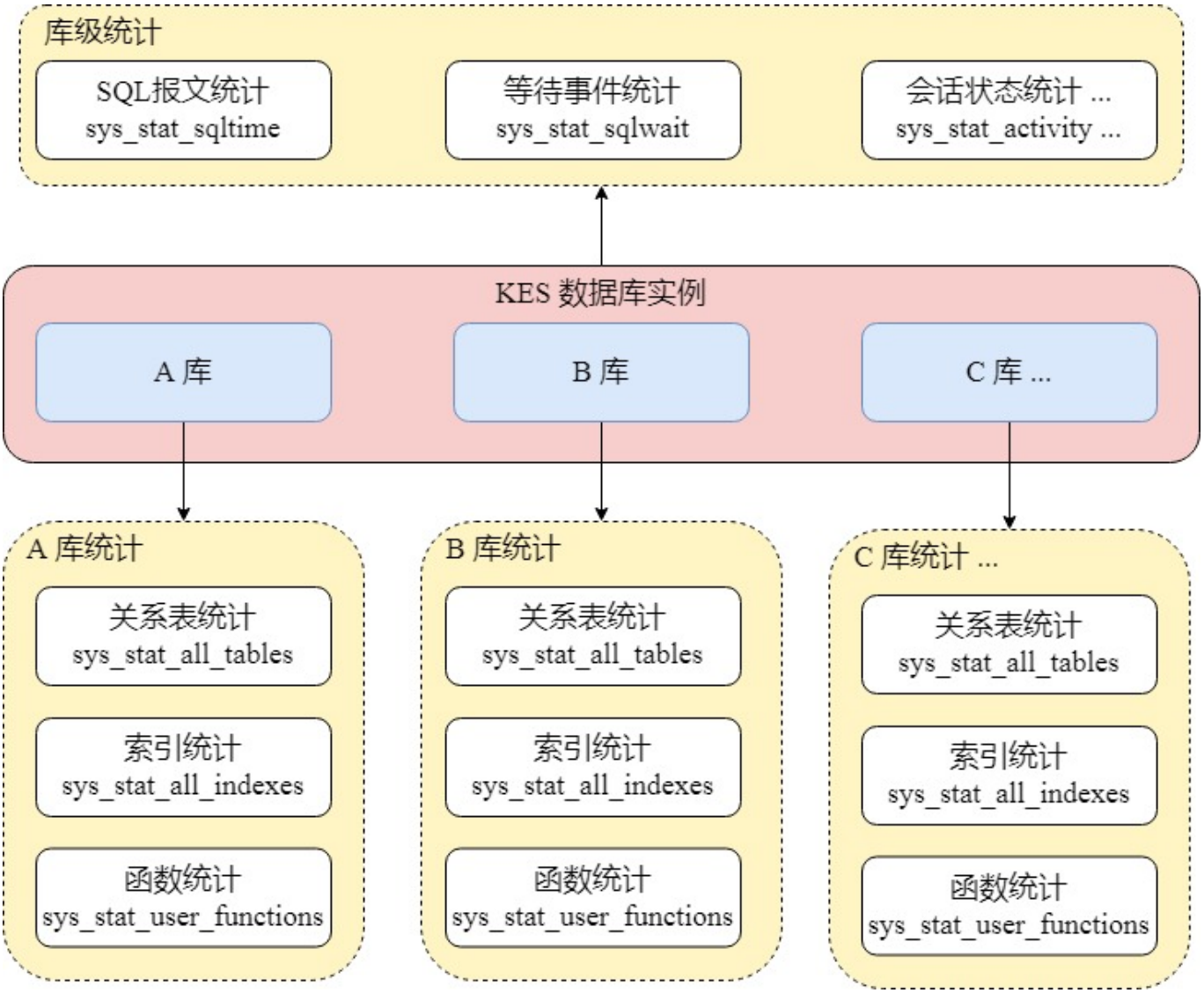
2、当前库的统计

当前库统计主要关注的是数据库对象的统计信息。数据库对象主要包括关系表、索引以及用户自定义函数等。

这部分统计信息无法跨数据库访问。例如，在 kingbase 库上只能查询该库的关系表统计，无法查看 benchmarksql 库的相关关系表统计数据。

清理当前库的数据库对象统计信息可以执行命令：`SELECT sys_stat_reset()`。

也可以清理当前库上某个数据库对象的统计信息。例如，要清理某个特定 OID 关系表的统计信息可以执行命令：`SELECT sys_stat_reset_single_table_counters(OID)`。



动态性能视图从统计时效上分：

1、实时状态的动态性能视图

实时状态统计反映 KingbaseES 数据库运行中的会话、执行的任务等的最新状态，统计结果实时更新。例如，sys_stat_activity 反映了当前客户端会话和后台工作进程状态信息。

```
kingbase=# SELECT pid, wait_event, state, substr(query,0,20) as query, backend_type FROM sys_stat_activity;
  pid   |     wait_event      | state  |        query         |         backend_type
--------+---------------------+--------+----------------------+------------------------------
  99491 | KshMain             | idle   |                      | ksh collector
  99489 | KwrMain             | idle   | select perf.snapshot | kwr collector
  99492 | LogicalLauncherMain |        |                      | logical replication launcher
  99486 | AutoVacuumMain      |        |                      | autovacuum launcher
  99490 | KshMain             | idle   | select perf.ksh_time | ksh writer
  99932 |                     | active | SELECT pid,wait_even | client backend
```

```
 191334 | SysSleep            | active | SELECT sys_sleep(10) | client backend
  99484 | BgWriterHibernate   |        |                      | background writer
  99483 | CheckpointerMain    |        |                      | checkpointer
  99485 | WalWriterMain       |        |                      | walwriter
(10 行记录)
```

从以上查询结果中可以看到 pid 是 191334 的客户端会话进程（client backend），当前正在执行语句 SELECT sys_sleep(10)，当前的状态是 active（活跃），当前等待事件是 SysSleep。其他的会话状态信息还包括：数据库 OID、数据库名、用户 OID、用户名、客户端连接信息、事务开始时间、查询开始时间、状态改变时间、等待事件、事务 ID 等。

备注:

实时状态统计一般没有清理接口。

2、**累积状态的动态性能视图**

累积式统计主要指的是数据库服务器在运行过程中不断积累的统计数据。这些统计值通常会在服务器关闭时被存储到特定的统计文件中，并在下一次启动时加载至统计进程中。需要注意的是，除非进行了重置（reset）操作，否则这类统计会持续以累积的方式进行记录和更新。

比如记录每个数据库统计值的 sys_stat_database：

```
kingbase=# SELECT datid, datname, numbackends, xact_commit, xact_rollback, tup_inserted, tup_updated, tup_
deleted FROM sys_stat_database WHERE datname = 'kingbase';
 datid | datname  | numbackends | xact_commit | xact_rollback | tup_inserted | tup_updated | tup_deleted
-------+----------+-------------+-------------+---------------+--------------+-------------+-------------
 14704 | kingbase |           4 |        5757 |             6 |        60409 |        3453 |       26334
(1 行记录)
```

累积式动态性能视图涵盖了非常广泛的统计对象，主要可分为以下几大类：

1. 数据库实例相关统计
2. 数据库对象相关统计
3. 数据库时间模型相关统计
4. TOP SQL 相关统计
5. 主机信息统计

3.3.3　数据库实时状态相关视图

数据库实时状态动态性能视图主要包括：

视图名称	说明
sys_stat_activity	每个服务器进程对应一行记录，展示了该进程当前活动的相关信息，包括状态、等待事件、正在执行的查询语句等
sys_stat_gssapi	每个连接（常规和复制）有一行，显示关于 GSSAPI 验证和加密的信息
sys_stat_ssl	每个连接（常规的或者复制）一行，显示在这个连接上使用的 SSL 的信息
sys_stat_subscription	每个订阅至少一行，显示有关该订阅的工作者的信息
sys_stat_progress_checkpoint	每个运行 CHECKPOINT 的后端对应一行记录，显示当前进度
sys_stat_progress_cluster	每个运行 CLUSTER 或 VACUUM FULL 的后端对应一行记录，显示当前进度
sys_stat_progress_create_index	每个后台运行 CREATE INDEX 或 REINDEX 的后端都有一行，显示当前的进度
sys_stat_progress_vacuum	每个运行着 VACUUM 的后端（包括 autovacuum 工作者进程）一行，显示当前的进度
sys_stat_replication	每一个 WAL 发送进程一行，显示有关到该发送进程连接的后备服务器的复制的统计信息
sys_stat_wal_buffer	统计 WAL BUFFER 实时运作情况
sys_stat_wal_receiver	只有一行，显示来自 WAL 接收器所连接服务器的有关该接收器的统计信息

上述动态性能视图涉及的 GUC 参数：

- track_activities
  - 跟踪 sys_stat_activity 视图，包括等待事件、事务状态等指标，默认启用
  - 跟踪 sys_stat_progress_xxx 查询创建索引等长耗时动作进度统计
- track_activity_query_size
  - 限制 sys_stat_activity 视图中 query 字段的最大长度，默认 1024
  - 当 query 字段长度超出限制时，保留 track_activity_query_size-1 个字符
  - 该参数不影响 sys_stat_statements 视图中 query 的长度
  - 修改该参数，需要重启服务器后才生效

此外，还有一类实时状态视图：实时 TPS 和 QPS 动态性能视图。

实时 TPS 和 QPS 动态性能视图展示了过去一小时内的每 15 秒时间段内的平均 TPS 和 QPS。相较于使用 tps() 和 qps() 函数查询时需要的 1 秒等待时间，用户可以即时获取这些动态性能数据。

实时 TPS 和 QPS 相关视图包括：

视图名称	说明
sys_stat_metric_name	实时统计项名称
sys_stat_metric_group	实时统计项分组
sys_stat_metric	实时统计结果，目前只支持 TPS 和 QPS
sys_stat_metric_history	实时统计最近一个小时的数据
sys_stat_sysmetric	同 sys_stat_metric，仅展示系统级统计
sys_stat_sysmetric_history	同 sys_stat_metric_history，仅展示系统级统计
sys_stat_sysmetric_summary	系统级实时统计汇总结果，包含多种关键指标，如最大值、最小值和平均值等

上述视图的相关 GUC 参数设置：

- track_real_stats
  - 跟踪实时 TPS 和 QPS 统计，默认 off

信息:

实时 TPS 和 QPS 统计功能依赖于 track_counts 开关获取底层数据支持

关于上述 GUC 参数的具体含义和配置说明，请参考《配置管理手册》中运行时统计数据章节。

3.3.4　数据库实例统计相关视图

数据库实例统计基于全实例级别进行，在所有数据库上查询这些动态性能视图，显示的都是同一组统计信息。

数据库实例的统计信息分为实时统计和累积式统计两类。例如，提供实时数据的 sys_stat_activity 和累积式统计的 sys_stat_bgwriter：

```
kingbase=# SELECT checkpoints_timed, checkpoint_write_time, checkpoint_sync_time, buffers_checkpoint,
buffers_backend FROM sys_stat_bgwriter;
 checkpoints_timed | checkpoint_write_time | checkpoint_sync_time | buffers_checkpoint | buffers_backend
-------------------+-----------------------+----------------------+--------------------+-----------------
                 9 |                685308 |                 3443 |              34170 |         4417599
(1 行记录)
```

以及数据库实例 SQL 语句数统计 sys_stat_sqlcount：

```
kingbase=# SELECT datid, sql_type, background, calls, times FROM sys_stat_sqlcount;
 datid | sql_type  | background |  calls   |   times
-------+-----------+------------+----------+------------
 14785 | DML       | t          |     3484 |   18761142
 14785 | DCL       | f          |        2 |       1568
 14784 | DML       | f          | 13649873 | 2661547505
 14785 | DML       | f          |       76 |     408783
 14785 | DDL       | t          |        3 |     542682
 14785 | DDL       | f          |        1 |       2739
 14785 | Other SQL | t          |       10 |       2352
 14785 | Other SQL | f          |       20 |        191
 14784 | DDL       | f          |       23 |    6871051
 14784 | TCL       | f          |   641742 |    4362233
(10 行记录)
```

数据库实例相关视图包括：

视图名称	说明
sys_stat_archiver	只有一行，显示有关 WAL 归档进程活动的统计信息
sys_stat_bgwriter	只有一行，显示有关后台写进程的活动的统计信息
sys_stat_database	每个数据库一行，显示数据库范围的统计信息
sys_stat_database_conflicts	每个数据库一行，显示数据库范围的统计信息，这些信息的内容是关于由于与后备服务器的恢复过程发生冲突而被取消的查询
sys_stat_sqlcount	数据库实例的 SQL 语句统计
sys_stat_dmlcount	数据库实例的 DML SQL 语句统计
sys_stat_transaction	数据库实例的事务数统计
sys_stat_shmem	数据库实例共享内存分配情况统计
sys_stat_cached_plans	数据库实例执行计划缓存情况统计
sys_stat_pre_archivewal	统计所有预归档 wal 日志信息

信息:

这部分动态性能视图没有 GUC 参数控制，其统计数据一直刷新。

3.3.5　数据库对象统计相关视图

数据库对象统计是针对当前数据库实例中的数据库对象（包括关系表、索引、用户自定义函数等）的统计。需要注意的是，KingbaseES 系统无法跨数据库访问其他数据库上的数据库对象。

例如，sys_stat_user_tables 记录的是当前数据库实例中的所有用户表的统计信息：

```
kingbase=# SELECT relid, seq_scan, idx_scan, n_tup_ins, n_tup_upd, n_tup_del FROM sys_stat_user_tables WHERE
relname = 'ksh_statements';
 relid | seq_scan | idx_scan | n_tup_ins | n_tup_upd | n_tup_del
-------+----------+----------+-----------+-----------+-----------
 17052 |        3 |    38592 |      2022 |         0 |      1292
(1 行记录)
```

使用 sys_stat_reset() 函数，可以清理当前库上全部统计信息：

```
kingbase=# SELECT relid, seq_scan, idx_scan, n_tup_ins, n_tup_upd, n_tup_del FROM sys_stat_user_tables WHERE
relname = 'ksh_statements';
 relid | seq_scan | idx_scan | n_tup_ins | n_tup_upd | n_tup_del
-------+----------+----------+-----------+-----------+-----------
 17052 |        0 |        0 |         0 |         0 |         0
(1 行记录)
```

数据库对象相关视图包括：

视图名称	说明
sys_stat_all_tables	当前数据库中每个表一行，显示有关访问指定表的统计信息
sys_stat_sys_tables	和 sys_stat_all_tables 一样，但只显示系统表
sys_stat_user_tables	和 sys_stat_all_tables 一样，但只显示用户表
sys_stat_all_indexes	当前数据库中的每个索引一行，显示与访问指定索引有关的统计信息。
sys_stat_sys_indexes	和 sys_stat_all_indexes 一样，但只显示系统表上的索引
sys_stat_user_indexes	和 sys_stat_all_indexes 一样，但只显示用户表上的索引
sys_stat_user_functions	每一个被跟踪的函数一行，显示与执行该函数有关的统计信息
sys_stat_xact_all_tables	和 sys_stat_all_tables 相似，但计数动作只在当前事务内发生
sys_stat_xact_sys_tables	和 sys_stat_xact_all_tables 一样，但只显示系统表
sys_stat_xact_user_tables	和 sys_stat_xact_all_tables 一样，但只显示用户表
sys_stat_xact_user_functions	和 sys_stat_user_functions 相似，但是只统计在当前事务期间的调用

见续表

表 3.3.4 – 续表

视图名称	说明
sys_statio_all_tables	当前数据库中的每个表一行，显示有关在指定表上 I/O 的统计信息
sys_statio_sys_tables	和 sys_statio_all_tables 一样，但只显示系统表
sys_statio_user_tables	和 sys_statio_all_tables 一样，但只显示用户表
sys_statio_all_indexes	当前数据库中的每个索引一行，显示与指定索引上的 I/O 有关的统计信息
sys_statio_sys_indexes	和 sys_statio_all_indexes 一样，但只显示系统表上的索引
sys_statio_user_indexes	和 sys_statio_all_indexes 一样，但只显示用户表上的索引
sys_statio_all_sequences	当前数据库中的每个序列一行，显示与指定序列上的 I/O 有关的统计信息
sys_statio_sys_sequences	和 sys_statio_all_sequences 一样，但只显示系统序列
sys_statio_user_sequences	和 sys_statio_all_sequences 一样，但只显示用户序列

上述动态性能视图涉及的 GUC 参数：

- track_counts
  - 跟踪对数据库对象的统计 sys_stat_all_tables，sys_stat_all_indexes（不包括用户自定义函数），默认 on
  - 也跟踪 SQL Count 统计（sys_stat_instevent）
  - 如果关闭该开关，AUTOVACUUM 自动回收功能无法进行
  - 如果关闭该开关，KWR 报告中数据库对象相关报告不存在有效报告数据
- track_io_timing
  - 跟踪对系统 IO 调用的统计，默认 off
  - 如果关闭该开关，共享块、本地块和临时块的读写时间不会被统计，影响 sys_stat_database、sys_stat_sqlio、sys_stat_statements 视图中 blk_read_time、blk_write_time IO 字段
  - 如果关闭该开关，执行计划里面不会打印以下信息：
    * I/O Read Time xxx ms
    * I/O Write Time xxx ms
  - 如果关闭该开关，KWR 报告中跟共享块、本地块和临时块 IO 读写时间相关的字段将为 0，比如“按 I/O 时间排序的 SQL 语句”报告
- track_functions
  - 跟踪对数据库对象中用户自定义函数的统计，包括统计执行次数和执行时间等

  - 可选的值有：
    * ‘none’：禁用函数统计跟踪
    * ‘pl’：只跟踪过程语言函数
    * ‘all’：跟踪 SQL 和 C 语言函数
  - 如果关闭该统计，KWR 报告中“按执行时间排序的函数”、“按执行次数排序的函数”两个报告内容不存在有效报告数据
  - 默认‘none’，如果 PL/PLPG 函数有性能问题，可以设置成‘pl’
- stats_temp_directory
  - 查询累积式统计信息时的统计文件存储路径，默认是 DATA/sys_stat_tmp
  - 可以使用基于 RAM 的文件系统来降低物理 IO 请求
  - 当服务器被正常关闭时，统计数据被永久存储到 sys_stat 目录，服务器再次启动时，统计信息从 sys_stat 目录恢复

关于上述 GUC 参数的意义和作用，具体内容请参见配置管理手册中关于运行时统计数据章节。

3.3.6 数据库时间相关视图

数据库时间模型相关的统计信息是 KWR 报告数据的重要组成部分之一，主要包括：

- SQL 报文统计：sys_stat_sqltime
- SQL 等待事件统计：sys_stat_sqlwait
- SQL IO 统计：sys_stat_sqlio
- 关键活动统计：sys_stat_instevent
- 实例 IO 统计：sys_stat_instio
- 锁活动统计：sys_stat_instlock

例如，sys_stat_sqltime 表用于查询计算数据库时间的 SQL 报文统计信息：

```
kingbase=# SELECT userid, datid, message, bgmsg, calls, times FROM sys_stat_sqltime WHERE queryid = -
1583140210177410494;
 userid | datid | message  | bgmsg |  calls  |  times
--------+-------+----------+-------+---------+----------
  18765 | 14784 | Sync     | f     |  144649 |    37672
  18765 | 14784 | Parse    | f     |      50 |     5425
  18765 | 14784 | Bind     | f     | 1446157 |  7193857
  18765 | 14784 | Execute  | f     | 1446157 | 62377528
  18765 | 14784 | Describe | f     | 1446157 |   558738
(5 行记录)
```

以及，sys_stat_sqlwait 表用于查询 SQL 语句关联的等待事件统计信息：

```
kingbase=# SELECT userid, datid, wait_event, bgwait, calls, times FROM sys_stat_sqlwait WHERE queryid = -
1583140210177410494;
 userid | datid |      wait_event      | bgwait | calls |  times
--------+-------+----------------------+--------+-------+---------
  18765 | 14784 | WALBufMappingLock    | f      | 14547 |  127323
  18765 | 14784 | DataFileRead         | f      | 36281 |  232636
  18765 | 14784 | extend               | f      |   698 | 1285056
  18765 | 14784 | csnlog_ctl           | f      |     2 |       7
  18765 | 14784 | DataFilePrefetch     | f      | 26926 |   31689
  18765 | 14784 | DataFileWrite        | f      | 16138 |  232958
  18765 | 14784 | DataFileExtend       | f      | 15081 |  505458
  18765 | 14784 | buffer_mapping       | f      |  4631 |   28842
  18765 | 14784 | CheckpointerCommLock | f      |   713 |    4077
  18765 | 14784 | lock_manager         | f      |   155 |    1362
  18765 | 14784 | wal_insert           | f      | 74834 |  597447
  18765 | 14784 | CommitSeqNoLock      | f      |     1 |       4
  18765 | 14784 | clog_ctl             | f      |     8 |      46
  18765 | 14784 | buffer_content       | f      |  1070 |   21913
  18765 | 14784 | buffer_io            | f      |    48 |    6537
(15 行记录)
```

数据库时间模型相关视图包括：

视图名称	说明
sys_stat_sqltime	SQL 报文执行次数和时间统计
sys_stat_sqlwait	SQL 语句执行过程产生的等待事件统计
sys_stat_sqlio	SQL 语句执行过程产生的 IO 块数和时间统计
sys_stat_instevent	数据库实例关键活动统计
sys_stat_instio	数据库实例 IO 统计
sys_stat_instlock	数据库实例轻量级锁统计
sys_stat_dbtime	数据库时间模型，包括基本的数据库时间分解
sys_stat_wait	数据库时间模型，包括等待事件时间分解
sys_stat_sql	数据库时间模型，包括 SQL 语句时间分解
sys_stat_msgaccum	同 sys_stat_sqltime，忽略部分字段，该视图不再推荐使用

见续表

表 3.3.5 – 续表

视图名称	说明
sys_stat_waitaccum	同 sys_stat_sqlwait，忽略部分字段，该视图不再推荐使用

上述动态性能视图涉及的 GUC 参数：

- track_sql
  - 跟踪对数据库时间模型的统计：sys_stat_sqltime（SQL 报文统计），sys_stat_sqlwait（SQL 等待事件），sys_stat_sqlio（SQL IO 统计），默认关闭
  - 如果关闭该开关，KWR 报告中数据库时间模型、SQL 报文统计、SQL 等待事件、SQL IO 等报告部分字段无有效数据
  - 建议开启该开关，并搭配：track_sql_item=‘sqltime, sqlwait, sqlio’
- track_instance
  - 跟踪对数据库时间模型的统计：sys_stat_instevent（关键活动），sys_stat_instio（实例 IO），sys_stat_instlock（轻量级锁活动），默认关闭
  - 如果关闭该开关，KWR 报告中数据库时间模型、实例 IO 统计、锁活动统计、关键活动统计等报告部分字段无有效数据
  - 建议开启该开关，并搭配：track_instance_item=‘instevent, instio, instlock’
- track_sql_item
  - 控制 sys_stat_sqltime，sys_stat_sqlwait，sys_stat_sqlio 的统计粒度
  - 不设置该参数时，默认统计上述三种信息，并且也统计 queryid
  - 推荐的典型用法：不统计 queryid，可以有效避免统计膨胀
    - track_sql=on
    - track_sql_item=‘sqltime, sqlwait, sqlio’
  - 可选的值有：
    - sqltime：跟踪 sys_stat_sqltime，但是不跟踪 queryid
    - sqlwait：跟踪 sys_stat_sqlwait，但是不跟踪 queryid
    - sqlio：跟踪 sys_stat_sqlio，但是不跟踪 queryid
    - sqltime_queryid：如果跟踪 sys_stat_sqltime，也会跟踪 queryid
    - sqlwait_queryid：如果跟踪 sys_stat_sqlwait，也会跟踪 queryid
    - sqlio_queryid：如果跟踪 sys_stat_sqlio，也会跟踪 queryid
- track_instance_item
  - 控制 sys_stat_instevent，sys_stat_instio，sys_stat_instlock 的统计粒度

- 不设置该参数时，默认统计上述三种信息，并且也跟踪 sys_stat_instio.relid
- 推荐的典型用法：使用默认统计粒度范围，可以有效避免统计膨胀
  * track_instance=on
  * track_instance_item= ‘instevent, instio, instlock’
- 可选的值有：
  * instevent：跟踪 sys_stat_instevent
  * instio：跟踪 sys_stat_instio，但是不跟踪 sys_stat_instio.relid
  * instlock：跟踪 sys_stat_instlock
  * instio_relid：如果跟踪 sys_stat_instio，也会跟踪 sys_stat_instio.relid

关于上述 GUC 参数的意义和作用，具体内容请参见配置管理手册中关于运行时统计数据章节。

3.3.7　TOP SQL 相关视图

TOP SQL 统计提供 SQL 语句级别执行情况的统计。其主键字段通常包括 userid、dbid 和 queryid，统计信息涵盖 SQL 语句的解析计划和执行次数与时、IO 块数、IO 时间等关键指标，例如通过 sys_stat_statements 进行统计信息查询：

```
kingbase=# SELECT userid, dbid, queryid, substr(query,0,20), total_parse_time, total_plan_time, total_exec_
time FROM sys_stat_statements LIMIT 10;
 userid | dbid  |       queryid        |        substr        |  total_parse_time  |   total_plan_time   |
total_exec_time
--------+-------+----------------------+----------------------+--------------------+---------------------+--
--------------------
  18765 | 14784 |  6443611319251330892 | create table bmsql_c |            0.07859 |                   0 |
           4.9792
     10 | 14785 |   765757783043909597 | select perf.ksh_time | 0.8814299999999999 | 0.20921000000000003 |
 337.9123400000001
  18765 | 14784 |  6987772690106732594 | INSERT INTO bmsql_cu |  4.248060000000001 |  3.2016599999999995 |
 7400.832050000276
  18765 | 14784 | -3050869663116709291 | create table bmsql_s |            0.03328 |                   0 |
          0.74234
     10 | 14785 |  3435481819261536772 | SELECT userid,dbid,q |            0.35208 |             0.11514 |
          9.19038
     10 | 14785 | -7657553955000651630 | set default_sql_pars |            0.04503 |                   0 |
          0.03561
     10 | 14785 |  2581027241184715520 | select * from pg_pro |            0.79491 |             1.22186 |
0.17242000000000002
  18765 | 14784 | -4525441006436104639 | SELECT w_name, w_str | 14.918589999999995 |  28.245969999999996 |
 5963.104380000166
     10 | 14785 |  8797235071638044795 | select ((select valu |            1.05817 |  0.8060799999999999 |
```

```
0.049300000000000004
  18765 | 14784 | -7381118047965252786 | alter table bmsql_wa |            0.06422 |                  0 |
          7.12041
(10 行记录)
```

TOP SQL 相关视图包括：

视图名称	说明
sys_stat_statements	顶层 SQL 语句统计列表
sys_stat_statements_all	全部 SQL 语句（包括嵌套语句）统计列表

上述动态性能视图涉及的 GUC 参数：

- sys_stat_statements.track
  - 控制语句统计级别，默认为‘none’，禁用语句统计信息收集
  - 如果关闭该参数，KWR 报告中的 TOP SQL 系列报告会不存在有效报告数据，而且全部统计里会没有 queryid
  - KWR、KSH、SQL 绑定变量、SQL 调优建议器、SQL 监控等性能调优特性都依赖 queryid 和 SQL 语句字符串
  - 可选的值有：
    - ‘none’：禁用语句统计信息收集
    - ‘top’：跟踪顶层 SQL，即客户端直接发出的 SQL 语句
    - ‘all’：跟踪全部 SQL，即包含了顶层 SQL 和嵌套的子 SQL
  - 在常规应用场景中，建议启用该功能开关’top’。但需注意的是，启用此功能可能会导致系统性能出现约 2% 至 5% 的下降
  - 在涉及大量 PLSQL 执行且存在性能瓶颈并需要进行深入分析的业务场景中，建议启用该功能开关’all’
- sys_stat_statements.track_parse
  - 是否跟踪解析时间和次数，默认为 on，不建议关闭
- sys_stat_statements.track_plan
  - 是否跟踪计划时间和次数，默认为 on，不建议关闭
- sys_stat_statements.track_utility
  - 是否跟踪 Utility 语句（即除 SELECT、INSERT、UPDATE、DELETE 和 MERGE 之外的所有命令），默认跟踪
  - 当统计数据显示 Utility 语句的比例显著较高时（占比超过 30%），建议禁用相关功能开关，以确保更多 DML 语句保留在统计列表里面

- sys_stat_statements.max
  - 最多存储的统计条目数，默认值为 5000。如果统计到的语句条目数超过配置值，最少被执行的语句统计条目会被丢弃
  - 在进行数据库性能调优的实际过程中，若发现统计项目的容量限制问题较为突出，建议采取如下优化措施：
    * 将 sys_stat_statements.max 参数值调整至 10000 或者 20000。但需要注意的是，此操作将会占用更多的系统内存资源，并引发 KWR 快照的膨胀问题
    * 关闭 Utility 语句统计：将 sys_stat_statements.track_utility 参数配置为 off
    * 执行 `SELECT sys_stat_statements_reset();` 重置统计信息
- sys_stat_statements.save
  - 是否在服务器关闭时将统计信息保存到统计文件，默认保存
  - 不建议随意将此参数设置为 off. 仅当需要在服务重启后自动重置统计信息时，才应调整该配置

关于上述 GUC 参数的意义和作用，具体内容请参见 sys_stat_statements 插件手册。

3.3.8 主机信息相关视图

当前版本尚未提供针对主机信息的动态性能视图，用户可通过相应的 SQL 函数接口获取主机的 CPU 使用率、内存占用情况、I/O 操作数据以及网络流量统计信息。

比如查看当前主机的内存使用情况：

```
kingbase=# SELECT total_memory, used_memory, free_memory, swap_total, swap_used, swap_free, shmem_memory
FROM perf.sys_memory_info();
 total_memory | used_memory | free_memory | swap_total | swap_used | swap_free | shmem_memory
--------------+-------------+-------------+------------+-----------+-----------+--------------
    501061696 |    22592768 |   208637056 |    4194240 |   1126336 |   3067904 |      8838528
(1 行记录)
```

主机信息相关统计的函数包括：

函数名称	说明
perf.sys_cpu_stats_info()	CPU 使用信息
perf.sys_load_avg_info()	CPU 负载
perf.sys_cpu_stat_by_process(int pid)	指定进程的 CPU 统计信息
perf.sys_io_stat_info()	主机磁盘设备的 IO 统计信息

见续表

表 3.3.7 – 续表

函数名称	说明
perf.sys_io_stat_byprocess(int pid)	指定进程 IO 统计信息
perf.sys_memory_info()	主机内存统计
perf.sys_mempage_stat_byprocess(int pid)	指定进程的内存统计信息
perf.sys_network_info()	主机网络流量统计

信息:

以上 SQL 函数依赖于 sys_kwr 插件的创建和配置。

3.3.9 管理动态性能视图

3.3.9.1 开启动态性能视图统计

在某些场景下，建议全面启用统计功能，尽可能详尽地采集相关信息以辅助潜在性能问题的分析。可通过修改 kingbase.conf 配置文件中的相关统计开关参数，实现对全部统计功能的启用：

```
track_activities = on
track_counts = on
track_functions = 'all'
track_sql = on
track_instance = on
track_io_timing = on
sys_stat_statements.track = 'all'
```

信息:

上述 GUC 参数均为会话级别设置，可在具体会话中直接完成配置操作：

例如：SET track_sql = on，该配置将在当前会话中即时生效。

除此之外，还可以在不中断 KingbaseES 服务运行的情况下，利用 sys_reload_conf() 函数来完成整个实例范围内的配置参数更新：

```
kingbase=# ALTER system SET track_sql TO on;
ALTER SYSTEM
kingbase=# SELECT sys_reload_conf();
 sys_reload_conf
```

```
-----------------
 t
(1 行记录)
```

3.3.9.2　关闭动态性能视图统计

在某些情况下，为了达到最佳系统性能，建议关闭全部统计功能。通过配置以下统计开关即可实现此目的：

```
track_activities = off
track_counts = off
track_functions = 'none'
track_sql = off
track_instance = off
track_io_timing = off
sys_stat_statements.track = 'none'
```

3.3.9.3　清理动态性能视图统计

在某些情况下，为了更好地观察和分析性能指标，建议定期清理之前的统计结果。可以通过执行特定的重置函数来清理对应的统计信息：

```
-- 清理全部共享对象统计（比如 sys_stat_bgwriter）
SELECT sys_stat_reset_shared('all');
-- 清理当前库的数据库对象统计
SELECT sys_stat_reset();
-- 清理全部 SQL 统计
SELECT sys_stat_statements_reset();
```

或者通过删除统计文件清理（通常情况下不建议）：

1. 停止 KingbaseES 服务
2. 手工删除 DATA/sys_stat 目录下所有文件
3. 启动 KingbaseES 服务

信息:

个别统计项比如 sys_stat_database 不受统计开关限制，在执行统计清理后仍会持续更新统计数据。

3.4　KWR 报告

本章节包含以下内容：

- KWR 快照概述
- 生成 KWR 快照
- 生成 KWR 报告
- KWR 按库展示报告
- KWR 数据库时间模型解析
- KWR 其它性能指标解析

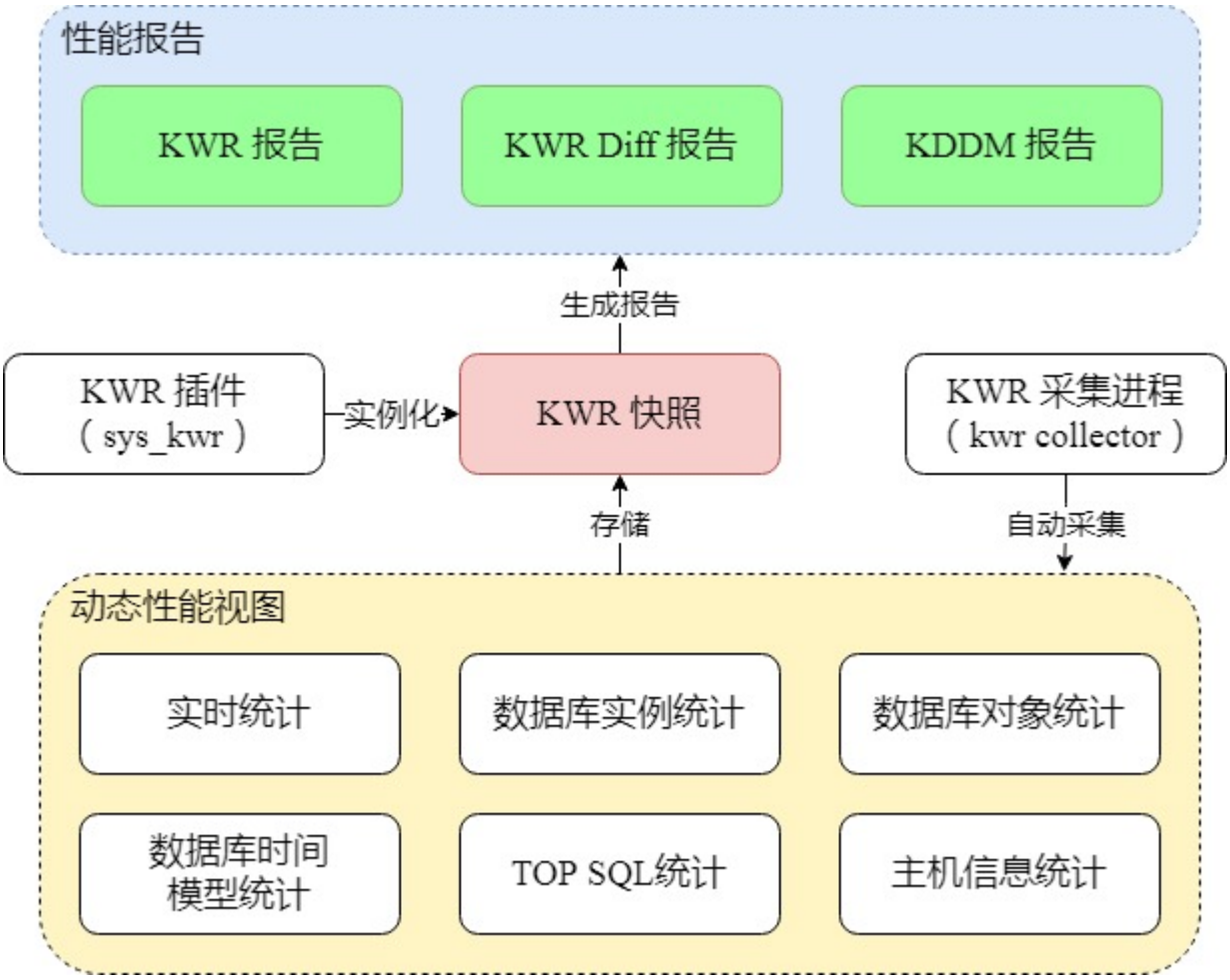
3.4.1　KWR 快照概述

KingbaseES 数据库在运行过程中动态生成的各类性能统计数据以性能视图的形式存在，然而这些数据会随着系统运行实时更新变化，导致 DBA 无法查看特定历史时期内的性能指标的变化情况。

KWR 快照通过记录两个不同时间段的动态性能视图差值来保存历史统计信息。该功能可由后台进程 kwr collector 按照预设的时间间隔（默认为每小时一次）自动执行快照操作，同时 DBA 也可以通过手动执行 SQL 语句的方式来创建快照。

KWR 快照默认保留 8 天，超过此时间后，在新的快照创建时，系统将自动清除已超过保留期限的快照数据。

这些 KWR 快照为性能分析工具 KWR、KDDM 以及 KWR DIFF 报告提供必要的统计基础数据，并用于生成数据库的时间模型，从而支持进行深入的性能调优工作。



3.4.2　生成 KWR 快照

1、创建 KWR 插件

要想使用 KWR 快照和报告功能，需要在某个库上创建 sys_kwr 插件：

```
kingbase=# \c benchmarksql
You are now connected to database "benchmarksql" as userName "benchmarksql".

benchmarksql=# CREATE EXTENSION sys_kwr;
CREATE EXTENSION

benchmarksql=# \dt perf.*
                          关联列表
 架构模式  |            名称            |  类型  |     拥有者
----------+----------------------------+--------+--------------
 perf     | ksh_history_data           | 数据表 | benchmarksql
 perf     | ksh_statements             | 数据表 | benchmarksql
```

```
 perf     | kwr_index_list          | 数据表 | benchmarksql
 perf     | kwr_last_bgwriter       | 数据表 | benchmarksql
 perf     | kwr_last_database       | 数据表 | benchmarksql
...
```

信息:

在特定数据库（如 benchmarksql）中创建 sys_kwr 插件后，KWR 快照表将在该数据库自动生成。后续查看快照和生成报告需连接到相应数据库进行操作。

2、**配置统计** GUC

在 kingbase.conf 里设置以下推荐 GUC 参数：

```
track_counts = on
track_io_timing = on
track_functions = 'all'
track_sql = on
track_instance = on
track_sql_item = 'sqltime, sqlwait, sqlio'
track_instance_item = 'instevent, instio, instlock'
sys_stat_statements.track = 'top'
```

3、**自动生成** KWR **快照**

通过配置以下参数可开启自动快照功能。KingbaseES 后台进程 kwr collector 每小时会自动采集动态性能视图统计信息并生成 KWR 快照：

```
sys_kwr.enable = on              # 开启自动快照
sys_kwr.database = 'kingbase'    # 自动快照所在库，默认'kingbase'
sys_kwr.interval = 60            # 自动快照间隔，默认 60 分钟
```

信息:

配置上述参数后，KingbaseES 会在 sys_kwr.database 库上检查 sys_kwr 插件是否存在。若未创建，则自动完成插件的创建。

4、**手工生成** KWR **快照**

执行 `SELECT perf.create_snapshot();` 可以手动生成 KWR 快照。

5、**查看和清理** KWR **快照**

通过 `perf.kwr_snapshots` 来查看全部快照表：

```
benchmarksql=# SELECT snap_id,sess_count FROM perf.kwr_snapshots;
 snap_id | sess_count
---------+------------
       1 |          1
       2 |          1
       3 |        102
       4 |        102
       5 |        102
(5 行记录)
```

有两种方法可以清理 KWR 快照：

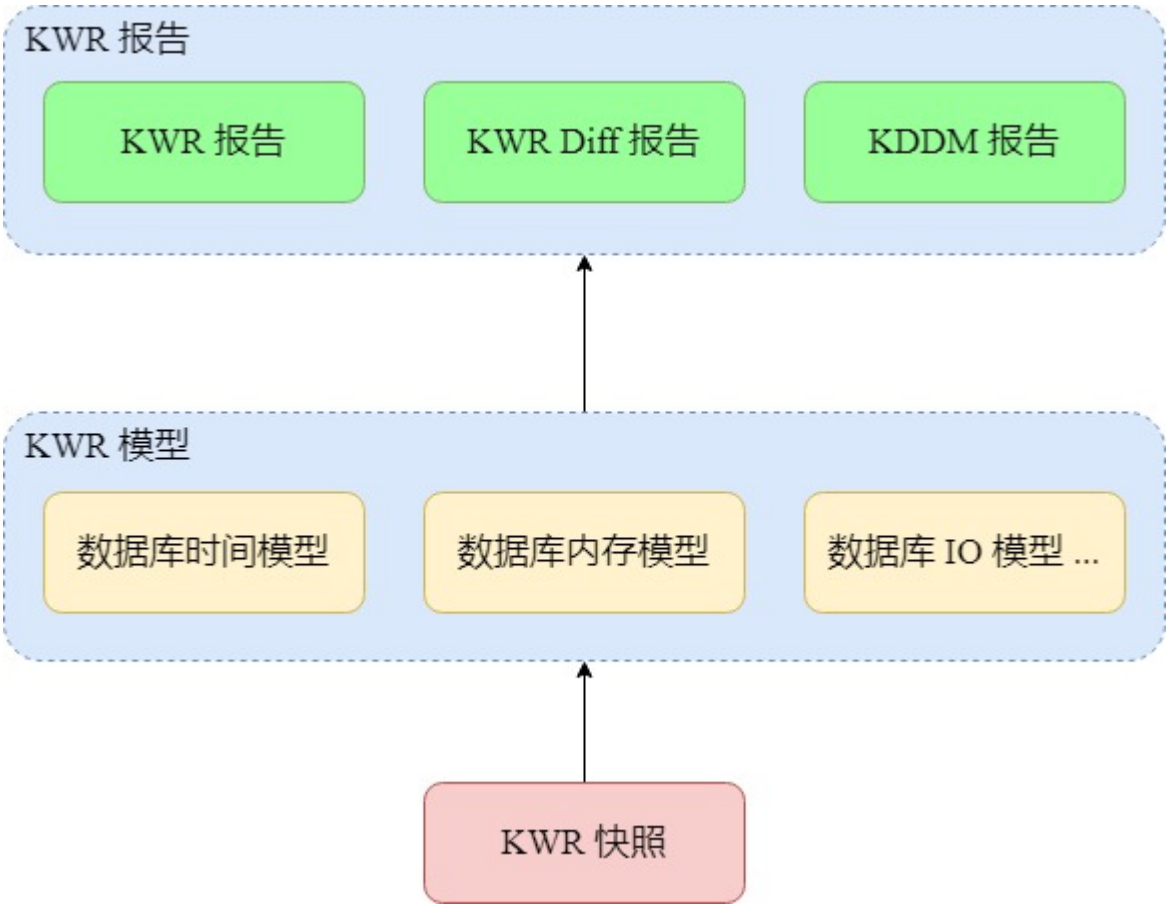
1）`SELECT perf.reset_snapshots();` 清理全部快照

2）重建 sys_kwr 插件：`DROP EXTENSION sys_kwr; CREATE EXTENSION sys_kwr;`

3.4.3　生成 KWR 报告

KWR 报告是以 KWR 快照为基础，通过构建数据库时间模型、内存模型、IO 模型等技术支撑体系，形成多维度的性能分析报告。该报告体系包含以下三个主要组成部分：

- KWR 报告：展示数据库的时间模型、内存使用情况及 IO 性能等关键指标
- KWR DIFF 报告：对比两组 KWR 快照之间的性能差异
- KDDM 报告：基于模型分析性能问题并提供优化建议



基于 KWR 快照生成报告的 SQL 示例：

- 生成 KWR 报告

```
SELECT * FROM perf.kwr_report(1, 2, 'html');
```

- 生成 KWR DIFF 报告

```
SELECT * FROM perf.kwr_diff_report(1, 2, 3, 4);
```

- 生成 KDDM 报告

```
SELECT * FROM perf.kddm_report(1, 2, 'html');
```

- 生成 KDDM SQL Report

```
SELECT * FROM perf.kddm_sql_report(1, 2, 3278935302158814650);
```

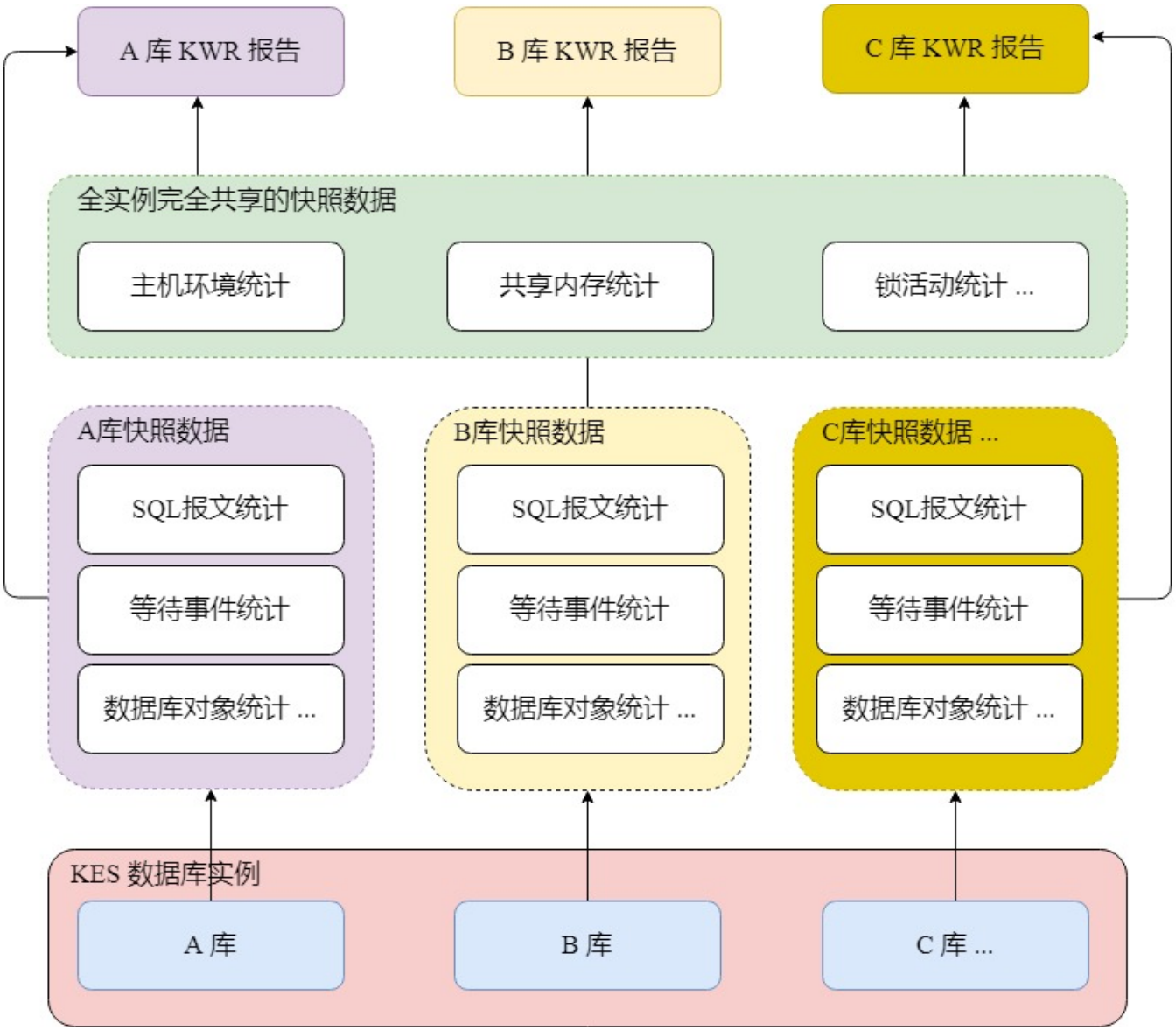
信息:

如需了解更多关于报告生成的 SQL 函数，可参考 sys_kwr 插件手册。

3.4.4　KWR 按库展示报告

由于 KWR 快照中仅数据库对象部分位于 sys_kwr 插件所在的特定数据库，其余部分均为全实例级别的统计数据，因此默认情况下会生成基于全实例的 KWR 报告（除数据库对象相关内容）。

在某些性能调优场景下，为了专注于某一个数据库的性能分析和优化，需要排除其他数据库的性能指标（例如 TOP SQL）对目标库分析结果的影响。此时需按库展示 KWR 报告，即：仅显示指定数据库的所有性能指标，而不展示任何其他数据库的相关数据。



以下是按库展示 KWR 报告的示例：

```
SELECT * FROM perf.kwr_report(1, 2, 'html', 'benchmarksql');
```

信息:

每个用于生成报告的 SQL 函数均包含一个’database’参数，该参数用于指定需按库展示的具体数据库名称，默认值为空（NULL），此时将按照全实例级别生成 KWR 报告。

以下为按库展示的实际效果示例：

按库展示	启动时间	发行版本	KWR版本	编译版本
benchmarksql	[redacted]	[redacted]	[redacted]	[redacted]

主机名	平台	处理器型号	处理器数	核心数	处理器槽位	内存
localhost.localdomain	Kylin Linux Advanced Server V10 (Sword)	HUAWEI,Kunpeng 920	128	128	2	478 GB

	快照 ID	快照时间	会话数
Begin Snap	1	[redacted] 14:32:12	102
End Snap	2	[redacted] 14:43:14	102
Elapsed		11.03 (mins)	
DB Time		1087.51 (mins)	

各子报告也仅显示指定的 benchmarksql 数据库的相关指标：

数据库统计

- 按数据库事务总数排序

数据库	提交数	回滚数	死锁数	读取块数	命中块数	命中率%	返回元组数	获取元组数	插入元组数	更新元组数	删除元组数
benchmarksql	1376061	126459	0	630880708	3488714584	84.69	23461823668	2058938891	8697772	16462905	595993

作为对比参考，以下是未按库展示的全实例报告示例，该报告会展示所有数据库的性能指标：

数据库统计

- 按数据库事务总数排序

数据库	提交数	回滚数	死锁数	读取块数	命中块数	命中率%	返回元组数	获取元组数	插入元组数	更新元组数	删除元组数
benchmarksql	1376061	126459	0	630880708	3488714584	84.69	23461823668	2058938891	8697772	16462905	595993
kingbase	769	0	0	17007	281973	94.31	298985	49231	44825	375	437
test	16	0	0	2036	1320	39.33	61487	231	0	0	0
security	0	0	0	0	0	0.00	0	0	0	0	0

3.4.5　KWR 数据库时间模型解析

本章节包含以下内容：

- 数据库时间模型概述
- 按 SQL 报文分解
- 按 SQL 语句类型分解
- 按等待事件分解
- 按关键活动分解

- 按 SQL 执行阶段分解
- 按 TOP SQL 分解

3.4.5.1　数据库时间模型概述

数据库时间（DB Time）是指服务器为了响应客户端请求花费的总时间，即客户端会话进程（Backend）执行 SQL 语句的总时间，以及客户端连接建立和断开的时间之和。

在处理客户端请求期间，数据库服务器经历多种状态：其中包括空闲等待（如等待客户端发送请求）与非空闲等待（如输入输出操作、锁等待事件），同时包含执行 SQL 语句所占用的 CPU 时间。

数据库时间定义为客户端会话进程处于非空闲等待状态的时间与执行 SQL 语句占用的 CPU 时间之和：

数据库时间 = 非空闲等待时间 + CPU 时间

信息:

当前版本中，数据库时间主要针对处理客户端请求的服务进程进行度量。后台服务进程（如检查点进程等）的时间未纳入本模型考量范围。此外，客户端连接建立和断开的计时功能尚未实现。

数据库时间模型是 KingbaseES 为了量化数据库性能指标，方便性能诊断和优化而开发出的一套模型。该模型可从多个维度分解数据库时间，从而深入分析性能问题的根本原因：

- 按 SQL 报文分解
- 按 SQL 语句类型分解
- 按等待事件分解
- 按关键活动分解
- 按 SQL 执行阶段分解
- 按 TOP SQL 分解

3.4.5.2　按 SQL 报文分解

数据库时间的主要来源是通过分析 SQL 报文进行计算。具体而言，前端标注的所有报文时间累计总和即为数据库时间的总值。

客户端在执行 SQL 语句时，会按照以下两种协议将其发送至服务器端：

- 简单报文协议：‘Q’报文协议，原始 SQL 语句发过来执行并返回结果
- 扩展报文协议：‘P/B/D/E/S’报文协议，客户端先发送 P 报文，服务器端解析 SQL，然后客户端后续多次发送’B’+’E’报文，通过绑定变量的方式来执行 SQL

数据库时间模型支持的 SQL 报文包括：

- Simple Message：简单查询报文时间
- Parse Message：Parse 报文时间
- Bind Message：Bind 报文时间
- Execute Message：Execute 报文时间
- Sync Message：Sync 报文时间
- Flush Message：Flush 报文时间
- Describe Message：Describe 报文时间
- Fastpath Message：Fastpath 报文时间
- Wait Message：Wait 报文时间
- Close Message：Close 报文时间

以下是 KWR 报告 [SQL 报文执行时间] 数据库时间按 SQL 报文分解示例：

报文	执行次数	总时间(s)	平均时间(ms)	% 数据库时间
Execute	9752743	23851.75	2.45	99.09
Bind	9649031	142.01	0.01	0.59
Wait Message	59098906	71.90	0.00	0.30
Sync	6592023	2.10	0.00	0.01
Simple Query	3	1.61	535.60	0.01
Describe	3103098	1.18	0.00	0.00
Parse	1	0.00	0.18	0.00

3.4.5.3　按 SQL 语句类型分解

按 SQL 语句类型分解数据库时间，有助于识别占用最多时间的 SQL 语句类型（如 INSERT 或 SELECT），从而便于针对性实施数据库性能优化策略。

这些 SQL 语句类型包括：

- DML：增删改查等数据操作类型
  - SELECT：查询语句
  - UPDATE：更新语句
  - INSERT：新增语句
  - DELETE：删除语句
  - MERGE：合并语句
- DDL：数据定义语句类型，CREATE TABLE, REINDEX 等相关
- DCL：数据控制语句类型，CREATE ROLE、GRANT 等权限相关
- TCL：事务相关语句，BEGIN、COMMIT 等相关
- OTHERS：其他语句类型

以下是 KWR 报告 [SQL 语句数统计]，将数据库时间按 SQL 语句数分解示例：

名称	总时间(s)	平均时间(ms)	% 数据库时间	总次数	每秒次数	每事务次数	次数占比
DML	5590.35	0.60	23.22	9337843	37054.93	26.47	92.98
Update	4039.20	1.54	16.78	2617347	10386.30	7.42	26.06
Select	1242.60	0.27	5.16	4529823	17975.49	12.84	45.11
Insert	205.45	0.10	0.85	2048955	8130.77	5.81	20.40
Delete	101.08	0.71	0.42	141638	562.06	0.40	1.41
TCL	5.95	0.01	0.02	704498	2795.63	2.00	7.02

3.4.5.4　按等待事件分解

等待事件构成了数据库时间的关键要素，其中并发操作可能导致资源竞争问题，从而引发资源等待情形。当这些等待事件占数据库总时间比例超过 5% 时，通常表明存在特定的性能瓶颈。

通过将数据库时间按等待事件分解，能够迅速识别出占比最高的等待事件，并结合查询标识符（queryid）获取更多统计指标，进而确定问题的根本原因并制定优化策略。

以下是 KWR 报告 [数据库统计]，数据库等待时间按等待事件分解示例：

等待事件	等待次数	总时间(s)	平均时间(ms)	每事务等待数	% 数据库时间
DB CPU	-	-	-	-	4.93
WALWriteLock	596294	17849.36	29.93	1.69	74.15
transactionid	56209	3542.22	63.02	0.16	14.72
wal_insert	1143047	563.63	0.49	3.24	2.34
tuple	8717	530.54	60.86	0.02	2.20
WALSync	8289	143.27	17.28	0.02	0.60
WALInitSync	546	78.29	143.38	0.00	0.33
DataFileRead	9054984	72.63	0.01	25.66	0.30
buffer_content	71067	43.38	0.61	0.20	0.18
DataFileWrite	3865794	40.33	0.01	10.96	0.17
WALBufMappingLock	565227	6.54	0.01	1.60	0.03
CheckpointerCommLock	176269	3.73	0.02	0.50	0.02
WALWrite	9969	2.77	0.28	0.03	0.01
extend	2352	2.54	1.08	0.01	0.01
WALInitWrite	538	1.61	2.99	0.00	0.01
DataFilePrefetch	788116	1.26	0.00	2.23	0.01
ClogGroupUpdate	17869	0.74	0.04	0.05	0.00
clog_ctl	41928	0.53	0.01	0.12	0.00
XidGenLock	32204	0.49	0.02	0.09	0.00
DataFileImmediateSync	12	0.40	33.65	0.00	0.00
DataFileExtend	62464	0.40	0.01	0.18	0.00

信息:

关于 KingbaseES 等待事件的详细描述，可以参考：等待事件

3.4.5.5　按关键活动分解

关键活动是数据库运行过程中耗时较多的一些关键功能执行，例如：PinBuffer、CommitTransaction、InitializePlan、Lock Acquire 等操作。这些活动主要源自 sys_stat_instevent 视图。

与火焰图采用的采样式统计方法不同，关键活动统计使用累积式统计方式，因此其记录的关键活动次数和耗时数据更为精确且可靠性更高。然而，该统计方法仅能对预定义的关键活动进行记录。

关键实例活动按类别划分如下：

- Buffer：内存缓冲区相关统计
- Executor：执行器相关统计
- Lock：对象锁相关统计
- Maintance：维护操作（创建索引等）相关统计
- SMGR：存储管理器相关统计
- Transaction：事务相关统计
- Wal：WAL 日志相关统计
- CheckPoint：检查点相关统计
- Misc：其他，比如登录、登出统计

以下是 KWR 报告 [关键活动按执行时间统计]，数据库时间按关键活动分解的示例：

类型	名称	执行次数	总时间(s)	平均时间(ms)	% 数据库时间
Buffer	Buffer Read	79358137	246.06	0.00	1.02
Wal	WAL Insert	14581909	719.75	0.05	2.99
Misc	Total Parses	10044841	0.00	0.00	0.00
SMGR	MD Read	9684036	80.71	0.01	0.34
Buffer	Buffer Flush	4137050	69.61	0.02	0.29
Buffer	Buffer Write Dirty	4137050	73.19	0.02	0.30
SMGR	MD Write	4137050	44.91	0.01	0.19
Lock	Lock ReleaseAll	704504	4.11	0.01	0.02
Lock	Lock Acquire	491345	4243.06	8.64	17.63
Transaction	Transaction Start	352252	0.63	0.00	0.00

3.4.5.6　按 SQL 执行阶段分解

数据库时间可以按照执行阶段分解，这些 SQL 执行阶段目前主要包括：

- Parse：SQL 语法解析
- Analyze：SQL 语义分析
- Rewrite：SQL 语句重写
- Plan：生成执行计划
- Initialize Plan：初始化执行计划

- Execute：执行各计划节点
- Commit Transaction：提交事务

以下是 KWR 报告 [时间模型统计] 中数据库时间分解示例：

统计项	时间(s)	% 数据库时间
DB CPU	1185.61	4.93
Foreground Wait Time	22884.94	95.07
Parse Time	0.01	0.00
Analyze Time	0.92	0.00
Rewrite Time	0.05	0.00
Plan Time	12.25	0.05
InitPlan Time	48.86	0.20
Execute Time	5596.83	23.25
CommitTransaction Time	19129.23	79.47
PLSQL Compile Time	0.00	0.00
PLSQL Execute Time	0.00	0.00
PLPGSQL Compile Time	0.00	0.00
PLPGSQL Execute Time	1.53	0.01
Foreground Net Read Time	33.56	0.14
Foreground Net Write Time	30.41	0.13
Foreground DateFile IO Time	0.00	0.00
Foreground Instance IO Time	124.53	0.52
DB Time	24070.55	100.00

3.4.5.7　按 TOP SQL 分解

数据库时间按 TOP SQL 分解主要基于 sys_stat_statements 视图。

KWR 报告中 [Top SQL 统计] 从多个子维度分解数据库时间：

- 按数据库时间排序的 SQL 语句
- 按 CPU 时间排序的 SQL 语句
- 按解析时间排序的 SQL 语句
- 按计划时间排序的 SQL 语句
- 按执行时间排序的 SQL 语句
- 按执行次数排序的 SQL 语句
- 按返回元组数排序的 SQL 语句
- 按 I/O 时间排序的 SQL 语句
- 按逻辑读块数排序的 SQL 语句
- 按物理读块排序的 SQL 语句
- 按逻辑写块数排序的 SQL 语句
- 按物理写块数排序的 SQL 语句

- 按临时块数读写排序的 SQL 语句
- 按本地数据块使用排序的 SQL 语句
- 完整 SQL 列表

以下是 KWR 报告 [按执行时间排序的 SQL 语句] 报告示例：

QueryID	执行次数	执行时间(s)	数据库时间(%)	平均时间(ms)	最小时间(ms)	最大时间(ms)	SQL 语句
5333761347722434871	151510	2997.81	12.45	6.97	0.00	70.25	UPDATE bmsql_warehouse SET w_ytd =…
7717458256612817612	151505	698.67	2.90	0.43	0.00	0.00	UPDATE bmsql_district SET d_ytd = …
2655240597523385636	158427	682.49	2.84	0.40	0.00	184.49	SELECT d_tax, d_next_o_id ,CTID,XM…
4383288571379112176	1581058	232.61	0.97	0.00	0.00	0.00	SELECT s_quantity, s_data, s_dist_…
-5332993246613577513	1581058	121.55	0.50	0.00	0.00	15.08	INSERT INTO bmsql_order_line (ol_…
3889337793880648542	14089	114.49	0.48	0.00	0.00	54.19	SELECT count(*) AS low_stock FROM …
-3903412935930362358	1581058	112.08	0.47	0.00	0.00	115.12	UPDATE bmsql_stock SET s_quantity …
-1376825226900098240	141811	98.67	0.41	0.00	0.00	0.00	DELETE FROM bmsql_new_order WHERE …
-6247288520632161420	141811	75.26	0.31	0.00	0.00	103.35	SELECT no_o_id FROM bmsql_new_orde…
7150728171464675656	141363	65.34	0.27	0.00	0.00	0.00	UPDATE bmsql_order_line SET ol_del…
-1732742306280230951	141361	48.12	0.20	0.00	0.00	26.12	UPDATE bmsql_customer SET c_balanc…
-1471418561662554298	141363	37.19	0.15	0.00	0.00	44.28	UPDATE bmsql_oorder SET o_carrier_…
-994111915640741594	14159	36.27	0.15	0.00	0.00	0.00	SELECT o_id, o_entry_d, o_carrier_…
5376249718113372425	1582623	26.61	0.11	0.00	0.00	7.43	SELECT i_price, i_name, i_data FRO…
-6268019545502362871	158427	22.30	0.09	0.00	0.00	72.48	INSERT INTO bmsql_oorder (o_id, o…
-6846873336084754536	151510	21.27	0.09	0.00	0.00	0.00	INSERT INTO bmsql_history (h_c_id…
7303131723753805870	158427	19.93	0.08	0.00	0.00	0.00	SELECT c_discount, c_last, c_credi…
2908894973773436196	99777	19.24	0.08	0.00	0.00	0.00	SELECT c_id FROM bmsql_customer WH…
-2551117905262579363	151510	14.77	0.06	0.00	0.00	75.69	SELECT c_first, c_middle, c_last, …
-7684618786315616167	136462	14.52	0.06	0.00	0.00	134.15	UPDATE bmsql_customer SET c_balanc…

此外，KDDM 和 KDDM SQL Report 可以以父子关系展示顶层 PLSQL 语句及其子 SQL 的层次调用关系，从而轻松识别是 PLSQL 自身的问题，还是某个叶子节点子 SQL 的性能问题：

```
[数据库时间分解]:

Event                                    DB Time  Percent  QueryID
-------------------------------------------------------------------------------
OnCpu                                    98.46 %
  Cpu                                    98.46 %    100 %
    call                                 77.65 %  81.85 %   -927218736833029675
    call                                 10.19 %  10.75 %   7361807630815516914
    call                                   3.2 %   3.38 %  -9125104228314308371
IO                                        1.15 %


[TOP SQL 建议]:

 queryid                    | dbtime     | parse    | plan     | execute
-------------------------------------------------------------------------------
-927218736833029675         | 77.65%     |      0%  |      0%  | 77.64%
  -6790214550964780291      | 77.61 %    |  0.01%   |      0%  |  77.6%
    -8873221078737522925    | 77.33 %    |      0%  |      0%  | 77.32%
      8489579778678244971   |  44.4 %    |      0%  |      0%  | 44.39%
      7507757688395121142   | 19.17 %    |      0%  |      0%  | 19.16%
      -1056961238992406066  |  9.69 %    |      0%  |      0%  |  9.69%
    -7666453913067878603    |  0.25 %    |  0.25%   |      0%  |     0%
    -6182309774224761661    |  0.03 %    |      0%  |      0%  |  0.03%
7361807630815516914         | 10.19%     |      0%  |      0%  | 10.19%
```

3.4.6　KWR 其它性能指标解析

本章节包含以下内容：

- KWR 快照时间
- 实例效率百分比
- IO 性能指标解析
- 内存统计指标解析
- 数据库对象指标解析

3.4.6.1　KWR 快照时间

3.4.6.1.1　快照时间概述

快照时间（Elapsed Time）是指生成 KWR 报告的两个快照之间的时间间隔。

在 KWR 报告中，多处可以查看到快照时间，比如报告头（以分钟为单位）：

	快照 ID	快照时间
Begin Snap	1	11:00:34
End Snap	2	11:04:46
Elapsed		4.20 (mins)

以及负载分析报告阅读说明（以秒为单位）：

负载分析

- 显示数据库快照期间的负载信息统计
- 字段 "每秒"：测量出来的总值除以快照时间（秒数）
- 字段 "每事务"：测量出来的总值除以快照期间事务总数
- 字段 "每执行"：测量出来的总值除以快照期间 SQL 语句执行总次数
- 快照总时间：252 秒，用于计算每秒统计值
- 总事务数：352832，用于计算每事务统计值

统计项	总值	每秒
DB Time(s)	24070.55	95.52

3.4.6.1.2　快照时间的作用

快照时间主要用于衡量数据库实例统计信息，或者后台统计的“每秒”值。这些统计指标通常是全实例或后台统计的结果，无法直接用数据库时间来度量。由于统计项的总值往往较大，在分析报告时使用“每秒”值会更加直观。

另外，如果需要比较两组快照期间某个全实例统计项的表现，由于两组快照的快照时间可能存在差异，仅通过总值进行比较并不合适。此时，使用每秒值或平均值进行比较则更为方便。

”每秒”值表示快照期间统计项在每秒钟内的平均统计值。结合报告头中提供的“处理器数”，可以评估出可能存在性能问题的指标项，例如：

主机名	平台	处理器型号	处理器数
localhost.localdomain	Kylin Linux Advanced Server V10 (Sword)	HUAWEI,Kunpeng 920	128

统计项	总值	每秒
DB Time(s)	24070.55	95.52
DB CPU(s)	1185.61	4.70
Foreground Wait Time(s)	22884.94	90.81
WAL Size(MB)	8694.46	34.50
Blocks Hit	65295943	259110.88
Blocks Read	9052584	35922.95
Blocks Dirty	3910403	15517.47
Blocks Write	3981575	15799.90
Blocks Read Size(MB)	70723.31	280.65
Blocks Write Size(MB)	31106.05	123.44
Parse Calls	10258	40.71
Plan Calls	142417	565.15
Execute Calls	10045014	39861.17
Tuples Return	195018448	773882.73
Tuples Fetch	107986670	428518.53
Tuples Insert	2054759	8153.81
Tuples Update	3892673	15447.12
Tuples Delete	141916	563.16
Commits	322999	1281.74
Rollbacks	29833	118.38
Transactions	352832	1400.13
Logon	0	0.00
Logoff	0	0.00
DeadLock	0	0.00

以下是分析示例：

- 处理器数：128，TPCC 并发数：100
- DB Times(s)：每秒 95.52，假设 TPCC 的 JDBC 客户端使用了 20 个 CPU，加上几个后台进程使用 CPU，则说明大部分 CPU 资源都处于忙碌或等待资源的状态

- Foreground Wait Times：每秒 90.81，表明每秒有 90.81 个 CPU 在等待资源。从该指标可以看出等待事件可能是性能瓶颈，需要结合 [前台等待事件] 报告进一步分析
- Parse Calls、Plan Calls、Execute Calls：每秒分别为 40.71、565.15、39861.17，说明解析和计划的重用率较高。如果这些值接近，通常意味着存在大量简单的 SQL 查询（Simple Query），导致解析和计划重用率较低
- Transactions：每秒 1400.13，表明每秒有 1400.13 个事务完成

3.4.6.1.3　快照时间、TPS 和 QPS

有了快照时间和快照期间的查询总值及事务总值，就可以计算出 TPS（Transactions Per Second）和 QPS（Queries Per Second）等性能指标。

KWR 报告负载分析中的每秒"Execute Calls" 即为快照期间的 QPS，而每秒"Transactions" 则为快照期间的 TPS。

3.4.6.2　实例效率百分比

KWR 报告中 [实例效率百分比] 报告展示了期望得到 100% 值的统计项。当他们高于 90% 甚至接近或者达到 100% 时，数据库运行时的效率最高。若其低于 90%，通常预示可能存在潜在问题。

实例效率百分比（目标 100%）

- 实例效率百分比里的值越高越好，目标是100%，低于90% 需要重点关注
- 字段 "Buffer Hit %"：数据块在共享缓存中的命中率
- 字段 "Parse Reused %"：解析重用率
- 字段 "Plan Reused %"：计划重用率
- 字段 "Buffer NoWait %"：获取数据缓冲区 LightLock 锁无需等待的比例
- 字段 "WAL Buffer NoWait %"：获取日志缓冲区 LightLock 锁无需等待的比例
- 字段 "LightLock Hit %"：获取全部 LightLock 锁无需等待的比例
- 字段 "Cached Plan %"：执行计划来自缓存命中而不是重新生成的比例，即软解析率

效率	百分比
Buffer Hit %	87.82
Parse Reused %	100.00
Plan Reused %	99.00
Buffer NoWait %	99.95
WAL Buffer NoWait %	70.57
LightLock Hit %	99.01
Cached Plan %	98.58

3.4.6.2.1　缓存命中率

缓存命中率（Buffer Hit%）衡量了数据库在读取共享块和本地块时，从缓存中成功获取数据的比例。该值越高越好，理想情况下应为 100%。若非 100%，则说明存在部分写入操作需要从数据文件中读取，导致磁盘 IO 的发生，对性能造成一定影响。

关于缓存命中率，有两个重要概念：

- Blocks Hits：数据块成功从缓存中获取（Hit），未发生磁盘 IO

- Blocks Reads：数据块需从数据文件中读取（Read），发生了磁盘 IO

KingbaseES 中的数据块分三种类型：

- 共享块：所有会话均可访问的数据块，例如普通的关系表数据块等
- 本地块：仅限于本地会话使用的临时表数据库
- 临时块：由于 work_mem 不足，本地会话在执行排序或 Hash 操作时所使用的临时磁盘文件块

缓冲命中率相关的计算公式：

```
缓存命中率 =（共享块和本地块 Hits * 100.00）/（共享块和本地块 Hits + 共享块和本地块 Reads）
```

3.4.6.2.2　解析和计划重用率

解析和计划重用率反映了 SQL 语句执行过程中，实际发生的解析次数与执行次数的比例，以及计划生成次数与执行次数的比例。

在理想环境下，应尽量减少解析和计划的次数，而以执行次数为主导，这表明解析和计划得到了充分复用。例如，在 TPCC 测试中，客户端使用扩展 SQL 报文协议时的表现最佳。

而在最不利的情况下，解析、计划和执行的次数将趋于一致，例如当客户端采用简单 SQL 报文协议时，解析和计划几乎未被复用，导致数据库执行 SQL 效率最低。

解析计划重用率的计算公式如下：

```
解析重用率 = (1 - 解析次数 / 执行次数) * 100.00
计划重用率 = (1 - 计划次数 / 执行次数) * 100.00
```

3.4.6.2.3　执行计划缓存率

执行计划缓存是指生成执行计划时，是从缓存中获取现有执行计划（软解析，Soft Parses），还是需要新生成执行计划（硬解析，Hard Parses）。

执行计划缓存率计算公式如下：

```
执行计划缓存率 = 软解析数 * 100.00 / (软解析数 + 硬解析数)
```

信息:

关于执行计划缓存更多的信息，可以参考手册：缓存执行计划

3.4.6.3　IO 性能指标解析

KWR 报告中的 IO（磁盘 IO）性能指标从以下几个维度展示：

- 主机 IO：主机级别的 IO 指标
- 实例 IO：数据库实例级别的 IO 指标
- SQL IO：SQL 语句级别的 IO 指标
- 数据库对象 IO：数据库对象的 IO 指标

分析数据库 IO 性能问题时，应先从主机环境和数据库实例入手。若确定存在 IO 问题，则需进一步从 SQL 语句和数据库对象层面进行深入分析。

3.4.6.3.1　主机 IO

KWR 报告 [主机 IO] 从宏观视角呈现 KingbaseES 所在主机的磁盘 IO 状况，主要关注每秒磁盘读写大小等性能指标：

设备	读取次数/s	读取大小/s	读取时间(s)	写入次数/s	写入大小/s	写入时间(s)
dm-0	0.00	0 bytes	0.00	0.48	6591 bytes	0.00
sdb	0.00	0 bytes	0.00	0.45	6591 bytes	0.06
sdb3	0.00	0 bytes	0.00	0.45	6591 bytes	0.06
sdc	0.41	1674 bytes	0.21	74.72	53 MB	243.94

可通过 data_directory 参数定位 KingbaseES 数据存储路径，并借助操作系统提供的 df 和 lsblk 命令确定数据所处磁盘设备。

```
-- 显示当前数据所在目录
kingbase=# show data_directory;
      data_directory
---------------------------
 /home/kingbase/data
(1 row)
```

```
-- 查找数据所在目录的装载点
[kingbase@localhost run]$ df -h /home
Filesystem              Size  Used Avail Use% Mounted on
/dev/mapper/klas-root  391G  365G   27G  94% /

-- 查找装载点对应的磁盘
[kingbase@localhost run]$ lsblk
NAME            MAJ:MIN RM   SIZE RO TYPE MOUNTPOINT
sda               8:0    0   1.8T  0 disk
sdb               8:16   0 446.1G  0 disk
 sdb1            8:17   0   600M  0 part /boot/efi
 sdb2            8:18   0     1G  0 part /boot
 sdb3            8:19   0 444.5G  0 part
   klas-root   252:0    0 390.5G  0 lvm  /
   klas-swap   252:1    0     4G  0 lvm  [SWAP]
   klas-backup 252:2    0    50G  0 lvm
sdc               8:32   0   3.5T  0 disk /data
```

3.4.6.3.2　实例 IO

实例 IO 基于 IO 类的等待事件统计生成，涵盖 IO 类型、发生次数和数据量等指标。这些信息在 KWR 报告中按以下维度展示：

- 实例 IO 按进程类型统计
- 实例 IO 按文件类型统计
- 实例 IO 按数据库名统计
- 实例 IO 按表空间统计
- 实例 IO 按数据库对象类型统计
- Top 10 读写的数据库对象统计

比如：按进程类型统计：

进程类型	总次数	读次数/s	写次数/s	读取大小	读取大小/s	写入大小	写入大小/s	平均读写(ms)
client backend	4390817	762.18	12.93	3656 MB	6097 kB	62 MB	103 kB	0.01
walwriter	87247	0.00	142.10	0 bytes	0 bytes	682 MB	1137 kB	0.06
ksh writer	157	0.00	0.00	16 kB	27 bytes	24 kB	40 bytes	0.00
kwr collector	15	0.02	0.00	120 kB	200 bytes	0 bytes	0 bytes	0.18
autovacuum launcher	7	0.01	0.00	41 kB	68 bytes	0 bytes	0 bytes	0.05

以及按文件类型统计：

文件类型	总次数	读次数/s	写次数/s	读取大小	读取大小/s	写入大小	写入大小/s	平均读写(ms)
Data File	4383058	762.22	0.01	3656 MB	6098 kB	56 kB	93 bytes	0.01
WAL File	87251	0.00	142.10	0 bytes	0 bytes	682 MB	1137 kB	0.06
Transaction File	7934	0.00	12.92	0 bytes	0 bytes	62 MB	103 kB	0.09

主要数据来源是：kwr_snap_inst_io。

3.4.6.3.3　SQL IO

SQL IO 通过跟踪 SQL 语句执行过程中涉及的共享块、本地块和临时块读写情况，以及 WAL 日志的读写操作，并关联到具体 SQL 语句（通过 queryid）进行统计分析。

KWR 报告 [IO 分析] 从整体层面展示这些 IO 操作每秒的读写情况：

统计项	每秒读写	每秒读	每秒写
Shared Blocks	7136.82	762.15	6374.67
Shared Blocks(MB)	55.76	5.95	49.80
Local Blocks	0.06	0.03	0.03
Local Blocks(MB)	0.00	0.00	0.00
Temp Blocks	0.00	0.00	0.00
Temp Blocks(MB)	0.00	0.00	0.00
WAL Size(MB)	-	-	0.90

同时，可通过 kwr_snap_sql_io 快照表在 SQL 语句级别查看每条语句的具体 IO 消耗情况。

```
kingbase=# SELECT snap_id,shared_blks_read,shared_blks_written,blk_read_time,blk_write_time FROM perf.kwr_
snap_sql_io LIMIT 10;
 snap_id | shared_blks_read | shared_blks_written | blk_read_time | blk_write_time
---------+------------------+---------------------+---------------+----------------
       1 |           481414 |              247629 |       6284359 |        4356914
       1 |           542845 |              277324 |       4735116 |        3006601
       1 |             2145 |               10130 |         22026 |          44815
       1 |            60733 |               30531 |       1152259 |         842139
       1 |            76612 |               39531 |        944586 |         940880
       1 |            22478 |               21614 |        209299 |         228643
       1 |            93176 |               47927 |        701539 |         665430
       1 |            44739 |               26248 |        376357 |         345169
       1 |            52863 |               26831 |        419423 |         499019
       1 |           118012 |               60469 |        809918 |         845736
(10 行记录)
```

3.4.6.3.4　数据库对象 IO

数据库对象 IO 允许从关系表和索引等数据库对象层面查看物理读（Reads）、逻辑读（Hits）页数及数据页缓存命中率等性能指标。

这些统计信息可在 KWR 报告 [数据库对象统计] 的以下子报告中找到：

- 按顺序扫描页数排序的关系表
- 按逻辑读页数排序的关系表
- 按物理读页数排序的关系表
- 按命中率排序的关系表
- 按逻辑读页数排序的索引
- 按物理读页数排序的索引
- 按命中数排序的索引

比如：[按物理读页数排序的关系表] 展示了物理读最多的关系表：

模式	表	堆		索引		Toast		Toast 索引	
		页数	占比(%)	页数	占比(%)	页数	占比(%)	页数	占比(%)
public	bmsql_order_line	1733730	39.56	644545	14.71	0	0.00	0	0.00
public	bmsql_stock	714616	16.31	20705	0.47	0	0.00	0	0.00
public	bmsql_customer	426612	9.74	32160	0.73	0	0.00	0	0.00
public	bmsql_oorder	89847	2.05	273625	6.24	0	0.00	0	0.00
public	bmsql_district	204474	4.67	50	0.00	0	0.00	0	0.00
public	bmsql_history	85863	1.96	0	0.00	0	0.00	0	0.00
public	bmsql_warehouse	81593	1.86	5	0.00	0	0.00	0	0.00

3.4.6.4　内存统计指标解析

3.4.6.4.1　KWR 中的内存统计

KWR 的内存统计主要包括主机内存和数据库共享内存。

[主机内存]- 展示快照开始和结束时刻主机总内存、已用内存和空闲内存等：

统计项	总内存	已用内存	空闲内存	总虚拟内存	已用虚拟内存	空闲虚拟内存	共享内存
Begin Snap	478 GB	28 GB	191 GB	4096 MB	436 MB	3660 MB	53 GB
End Snap	478 GB	27 GB	129 GB	4096 MB	436 MB	3660 MB	87 GB

[内存统计]- 展示快照开始和结束时刻 KingbaseES 共享内存占主机内存的比例：

统计项	开始大小	结束大小
Host Memory	478 GB	478 GB
SGA	165 GB	165 GB
% Host Mem used for SGA	34.51	34.51

[Top 10 共享内存统计]- 分解 KingbaseES 共享内存：

共享内存	开始大小	开始占比(%)	结束大小	结束占比(%)
(Total)	165 GB	100.00	165 GB	100.00
Buffer Blocks	160 GB	97.02	160 GB	97.02
Buffer Descriptors	1280 MB	0.76	1280 MB	0.76
clog	1034 MB	0.61	1034 MB	0.61
XLOG Ctl	1025 MB	0.61	1025 MB	0.61
Buffer IO Locks	640 MB	0.38	640 MB	0.38
Checkpointer Data	480 MB	0.28	480 MB	0.28
Checkpoint BufferIds	400 MB	0.24	400 MB	0.24
WAL Cxt	64 MB	0.04	64 MB	0.04
sys_ksh	25 MB	0.01	25 MB	0.01
plsql_plprofiler state	4688 kB	0.00	4688 kB	0.00

3.4.6.5　数据库对象指标解析

KWR 报告 [数据库对象统计] 章节提供了当前库中的关系表、索引、函数统计指标。

KWR 中数据库对象指标包括了以下报告：

- 按顺序扫描页数排序的关系表

- 按逻辑读页数排序的关系表
- 按物理读页数排序的关系表
- 按 DML 行数排序的关系表
- 按命中率排序的关系表
- 按逻辑读页数排序的索引
- 按物理读页数排序的索引
- 按命中数排序的索引
- 存在未使用的索引
- 按执行时间排序的函数
- 按执行次数排序的函数

比如通过 [按 DML 行数排序的关系表] 查看 DML 操作总数最多的关系表：

模式	表	插入	更新	删除	HOT 更新
public	bmsql_order_line	79130585	70377918	0	25585457
public	bmsql_stock	0	79130585	0	66655498
public	bmsql_district	0	15490681	0	11783507
public	bmsql_oorder	7921198	7038200	0	0
public	bmsql_new_order	7921198	0	7038200	0
public	bmsql_customer	0	14607683	0	9922813
public	bmsql_history	7569483	0	0	0
public	bmsql_warehouse	0	7569483	0	7229967

信息:

KWR 报告中，仅仅数据库对象报告是库级统计，其他性能指标都是全实例级别的。

3.5　KDDM 报告

3.5.1　KDDM 概述

KDDM 是一项针对 KingbaseES 数据库系统提供自动化性能诊断与优化建议的重要工具。该系统基于从 KWR 快照中采集的性能指标和数据库时间模型（DB Time），通过分析等待事件、IO 操作、网络传输、内存使用、SQL 执行时长等关键指标，自动生成一系列性能优化建议。借助 KDDM 报告，能够实现对数据库性能问题的快速定位与调优。

3.5.2　KDDM 报告生成

基于 KWR 快照，生成 KDDM 报告：

```
kingbase=# SELECT snap_id,sess_count FROM perf.kwr_snapshots;
 snap_id | sess_count
---------+------------
       1 |        102
       2 |        102
       3 |        102
       4 |        102
(4 行记录)
kingbase=# SELECT * FROM perf.kddm_report(1,2);
```

3.5.3　KDDM 报告解析

1. **整体目录结构**

KDDM 报告首先展示建议目录，其内容大致分为以下四个主要部分：

- 数据库时间分解：对总体数据库时间进行多维度拆解分析
- 等待相关的建议：基于等待事件的高发情况给出优化建议
- CPU 相关建议：结合 CPU 使用率提供性能调优方案
- 完整 SQL 列表：列出所有子报告中涉及的 SQL 语句文本

值得注意的是，第二部分和第三部分的呈现顺序取决于等待事件或 CPU 占用率的高低。具体而言，若等待事件占比显著，则等待相关建议将优先展示；反之，则以 CPU 相关建议将优先展示。以下是示例：

```
[建议列表]：
    数据库时间分解
    等待相关建议
        TOP 等待事件建议
        LightLock 类等待事件
        IO 类等待事件
        Client 类等待事件
    CPU 相关建议
        TOP SQL 建议
        优化堆页面裁剪建议
    完整 SQL 列表
```

KDDM 目录后面是具体的子报告内容：

```
[数据库时间分解]：XXX
[TOP 等待事件建议]：XXX
```

```
...
[完整 SQL 列表]：XXX
```

备注:

在 KDDM 报告中，如果 SQL 语句显示为“Unknown”，则表示该语句的等待事件统计值被高频采集到了，但其 SQL 语句字符串等数据未被采集到。这是因为 sys_stat_statements 的容量过小和性能影响导致无法捕获这些低频统计数据。若此类情况持续出现，可通过适当增大 `sys_stat_statements.max` 的值来解决。但需注意，这可能会影响系统性能，建议在调整前进行充分的权衡和测试。

---

2. **数据库时间分解**

```
[数据库时间分解]：

Event                                      DB Time  Percent  QueryID
-------------------------------------------------------------------------------
OnCpu                                       4.93 %
  Cpu                                       4.93 %    100 %
    COMMIT                                  4.93 %    100 %   3694949039461716331
LightLock                                  76.72 %
  WALWriteLock                             74.15 %  96.65 %
    COMMIT                                 73.62 %  99.29 %   3694949039461716331
  wal_insert                                2.34 %   3.05 %
Lock                                       16.93 %
  transactionid                            14.72 %  86.92 %
    UPDATE bmsql_warehouse SET w_ytd =      9.82 %  66.70 %   5333761347722434871
    UPDATE bmsql_district SET d_ytd = d     2.38 %  16.17 %   7717458256612817612
    SELECT d_tax, d_next_o_id ,CTID,XMI     2.29 %  15.53 %   2655240597523385636
  tuple                                     2.20 %  13.02 %
    UPDATE bmsql_warehouse SET w_ytd =      2.02 %  91.48 %   5333761347722434871
IO                                          1.42 %
```

数据库时间分解报告从整体上将总的数据库时间按等待事件和 CPU 时间拆解。所谓 CPU 时间就是数据库时间减去等待时间，可以将 CPU 时间看成一个特殊的等待事件，其分类固定为 OnCPU。

数据库时间报告按照三层目录展示：

```
等待事件分类
    等待事件
        SQL 语句   数据库时间百分比   占当前等待事件百分比   QueryID
```

从上述报告中可以看出，COMMIT 语句（3694949039461716331）占了大部分的数据库时间，其中 CPU 时间 4.93%，WALWriteLock 等待时间 73.62%，即该语句的等待时间是主要性能瓶颈，需要对这个语句进行优化。

通过此报告可以从宏观层面快速识别出最需要优化的 SQL 语句（COMMIT）及其对应的等待事件类型（WALWriteLock）。

3. **等待相关建议**

```
[TOP 等待事件建议]:

等待事件:
---------
    WALWriteLock:          74.15 %     LightLock
    transactionid:         14.72 %     Lock
    wal_insert:             2.34 %     LightLock
    tuple:                  2.20 %     Lock

建议动作:
---------
    分析具体的等待事件产生的原因。

参考信息:
---------
    WALWriteLock:正等待 WAL 缓冲区被写入到磁盘
    关联 SQL:
    queryid              query                                dbtime  percent
   ---------------------------------------------------------------------------
    3694949039461716331  COMMIT                              73.62 %  99.29 %
   -5332993246613577513  INSERT INTO bmsql_order_line ( ol    0.10 %   0.13 %
   -3903412935930362358  UPDATE bmsql_stock SET s_quantity    0.09 %   0.12 %

    transactionid:正等待一个事务结束
    关联 SQL:
    queryid              query                                dbtime  percent
   ---------------------------------------------------------------------------
    5333761347722434871  UPDATE bmsql_warehouse SET w_ytd     9.82 %  66.70 %
    7717458256612817612  UPDATE bmsql_district SET d_ytd =    2.38 %  16.17 %
    2655240597523385636  SELECT d_tax, d_next_o_id ,CTID,X    2.29 %  15.53 %
```

该报告展示排名靠前的等待事件及其对应的 SQL 语句。

从数据中可以看到，COMMIT 语句的 WALWriteLock 等待时间占了数据库总时间的 73.62%。

4. WAL 文件写锁冲突建议

若在快照期间检测到大量 WALWriteLock 等待事件，KDDM 将自动生成 [WAL 文件写锁冲突建议]：

```
[WAL 文件写锁冲突建议]:

建议依据:
---------
    当 wal 日志写较为频繁时，如果仍有大量事务时会导致 WALWriteLock 等待事件较高，
可以减少提交次数增加单位时间内事务的提交量。

建议动作:
---------
    1、使用分组提交:
        优化 commit_delay 参数，当前值: 0，建议值: 10。
优化 commit_siblings 参数，当前值: 5，建议值: 16。
减少写 wal 日志次数，提升效率，但是每次提交事务会浪费 commit_delay
的时间。

    2、使用异步提交（有风险）:
        设置 synchronous_commit = off。
        使用异步提交可以在提交事务的时候不等 wal 日志写入磁盘就返回，有丢事务的风
        险，需要谨慎应用。

    3、关闭整页写功能（有风险）:
        设置 full_page_writes = off。
        关闭 full_page_write，在系统崩溃时，可能有丢失数据的风险，如果服务器硬件
        特质特殊可以减少部分页面写入的风险，比如支持电池供电的磁盘控制器，或者文
        件系统支持原子操作，并且知道写入风险在一个可接受的范畴，可以关闭此参数。

参考信息:
---------
    前台等待事件:
        WALWriteLock: 74.15 %
    参数信息:
        commit_delay = 0
        commit_siblings = 5
        synchronous_commit = on
        full_page_writes = on
```

此类建议通常包含以下三个部分：

- 建议依据：详细解释为何需要采取当前优化措施
- 建议动作：提供针对性的解决方案或配置调整方案
- 参考信息：跟当前建议相关的一些数据库时间百分比、配置参数等

5. **完整 SQL 列表**

```
[完整 SQL 列表]:

Query Id:3694949039461716331
COMMIT

Query Id:-5332993246613577513
INSERT INTO bmsql_order_line ( ol_o_id, ol_d_id, ol_w_id, ol_number, ol_i_id,
ol_supply_w_id, ol_quantity, ol_amount, ol_dist_info) VALUES ($1, $2, $3, $4,
$5, $6, $7, $8, $9)

Query Id:2655240597523385636
SELECT d_tax, d_next_o_id ,CTID,XMIN FROM bmsql_district WHERE d_w_id = $1 AND
d_id = $2 FOR UPDATE

Query Id:-6247288520632161420
```

[完整 SQL 列表] 报告展示 KDDM 报告中出现的全部 QueryID 对应的 SQL 语句字符串，便于进一步分析和优化。

3.5.4　KDDM 建议应用

遵循 KDDM 提供的 [WAL 文件写锁冲突建议]，对数据库配置参数进行了相应的调整：

```
commit_delay = 10
commit_siblings = 16
synchronous_commit = off
full_page_writes = off
```

在完成上述配置调整后，重新执行了 TPCC 测试，并生成了 KWR 快照和 KDDM 报告。结果显示，COMMIT 语句已不再是最耗时的 SQL 操作，WALWriteLock 等待事件占数据库总时间的比例也显著下降至 57.33%：

```
[数据库时间分解]:

Event                                      DB Time  Percent  QueryID
-------------------------------------------------------------------------------
OnCpu                                      18.53 %
  Cpu                                      18.53 %    100 %
    SELECT count(*) AS low_stock FROM (     3.11 %  18.31 %   3889337793880648542
    SELECT s_quantity, s_data, s_dist_0     2.79 %  16.45 %   4383288571379112176
    SELECT no_o_id FROM bmsql_new_order     2.02 %  11.91 %  -6247288520632161420
LightLock                                  61.49 %
  WALWriteLock                             57.33 %  93.22 %
    SELECT count(*) AS low_stock FROM (    16.94 %  29.55 %   3889337793880648542
    SELECT s_quantity, s_data, s_dist_0    13.82 %  24.11 %   4383288571379112176
    SELECT o_id, o_entry_d, o_carrier_i     6.25 %  10.90 %   -994111915640741594
  CheckpointerCommLock                      2.82 %   4.59 %
  buffer_content                            1.23 %   1.99 %
IO                                         14.92 %
  DataFileRead                             10.63 %  71.23 %
    SELECT count(*) AS low_stock FROM (     5.53 %  52.06 %   3889337793880648542
    SELECT s_quantity, s_data, s_dist_0     3.74 %  35.16 %   4383288571379112176
  DataFileWrite                             3.56 %  23.87 %
    SELECT count(*) AS low_stock FROM (     1.10 %  30.86 %   3889337793880648542
Lock                                        5.04 %
  transactionid                             3.34 %  66.28 %
    DELETE FROM bmsql_new_order WHERE n     1.10 %  32.85 %  -1376825226900098240
  extend                                    1.55 %  30.72 %
```

此外，KDDM 还提供了 [优化共享缓存建议]：优化 shared_buffers 参数，当前该参数的值为 128MB，建议调整至 3 GB。

继续根据新的建议进行性能调优，直至 KDDM 不再提供可用的优化建议或测试结果达到预期目标为止。

3.6　KWR DIFF 报告

3.6.1　KWR DIFF 概述

本报告基于两组快照数据，分别构建相应的数据库时间模型，并以此为基础对比分析系统性能差异。

3.6.2　KWR DIFF 报告生成

生成 KWR DIFF 报告的 SQL 函数是：

- perf.kwr_diff_report：基于两组快照生成 KWR DIFF 报告
- perf.kwr_diff_report_to_file：生成 DIFF 报告到指定文件

生成报告示例：

```
SELECT * FROM perf.kwr_diff_report(1, 2, 3, 4);
SELECT * FROM perf.kwr_diff_report_to_file(1, 2, 3, 4, 'diff.html');
```

3.6.3　KWR DIFF 报告解析

KWR DIFF 报告与 KWR 报告的目录结构和内容基本一致，具体内容不再详细描述。

下面以 [负载分析] 报告来如何解析 DIFF 报告：

负载分析

统计项	总值			每秒		
	1st	2nd	% Diff	1st	2nd	% Diff
DB Time(s)	24070.55	23391.79	-2.82	95.52	97.06	1.61
DB CPU(s)	1185.61	4334.18	265.57	4.70	17.98	282.55
Foreground Wait Time(s)	22884.94	19057.61	-16.72	90.81	79.08	-12.92
WAL Size(MB)	8694.46	4527.79	-47.92	34.50	18.79	-45.54
Blocks Hit	65295943	347489972	432.18	259110.88	1441867.10	456.47
Blocks Read	9052584	54495347	501.99	35922.95	226121.77	529.46
Blocks Dirty	3910403	15738627	302.48	15517.47	65305.51	320.85
Blocks Write	3981575	16045564	303.00	15799.90	66579.10	321.39
Blocks Read Size(MB)	70723.31	425744.90	501.99	280.65	1766.58	529.46
Blocks Write Size(MB)	31106.05	125355.97	303.00	123.44	520.15	321.38
Parse Calls	10258	13360	30.24	40.71	55.44	36.18
Plan Calls	142417	173079	21.53	565.15	718.17	27.08
Execute Calls	10045014	35219941	250.62	39861.17	146140.83	266.62
Tuples Return	195018448	809716465	315.20	773882.73	3359819.36	334.15
Tuples Fetch	107986670	517771117	379.48	428518.53	2148427.87	401.36
Tuples Insert	2054759	7214612	251.12	8153.81	29936.15	267.14
Tuples Update	3892673	13609889	249.63	15447.12	56472.57	265.59
Tuples Delete	141916	491862	246.59	563.16	2040.92	262.41
Commits	322999	1135297	251.49	1281.74	4710.78	267.53
Rollbacks	29833	104731	251.06	118.38	434.57	267.10
Transactions	352832	1240028	251.45	1400.13	5145.34	267.49
Logon	0	0	0.00	0.00	0.00	0.00
Logoff	0	0	0.00	0.00	0.00	0.00
DeadLock	0	0	0.00	0.00	0.00	0.00

1、字段说明

- 1st：第一组快照的统计值（本例中指快照 1 和 2 之间的统计值）
- 2nd：第二组快照的统计值（本例中指快照 3 和 4 之间的统计值）
-
- 总值：由于两组快照的时间范围不同（1st 持续时间为 4.20 分钟，2nd 持续时间为 4.02 分钟），因此总值比较在此处无实际意义
- 每秒：1st 和 2nd 两组快照的统计值分别除以各自快照时间

2、内容比较

从 [负载分析] 报告中提取“每秒”相关的数据，并对部分统计项进行简化，便于更清晰地观察变化：

- DB Time(s)：每秒 DB Time 变化不大

- DB CPU(s)：每秒 DB CPU 因为等待时间少了，所以提升了 265.57%
- Foreground Wait Time(s)：前台等待时间整体下降 16.72%
- WAL Size(MB)：因为应用了写 WAL 日志的优化，每秒写 WAL 量下降 47.92%
- Transactions：每秒事务数（TPS）由每秒 352832 提升到 1240028，提高了 251.45%

TPS 值的变化量也可以跟 benchmarksql 的测试（5 分钟）结果对比：

第一次：Transaction Count = 352832 = 352832 / (5 * 60) = 1400.13

第二次：Transaction Count = 1240028 = 1240028 / (5 * 60) = 5145.34

第二次比第一次提升：(5145.34 - 1400.13) * 100 / 1400.13 = 267.49

3.7　KSH 报告

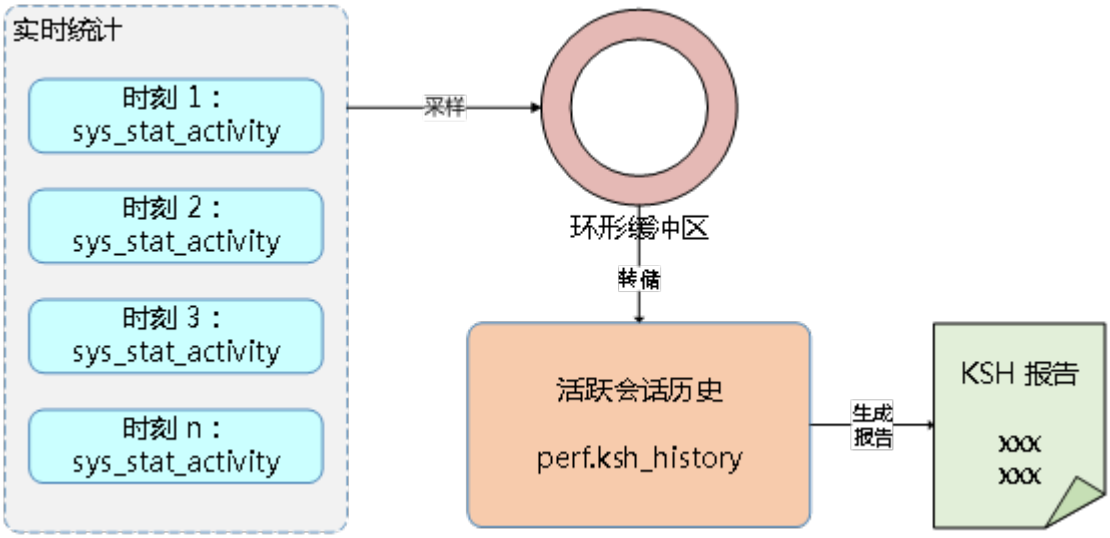
3.7.1　KSH 概述

KSH（KingbaseES Session History）活跃会话历史是 KWR 性能调优体系的重要成员。不同于 KWR 使用的累积式统计，KSH 使用的是采样式统计结果。

	KWR	KSH
定位	周期时间累积差异，查询特定时间段的累积变化	识别系统短暂变化，秒级时刻的实时值
原理	以累积式计数反馈系统在一定周期内的整体负载和瓶颈	基于定时的采样方式进行会话和数据收集
场景	本次系统升级后有哪些整体改进，当前时段与基线相比指标如何变化。	当前或历史某个时点，发生了什么样的异常，系统在执行/运行什么任务
数据留存	默认 1 小时采集，快照保留 8 天，超过 8 天后自动删除	默认 1 秒，实时数据保留 1 小时，过后以 1/10 的比例存入历史数据

因为 KWR 默认生成快照时间是 1 小时，如果快照期间出现了短暂的性能瓶颈，比如 5 分钟内某个等待事件高频出现，这些等待事件最终会被平均到 1 小时内，导致 KWR 报告中可能会无法观测到该等待事件。所以 KSH 能够解决突发的性能问题。

KSH 通过后台工作进程 ksh collector 和 ksh writer 每秒采集活跃会话的状态信息，存于基于共享内存的环形缓冲区。每 10 分钟或者环形缓冲区容量达到 80% 以上的时候，将环形缓冲区的统计数据以一定比例（比如 1/10）存储到 KSH 快照表 ksh_history_data。KSH 基于快照表生成指定时间段的 KSH 报告。



3.7.2　KSH 报告生成

3.7.2.1　配置 GUC 参数

要开启 KSH 采集，至少需要配置以下 GUC 参数：

```
track_activities = on                # KSH 依赖该开关，默认开启
sys_stat_statements.track = 'top'    # KSH 依赖该开关，也可以是 'all'
sys_kwr.collect_ksh = on             # 开启自动采集 KSH
```

更多的 GUC 参数，可以参考《KingbaseES 插件参考手册》的“KSH 推荐参数配置”章节。

在使用 KSH 功能之前，需要确认 sys_kwr 插件已经在目标库（由 sys_kwr.database 指定，默认 kingbase 库）已经被创建，后续生成 KSH 报告等操作也在该库上进行。

3.7.2.2　KSH 内部视图和关系表

KSH 内部使用以下关系表和视图：

- ksh_history_data：基于 sys_stat_activity 视图的 KSH 采样统计表
- ksh_history 视图：基于 ksh_history_data 的视图
- ksh_statements：存储 SQL 语句字符串

KSH 采集的数据量跟业务的负载尤其是客户端活跃会话数密切相关，sys_kwr.history_days 默认保留八天的数据量，如果 ksh_history_data 关系表存储空间超过 10GB，可以将该 GUC 改为三天或者一天，相应 KWR 的快照保留天数也会同时变更。

3.7.2.3　生成 KSH 报告

有 4 个函数可以用来生成 KSH 报告：

- perf.ksh_report：根据指定时间区间生成 KSH 报告
- perf.ksh_report_to_file：根据指定时间区间生成 KSH 报告到指定文件
- perf.ksh_report_by_snapshots：指定快照号来生成 KSH 报告
- perf.ksh_report_to_file_by_snapshots：指定快照号来生成 KSH 报告到指定文件

以下是示例：

```
SELECT * FROM perf.ksh_report();
SELECT * FROM perf.ksh_report_by_snapshots(3, 4);
SELECT * FROM perf.ksh_report_to_file(file_path => '/data/ksh.txt');
SELECT * FROM perf.ksh_report_to_file_by_snapshots(3, 4, file_path => '/data/ksh.txt');
```

3.7.3　KSH 报告解析

KSH 有报告头和报告主体组成。

报告头主要包括：版本信息、主机概要信息和 KSH 采样信息三个子报告。

报告主体包括以下报告：

- Top 用户事件
- Top 后台事件
- Top 数据库
- Top 服务/模块
- Top PL/SQL 过程
- Top 简单查询
- 等待事件高的 Top SQL
- Top 会话
- Top 客户端
- Top 并行 SQL 等待事件
- Top 阻塞会话事件
- Top 数据库对象
- Top 重量级锁等待事件
- Top 轻量级锁等待事件

- Top SQL 命令类型
- Top 执行阶段
- 分段时间等待事件统计
- 完整 SQL 列表

3.7.3.1　KSH 报告头解析

报告头的采样信息：

处理器型号	处理器数	核心数	处理器槽位	共享内存	KSH共享内存
HUAWEI,Kunpeng 920	128	128	2	394 MB	25 MB

名称	采样信息	数据来源
Analysis Begin Time	15:03:04	Memory
Analysis End Time	15:07:04	Memory
Elapsed Time	240	
Sample Count	23543	
Avg Active Sessions	98.10	
Avg Active Sessions per CPU	0.77	

以下是关键字段解析：

- Elapsed Time：采样时间，即本报告采样的起止时间范围，单位为秒数

  计算方法：采样结束时间 - 采样开始时间

- Sample Count：总采样数，即采样期间采样到的活跃会话数量

  计算方法：perf.ksh_history_data 中 ts 在采样起止时间范围内的记录数

- Avg Active Sessions：平均活跃会话数

  计算方法：总采样数 / 采样时间，即：23543 / 240.0 = 98.10。

  该指标说明平均每秒有 98.10 个活跃会话。

- Avg Active Sessions per CPU：每 CPU 平均活跃会话数

  计算方法：平均活跃会话数 / CPU 核心数，即：98.10 / 128 = 0.77

  该指标说明采样期间有 77% 的 CPU 在忙碌（活跃会话）。

3.7.3.2　KSH 报告主体解析

以 [Top 轻量级锁等待事件] 报告为例，

Top 轻量级锁等待事件

- 显示轻量级锁信息
- 字段“活动占比”：锁数量除以总的采样次数
- 字段“平均会话数”：锁数量除以总的采样时间

锁名称	锁数量	锁占比	活动占比	平均会话数
WALWriteLock	13490	91.87	57.30	56.21
CheckpointerCommLock	870	5.93	3.70	3.63
buffer_content	297	2.02	1.26	1.24
wal_insert	22	0.15	0.09	0.09
buffer_io	2	0.01	0.01	0.01
buffer_mapping	1	0.01	0.00	0.00
XidGenLock	1	0.01	0.00	0.00

- 我们以 WALWriteLock 为例，其采样数 13490，全部锁的采样数是：（13490 + 870 + 297 + 22 + 2 + 1 + 1）= 14683
- **锁占比（事件占比）**：某个采样到的事件，占该该类全部事件的比例

  计算方法：WALWriteLock 数量 * 100.0 / 全部锁采样数，即：13490 * 100.0 / 14683 = 91.87，说明该类型轻量级锁占了全部轻量级锁的 91.87%。

- **活动占比**：某个采样到的事件，占全部采样数量的比例

  计算方法：锁活动数量 * 100.0 / 总采样数，即：13490 * 100.0 / 23543 = 57.30，说明全部采样中有 57.30% 的情况采样到了 WALWriteLock 锁活动。

- **平均会话数**：采样时间内，平均每秒采样到该事件的数量

  计算方法：锁活动数量 / 采样总时间，即：13490 / 240.0 = 56.21，说明采样期间内，平均每秒采样到了 56.21 个 WALWriteLock 锁活动。

第4章　性能问题调优

本章节包含以下内容：

- 整体过程
- 定义性能目标
- 性能诊断，定位资源瓶颈
- 性能诊断，定位优化点
- 性能调优，解决性能问题
- 性能诊断与优化经验

4.1　整体过程

KingbaseES 实例性能调优的整体过程如下：

1. 定义性能目标

   从用户那里获得关于性能问题范围的直接反馈，据此定义要达成的性能目标，如 QPS、TPS 或响应时间等。

   准确确定应用场景，包括相关的测试用例以及测试方法。

   参见定义性能目标

2. 测试并检查资源使用情况，定位性能瓶颈，找出性能问题的直接原因

   在实施测试过程中，需要收集从测试程序到数据库之间完整链路的操作系统资源负载信息、数据库负载信息、性能指标信息。通过这些信息，确定性能瓶颈在整个链路中的具体位置（如：应用、数据库、中间件等），以及对应的瓶颈资源（如：CPU、IO、网络、等待等）。

   备注:

需注意的是，有时性能瓶颈可能出现在测试服务器或应用服务器端，这种情况下仅在数据库端进行调优可能难以达到理想效果。

建议采用 NMON 工具采集操作系统资源负载信息，并使用 KWR 工具获取数据库负载信息。

具体参见性能诊断，定位资源瓶颈

3. 检查资源使用的分布情况，找出性能问题的根本原因

   在定位到性能瓶颈后，还需进一步分析各项软硬件资源的使用分布情况，特别是针对瓶颈资源进行详细排查。通过识别资源使用的不当情况或确认其无法满足当前性能指标的要求，从而找出导致性能问题的根本原因，以确保优化措施能够产生最大化的效益。

   针对数据库相关问题，建议使用 KDDM 工具进行自动化分析。若该工具未能提供有效建议，可转而手动分析 NMON、KWR 等采集到的数据。

   参见性能诊断，定位优化点

4. 对于不合理的资源使用，针对性给出调优手段

   在确定了性能问题的根本原因后，应合理运用数据库提供的各项优化手段解决问题。例如，当瓶颈出现在数据库服务器的 CPU 计算能力上时，应对 TOP SQL 语句做 SQL 调优。

   参见性能调优，解决性能问题

5. 除了 2、3、4 自顶向下的分析之外，也可以考虑检查一些常见的优化经验

   参见性能诊断与优化经验

6. 提出需要进行的优化手段和实施优化手段后的预期结果。然后，应用这些优化手段并监测应用程序的性能表现，包括性能指标

7. 检查是否已解决步骤 1 提出的问题，并确认是否达到了预期的性能目标。若未解决，则需重复执行步骤 2、3、4、5，继续进行分析与调优

4.2　定义性能目标

在尝试给出优化方案之前，必须充分了解性能调优的目标和问题的性质。缺乏准确理解的情况下，实施有效优化措施的可能性较低。在这一阶段收集的信息有助于确定下一步要采取的措施。

需要收集的信息包括：

1. 确定性能指标

   可接受性能指标是什么？如响应时间、吞吐量的要求

备注:

- 对于定义性能目标，最低的目标是满足应用需求。若已达标，则可能无需进行性能诊断和调优。

- 对于 pk 的场景，性能目标是性能越高越好，直到无法再调优或者很难调优。

2. 确定问题涉及的范围

   由于执行的缓慢影响到了什么？例如，整个实例是否缓慢？是否影响到特定的应用、程序、特定的操作还是单个用户？

3. 确定问题发生的时间

   这个问题只在高峰时段出现吗？一天中的性能表现会变差吗？在数月或数个星期的时间段内是逐渐缓慢还是突然缓慢？

4. 量化性能差距

   这有助于评估问题的严重程度，并作为优化工作是否生效的衡量依据。需确定可测量的性能指标，比较当前状态与预期状态之间的差异。

   以某电商业务为例，用户涉及 4 个典型业务场景：查看商品、下单、付款、查看订单状态。使用 jmeter 来录制并模拟 1000 个用户同时并发的场景，期望 TPS 能达到 1000，并确保 90% 的响应时间分别控制在 100ms、300ms、300ms、100ms 以内。

业务	目标 TPS	当前 TPS	目标响应时间	当前响应时间
商品	5000	1232	100ms	332ms
下单	1000	555	300ms	511ms
付款	1000	737	300ms	443ms
查看订单状态	5000	1237	100ms	322ms

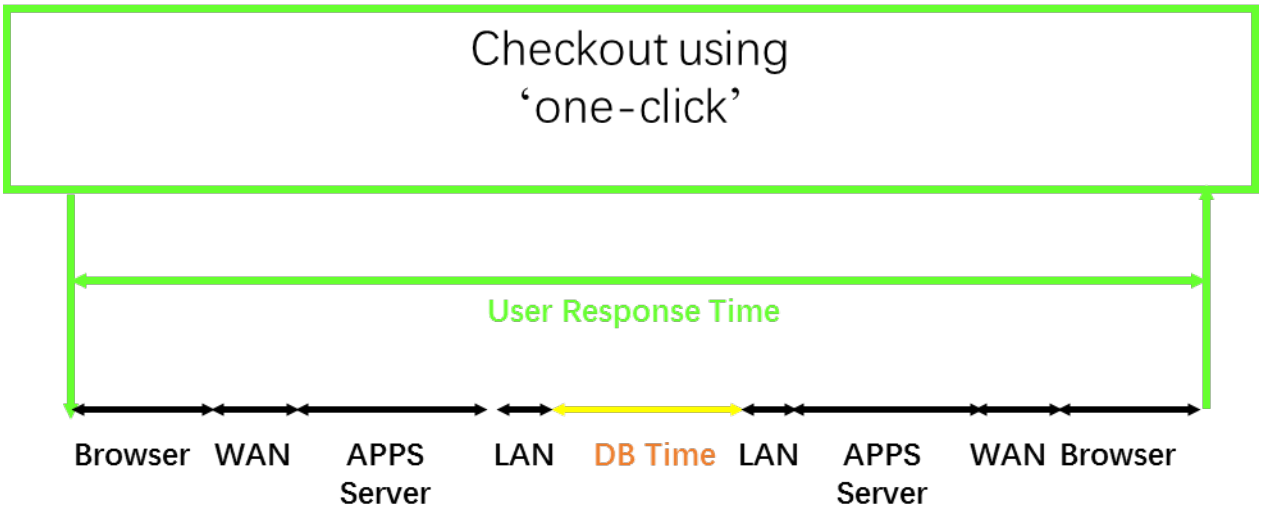
5. 是否存在性能达标的基线，如果是确定发生了哪些改变

   确定自性能达标以来发生了哪些变化。这可能会迅速缩小潜在原因的范围。例如，操作系统软件、硬件、应用程序软件或 KingbaseES 数据库版本是否已升级？系统中是否加载了更多的数据，或者数据量和用户数量是否增加？

4.3　性能诊断，定位资源瓶颈

性能调优的核心目标在于定位性能问题产生的根本原因，即性能瓶颈。准确识别性能瓶颈能够显著提升优化效果。

性能问题是由于某些系统资源争用或耗尽所导致的结果。测试程序/用户终端与数据库之间通常会部署多台主机设备，包括但不限于测试主机、负载均衡设备、应用服务器和数据库服务器等。



以 OA 应用为例，如上图所示，用户的响应时间包含了：数据库的响应时间、应用处理时间、浏览器渲染时间、各组件之间的网络传输时间。因此，定位性能瓶颈，需要收集整个链条上所有软硬件资源的性能监控数据。

定位性能瓶颈的方法包括：操作系统资源利用率、响应时间测量等。

4.3.1　资源利用率瓶颈定位法

以资源利用率方法为例，分析链路上所有所有组件的资源利用率：

- CPU 利用率接近 100%
- IO 利用率接近 100%
- 内存不够，开始写 Swap
- 网络流量达到带宽上限、网络传输存在大量错误

如果确定是数据库的问题，则建议使用 KWR/KSH/KDDM 结合 NMON 来进一步确定数据库内部资源瓶颈，重点关注时间模型相关报告。

- 如果 DB CPU 占 100%，则表示瓶颈在 CPU 计算上
- 如果等待事件超过了 5%，则应该关注等待事件相关的资源冲突，如 IO、锁、IPC 等

4.3.2　响应时间测量瓶颈定位法

此方法相对复杂，需要明确用户响应时间的动作和数据库响应时间的动作的对应关系，然后换算数据库响应时间的占比。

以 jmeter 压测单条 SELECT 语句为例，压测完成后，jmeter 报告会输出单条 SQL 的响应时间，数据库 KWR 的 TOP SQL 报告会有数据库单条 SQL 的平均响应时间，如此则可以看出数据库响应时间在用户响应时间中的占比。

如果用户的一个响应时间对应的动作是一个事务或者多个事务，则在数据库内部的对应关系会相对复杂，需要根据实际情况来想办法映射。

4.4　性能诊断，定位优化点

对于可以确定为数据库瓶颈的情况，还需要进一步分析该瓶颈所涉及资源的具体使用分布情况，并针对其中不合理的资源使用情况进行调优。

在分析资源分布情况时，可采用以下方法：

- 时间模型分析（KWR 时间模型章节）
- 等待事件分析（KWR 等待事件相关章节）
  - IO 等待分析
  - 锁等待分析
  - 网络等待分析
  - 其他等待分析
- CPU 分布分析（KWR 的 TOP SQL 章节）
  - 耗时语句时间分布
  - SQL 执行计划分析

除了人工分析 KWR 报告外，建议使用 KDDM 报告来直接给出建议。

有关资源分布情况的详细定位方法将在操作系统性能诊断 和数据库系统性能分析与优化 以描述。

4.5　性能调优，解决性能问题

在定位到性能优化点之后，可以采取以下措施进行性能调优：

1. 数据库参数优化

   针对不同应用场景，调节相关内核参数以实现优化。例如：对于频繁访问大量数据的情况，建议增大 shared_buffers 参数值；在涉及临时文件写入的操作中，可适当调大 work_mem 参数。

   上述内存相关参数见配置管理手册的资源消耗章节。

2. SQL 优化

   可以考虑创建合适的索引、使用分区、通过 HINT 控制优化器使用最优的执行计划、使用并行等方式。

3. 操作系统优化

   考虑磁盘调度策略的优化、预读的优化以及启动磁盘写缓存策略等方式。

4. 应用优化

   考虑改写 SQL 语句、使用 PBE 缓存执行计划、错峰执行等情况。

5. 硬件优化

   扩展 CPU、内存、存储、网络等硬件能力。

6. 架构优化

   使用缓存、读写分离、分库分表等架构来进行优化。

有关性能调优方法的运用将在性能调优工具 予以描述。

4.6　性能诊断与优化经验

除了上述从瓶颈分析、根因分析到实施调优手段这种自顶向下追溯原因的方法，KingbaseES 也有一些常见的优化经验供用户参考：

- 如果遇见查询计划不准的情况，首先需要考虑是否做了 analyze。
- 面对数据量大导致性能缓慢的情况，建议优先检查是否需要增大 shared_buffers 配置。
- 对于涉及排序和 join 操作的查询慢情况，在数据量稍微大时即可检查是否需要增大 work_mem，确保不写临时文件，但同时也要避免并发特别多时设置太大可能导致的内存不足。
- 全表扫描慢的情况，可以考虑采用索引，尤其是表达式索引和条件索引，调整 SQL 的执行计划，选择合适的索引。
- 多表 join 慢的情况，可以考虑优化 join 顺序，调整执行计划得到最优的 join 的顺序。
- 表的 join 慢的情况，比如原来采用了 nestloop，数据无序可以考虑采用 hashjoin，有序可以考虑 mergejoin。
- 对于慢语句，在 CPU 资源比较充裕的情况下，可以考虑并行技术。
- 数据库层的调优效果不明显时，可以考虑操作系统调优。比如 IO 问题，可以考虑磁盘的调度策略（通常数据库推荐使用 deadline 策略），还有操作系统预读和 IO 请求队列的调整。
- 在考虑了操作系统调优后，可以考虑通过升级硬件来提高磁盘 IO 和 CPU 的吞吐量。
- 针对 IO 瓶颈且 CPU 资源充裕的情况，可以引入数据压缩技术进行优化。
- 对于同类 SQL 请求比较多的情况，可以尝试通过扩展协议进行 SQL 的变量绑定或者使用普通协议的全局计划缓存，减少 SQL 硬解析开销。尤其在面对大量短查询（ms 级）场景时，能够不经过解析过程直接使用缓存的执行计划，效率可以得到极大提升。
- 数据量庞大的情况下，建议考虑分区进行优化。
- 数据库高效的运行的最关键因素是需要足够的内存，来缓存住数据并在内存中先完成更新。但不同的业务对内存需求程度不一样，比如：一般推荐内存大小要占到数据量的 15~25%；而对于数据访问量比较大的业务，内存大小基本要达到数据库大小的 80%。
- 当单机性能无法满足需求时，读多写少的场景可以考虑读写分离。

- 若出现连接数过多造成资源争用严重的情况，考虑使用外部连接池来减缓竞争压力。

第5章　操作系统 VS 数据库诊断

操作系统层面的性能监控工具能够实现端到端的性能监控，提供了丰富的指标数据，但这些指标缺乏统一的度量单位，因此难以对数据库相关问题进行深入分析。相比之下，KingbaseES 数据库自带的性能监控工具更加关注数据库服务器端的性能表现，并提供具备明确数据库语义的数据与诊断模型，然而其局限性在于无法识别数据库外部的问题。

所以，操作系统层面的性能诊断，期望能够通过监控和分析端到端的资源使用情况，定位到性能瓶颈是否由数据库造成，瓶颈发生在什么资源上。

备注:

需要注意的是，如果仅根据瓶颈信息在操作系统层面实施优化措施，虽然可能取得一定效果，但由于无法直接定位到问题根源，这种优化方式可能会导致投入与收益不成比例。因此，建议谨慎采用。

若确定性能瓶颈源于数据库，则应借助数据库提供的诊断工具，结合操作系统层面获取的瓶颈信息进行交叉验证，从而准确识别根本原因，并据此制定相应的优化方案。

第6章　操作系统性能诊断

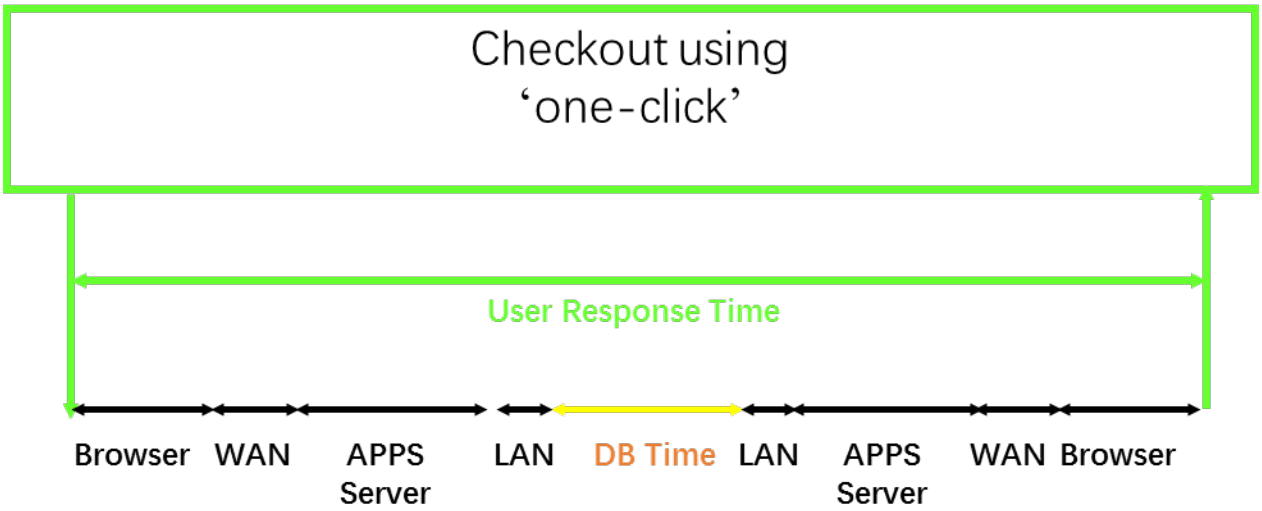
本章节包含以下内容：

- 操作系统诊断概述
- 操作系统资源监控工具
- 操作系统资源分析

6.1　操作系统诊断概述

性能诊断的目的是识别影响性能目标的关键因素，并为性能调优提供方向。本部分主要阐述了从性能瓶颈分析到性能优化点定位的诊断方法。

当进行性能诊断时，要优先分析瓶颈问题，然后从整体到局部逐渐分解，逐步向下找到问题所在，往往能够起到事半功倍的效果。



以 OA 应用为例，如上图所示，用户的响应时间包含了：数据库的响应时间、应用处理时间、浏览器渲染时间、各组件之间的网络传输时间。所以，性能瓶颈的定位，需要收集整个链条上所有软硬件资源的性能监控数据。

- 各主机 CPU 资源是否饱和？如果饱和，是否是数据库占用了过多的 CPU？

方法参见”CPU 资源分析 ”章节。

- 各主机网络是否稳定？流量是否饱和？如果饱和，是否是数据库占用了过多的网络资源？

  方法参见”网络资源分析 ”章节。

- 各主机 I/O 资源是否饱和？如果饱和，是否是应用或数据库占用的 I/O？

  方法参见”磁盘资源分析 ”章节。

注意:

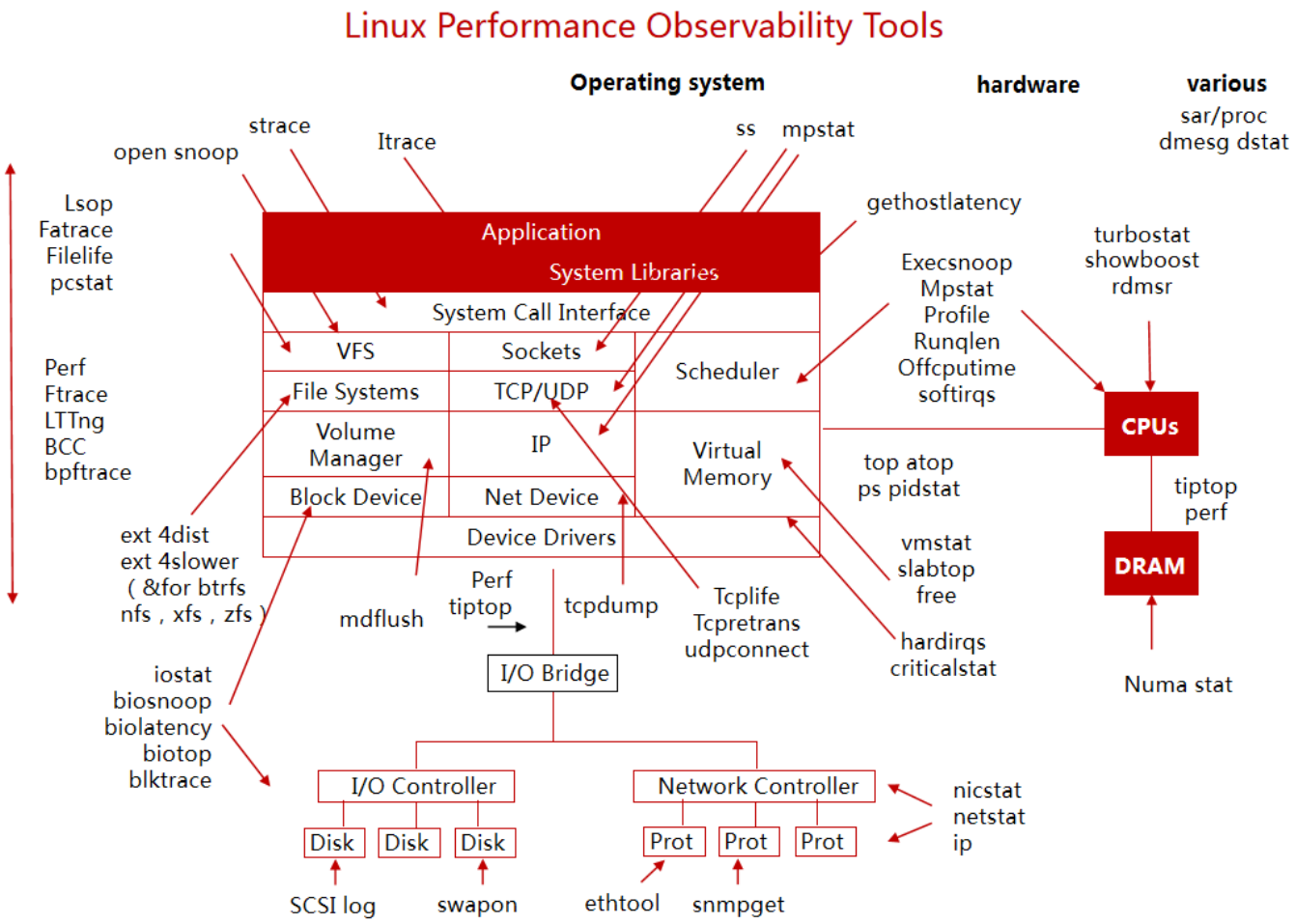
如果确定是数据库所在的服务器存在资源瓶颈，还需进一步确定瓶颈是否由数据库引起。例如，KingbaseES 数据库与 MySQL 同时部署在一台服务器上，若 KingbaseES 占用了 10% 的 CPU，而 MySQL 占用了 90% 的 CPU，则更应该优先关注 MySQL 占用 CPU 资源较高的情况。

此时，有可能已经发现了问题，比如操作系统磁盘调度问题或其他活动占用了资源。很多时候，从操作系统层面反映出来的瓶颈，并不能直接揭示问题的根本原因，此时就要向下分析。

备注:

有关响应时间测量的方法相对复杂，这里提供一个简单的可行思路，在应用端监控响应时间和吞吐量，通过 KWR 的 LOAD PROFILE 报告获得数据库端信息，将两者的事务或者操作映射到一个单位上，可以得到应用响应时间和数据库响应时间之间的比例关系。

6.2　操作系统资源监控工具



Linux 内核提供了丰富的性能分析工具，涵盖了 CPU、内存、磁盘 I/O 以及网络等方面的性能监测。

1. CPU 性能工具

从 CPU 角度来讲，主要性能指标包括：平均负载、CPU 使用率、上下文切换以及 CPU 缓存命中率等。其中 top 是最常用的工具，可以实时查看系统和进程的 CPU 状态。

- 平均负载：top 命令提供比较全面的指标
- CPU 使用率：包括系统和进程的 CPU 使用率，常用 top、atop、ps、pidstat 等工具查看。其中 top 不仅可以查看系统 CPU 使用率，也可以查看进程的 CPU 使用率
- 上下文切换：通过 pidstat 工具查看进程上下文切换
- CPU 缓存命中率：使用 perf stat 命令查看

2. 内存性能工具

从内存角度来讲，主要性能指标包括：系统内存的分配与使用、进程内存的分配与使用以及 Swap 的使用。通常使用 top、free 等简单工具进行查看，如果问题比较复杂，可通过 vmstat 和 sar 工具进一步分析。

- 系统内存的分配与使用：通常使用 free、vmstat、sar 等工具，free 命令比较简单，vmstat 和 sar 信息比较全面
- 进程内存的分配与使用：可借助 top、ps、pidstat 等工具进行监控
- Swap：可以通过 free、vmstat、sar 等工具进行监控

3. 磁盘 IO 性能工具

从磁盘 IO 角度来讲，主要性能指标包括：磁盘 IO 使用率、吞吐量和延迟等。通常使用 iostat 工具来查看系统磁盘 IO。

- 系统 IO 性能分析：系统磁盘 IO 使用率、吞吐量、延迟使用 iostat、sar 等工具监控；块设备 IO 使用 blktrace 工具跟踪
- 进程 IO 性能分析：系统调用跟踪可使用 strace、perf 等工具；块设备大小可使用 biosnoop、biotop 等专用工具分析

4. 网络性能工具

从网络角度来讲，主要性能指标包括：吞吐量、响应时间、连接数、丢包数等。通常使用 sar 工具进行分析。

通常使用 sar 工具查看网络吞吐量，使用 neststat 工具查看连接数，使用 ping 工具查看网络延迟，使用 tcpdump 工具进行网络抓包。

6.3　操作系统资源分析

本章节包含以下内容：

- CPU 资源分析
- 内存资源分析
- 磁盘资源分析
- 网络资源分析

6.3.1　CPU 资源分析

6.3.1.1　CPU 信息简介

CPU 资源监控的工具较多，大部分工具的监控内容相似，本节以 mpstat 为例做重点介绍。

mpstat 是一个实时的系统监控工具，用于查看系统的整体性能和进程运行情况。它生成的报告包含与 CPU 相关的统计信息，这些数据来源于/proc/stat 文件。

用法：

```
mpstat [-P { | ALL } ] [ 时间间隔 [ 次数] ]
```

- -P 选项用于指定要监控的 CPU 编号，在范围 [0，CPU 个数-1] 内取值。

- 时间间隔：相邻的两次采样的间隔时间。
- 次数：采样的次数。

示例：

```
[kingbase@localhost ~]$ mpstat -P 0,1 2 2
Linux 3.10.0-862.el7.x86_64 (rdjc2)     MM/dd/yyyy      _x86_64_        (40 CPU)

03:42:09 PM  CPU    %usr   %nice    %sys %iowait    %irq   %soft  %steal  %guest  %gnice   %idle
03:42:11 PM    0    3.47    0.00    4.95    0.00    0.00    0.50    0.00    0.00    0.00   91.09
03:42:11 PM    1    1.01    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00   98.99
03:42:13 PM    0    1.01    0.00    1.01    2.51    0.00    0.00    0.00    0.00    0.00   95.48
03:42:13 PM    1    0.50    0.00   12.87    0.00    0.00    0.00    0.00    0.00    0.00   86.63
Average:       0    2.24    0.00    2.99    1.25    0.00    0.25    0.00    0.00    0.00   93.27
Average:       1    0.75    0.00    6.50    0.00    0.00    0.00    0.00    0.00    0.00   92.75
```

字段说明：

- %usr：在 internal 时间段里，用户态消耗的 CPU 时间比例。
- %sys：在 internal 时间段里，系统内核消耗的 CPU 时间比例。
- %nice：在 internal 时间段里，带 NICE 优先级的用户态消耗的时间比例。
- %iowait：在 internal 时间段里，CPU 等待 IO 操作完成所花费时间的百分比。
- %irq：在 internal 时间段里，硬中断时间（%）。
- %soft：在 internal 时间段里，软中断时间（%）。
- %steal：在 internal 时间段里，管理程序维护另一个虚拟处理器时，虚拟 CPU 的等待时间比例。
- %guest：在 internal 时间段里，运行虚拟机的 CPU 时间（%）。
- %gnice：在 internal 时间段里，经过优先级调整后的，运行虚拟机的 CPU 时间。
- %idle：在 internal 时间段里，CPU 闲置时间（%）。

6.3.1.2　CPU 瓶颈分析

主要分析：

- %idle：空闲 CPU 率 $< 20\%$，说明系统 CPU 非常忙碌，可能存在 CPU 瓶颈。
- 如果连接数并不多，但通过 top 观察到的进程 CPU 利用率接近 100%，也表明存在 CPU 瓶颈问题。

6.3.1.3　CPU 瓶颈示例 - 单进程

以 TPCH 10GB 数据集为测试数据，启动一个数据库连接进行测试，执行 TPCH 查询语句 Q2，通过 top 工具可以实时动态的观察数据库服务器资源使用情况，可以发现数据库端的 CPU 利用率达到了 100%，这表明瓶颈在数

据库端，CPU 计算比较慢。

```
[kingbase@localhost ~]$ top
top - 19:10:27 up 18:36,  4 users,  load average: 0.82, 0.48, 0.38
Tasks: 215 total,   3 running, 212 sleeping,   0 stopped,   0 zombie
%Cpu(s): 20.3 us, 14.0 sy,  0.0 ni, 59.8 id,  0.0 wa,  0.0 hi,  5.9 si,  0.0 st
KiB Mem :  8157204 total,   130540 free,   552648 used,  7474016 buff/cache
KiB Swap:  5242876 total,  4932092 free,   310784 used.  1868744 avail Mem

   PID USER      PR  NI    VIRT    RES    SHR S  %CPU %MEM     TIME+ COMMAND
 20201 kingbase  20   0 7375172 173352  66612 R 100.0  2.1   0:08.36 kingbase
 20285 kingbase  20   0  162116   2412   1584 R  10.2  0.0   0:00.58 top
    42 root      20   0       0      0      0 S   8.2  0.0   2:01.31 kswapd0
  2028 kingbase  20   0 7274484 751120 742480 S   5.2  9.2   1:12.70 kingbase
   757 root      20   0  320252   2064   1708 S   4.6  0.0  13:11.83 vmtoolsd
   776 root      20   0  115300    852    656 S   3.6  0.0   0:07.68 ksmtuned
     9 root      20   0       0      0      0 R   3.3  0.0   2:07.68 rcu_sched
    14 root      20   0       0      0      0 S   2.6  0.0   1:04.70 ksoftirqd/1
```

6.3.1.4　CPU 瓶颈示例 - 多进程

以 kbbench 高并发 SELECT 查询为例，在 test 库上初始化测试表，并生成 10 万条数据记录进行测试：

命令

```
./kbbench -p 54339 -i kingbase -s 10
```

信息:

-p KingbaseES 端口号，kingbase 是数据库名，-s 数据规模

压测脚本

```
SELECT aid, sum(abalance) AS sum_a FROM kbbench_accounts GROUP BY aid ORDER BY sum_a ASC;
```

压测

```
./kbbench -M extended -n -r -f ./kbbench_read_test.sql -P 5 -c 4 -j 4 -T 60 -p 54339 -U kingbase kingbase
```

信息:

-p KingbaseES 端口号，-U 用户名，最后一个 kingbase 是数据库名

通过 top 工具，按 1 键查看所有 CPU 核的使用情况，发现四个 CPU 的 user 和 sys 使用率之和达到 100%，CPU idle 为 0，可以认为瓶颈在数据库端，CPU 计算比较慢。

```
[kingbase@localhost ~]$ top
top - 11:17:41 up 47 days, 17:55,  9 users,  load average: 3.86, 2.63, 2.17
Tasks: 269 total,   6 running, 262 sleeping,   1 stopped,   0 zombie
%Cpu0  : 86.0 us, 14.0 sy,  0.0 ni,  0.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
%Cpu1  : 88.4 us, 11.6 sy,  0.0 ni,  0.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
%Cpu2  : 88.1 us, 11.9 sy,  0.0 ni,  0.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
%Cpu3  : 86.4 us, 13.6 sy,  0.0 ni,  0.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
KiB Mem :  4028428 total,   638284 free,  1005916 used,  2384228 buff/cache
KiB Swap:  2097148 total,  1413884 free,   683264 used.  2158584 avail Mem

   PID USER      PR  NI    VIRT    RES    SHR S  %CPU %MEM     TIME+ COMMAND
 43265 kingbase  20   0 5268972 225548  59540 R  90.4  5.6   0:36.33 kingbase
 43266 kingbase  20   0 5228088 189644  59540 R  89.7  4.7   0:36.12 kingbase
 43267 kingbase  20   0 5178940 142384  59532 R  87.7  3.5   0:36.46 kingbase
 43264 kingbase  20   0 5211704 174064  59536 R  81.8  4.3   0:36.22 kingbase
 43255 kingbase  20   0  390912   4380   2868 S  38.1  0.1   0:15.62 kbbench
 73665 kingbase  20   0 5136944  56972  18332 S   1.0  1.4   0:40.96 kingbase
  2280 kingbase  20   0  810088  15164    624 S   0.7  0.4 192:51.36 gsd-color
   711 root      20   0  396624   1800   1072 S   0.3  0.0   1093:44 accounts-daemon
   725 root      20   0   21672    408    272 S   0.3  0.0   4:12.36 irqbalance
  2223 kingbase  20   0  611676   1232    688 S   0.3  0.0 438:42.94 gsd-account
 42563 kingbase  20   0  162116   2480   1588 R   0.3  0.1   0:00.32 top
 73663 kingbase  20   0  389176   6724    908 S   0.3  0.2   0:20.73 kingbase
```

6.3.2　内存资源分析

6.3.2.1　内存信息简介

内存资源监控的工具较多，大部分工具的监控内容相似，本节以 top 为例做重点介绍。top 命令可以显示 CPU、内存等信息：

```
[kingbase@localhost bin]$ top
top - 16:54:26 up 53 days,  6:37,  8 users,  load average: 0.00, 0.00, 0.00
Tasks: 1277 total,   1 running, 1276 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem : 489318.1 total,  18192.0 free,  11039.9 used, 460086.1 buff/cache
MiB Swap:   4095.9 total,   3767.9 free,    328.0 used. 450235.7 avail Mem
```

跟内存相关的 2 个字段为 Mem 和 Swap，分别表示物理内存和交换内存。**字段说明如下：**

- Total：物理/交换内存总量。

- Used：已使用的物理/交换内存总量。
- Free：空闲物理/交换内存总量。
- Buffers：用作内核缓存的内存量。
- Cached：缓冲的交换区总量。

6.3.2.2　内存资源分析

主要分析：

- Swap 是否被使用：如果存在 Swap 空间被使用的情况，说明发生了磁盘 IO 操作。这种情况可能导致数据库因 CPU 和 IO 时间消耗而导致性能下降。
- 空闲内存是否比较少：通常情况下，若空闲内存/物理内存大于 70%，可认为内存性能较好；反之，若该比例小于 20%，则可认为内存性能差，需要添加内存。

如果在内存占用率较低的场景下，发现数据库查询速度较慢，同时存在数据量很大并且 IO 很多的情况，可以考虑调大 shared_buffers 参数值。此举有助于提升缓存命中率，优化整体性能表现。

6.3.2.3　内存瓶颈示例

在执行 TPCH 基准测试中的 Q18 查询时，通过 top 工具实时监控数据库服务器资源使用情况，可以观察到数据库端的内存利用率达到了 84%，剩余内存仅有 137MB，且 Swap 空间已使用 3.18GB。表明系统已出现明显的内存瓶颈问题，进而导致性能显著下降。

```
[kingbase@localhost bin]$ top
top - 21:11:09 up 20:37,  5 users,  load average: 1.07, 1.07, 0.94
Tasks: 217 total,   1 running, 216 sleeping,   0 stopped,   0 zombie
%Cpu(s):  1.2 us,  2.1 sy,  0.0 ni, 74.4 id, 22.1 wa,  0.0 hi,  0.1 si,  0.0 st
KiB Mem :  8157204 total,   140984 free,  4460668 used,  3555552 buff/cache
KiB Swap:  5242876 total,  1908384 free,  3334492 used.    79448 avail Mem

   PID USER      PR  NI    VIRT    RES    SHR S  %CPU %MEM     TIME+ COMMAND
 20201 kingbase  20   0   11.5g   6.5g   2.8g D  10.9 84.0   7:48.26 kingbase
    42 root      20   0       0      0      0 S   2.0  0.0   3:11.73 kswapd0
     9 root      20   0       0      0      0 S   0.3  0.0   2:22.21 rcu_sched
   757 root      20   0  320252   1540   1184 S   0.3  0.0  13:49.99 vmtoolsd
  2031 kingbase  20   0  388660    304     32 S   0.3  0.0   0:02.83 kingbase
 21826 root      20   0       0      0      0 S   0.3  0.0   0:00.09 kworker/1:1
 22009 root      20   0       0      0      0 S   0.3  0.0   0:00.09 kworker/2:3
```

此外，通过 free 工具也可以获取更为详尽的系统内存使用详情。

```
[kingbase@localhost ~]$ free -h
              total        used        free      shared  buff/cache   available
```

```
Mem:          7.8G       5.3G       120M       2.2G       2.4G        54M
Swap:         5.0G       3.4G       1.6G
```

6.3.3　磁盘资源分析

6.3.3.1　IO 信息简介

I/O 资源监控的工具较多，大部分工具的监控内容相似，本节以 iostat 为例做重点介绍。iostat 可以显示 CPU 和 I/O 系统的负载情况及分区状态信息。

用法:

`iostat [ 选项] [ < 时间间隔> [ < 次数> ] ]`

- 选项：可选参数较多，建议查看 help。其中，-x 可以显示更为全面的信息。
- 时间间隔：信息的输出间隔。
- 次数：输出信息的次数。

示例：

```
[root@localhost ~]$ iostat -xtk 1
Linux 3.10.0-1160.el7.x86_64 (localhost.localdomain)    yyyy 年 MM 月 dd 日  _x86_64_        (64 CPU)

yyyy 年 MM 月 dd 日 17 时 07 分 04 秒
avg-cpu:  %user   %nice %system %iowait  %steal   %idle
           8.15    0.00    0.59    0.70    0.00   90.55

Device:         rrqm/s   wrqm/s     r/s     w/s    rkB/s    wkB/s avgrq-sz avgqu-sz   await r_await w_await
svctm  %util
sda               1.27     5.09    2.33   18.75    55.47  1519.95   149.47     0.11    5.07   22.80    2.86
 4.32   9.10
sdb               0.00     0.00    0.00    0.00     0.00     0.00    63.78     0.00    3.47    3.47    0.00
 2.19   0.00
dm-0              0.00     0.00    1.02    0.22     9.54     2.53    19.48     0.03   26.88   17.08   73.38
 6.13   0.76
dm-1              0.00     0.00    2.01    4.47     8.03    17.89     8.00     0.44   68.12   20.20   89.63
 0.95   0.62
dm-2              0.00     0.00    0.60   19.15    37.90  1499.53   155.72     0.56   28.49   35.48   28.27
 4.45   8.78
```

iostat 不仅可以用于监控 I/O 性能，还可以同时监测 CPU 状态。

IO 相关的字段说明如下：

- rrqm/s：每秒进行 merge 的读操作数目，即 delta(rmerge)/s。

- wrqm/s：每秒进行 merge 的写操作数目，即 delta(wmerge)/s。
- r/s：每秒完成的读 I/O 设备次数，即 delta(rio)/s。
- w/s：每秒完成的写 I/O 设备次数，即 delta(wio)/s。
- rkB/s：每秒读 K 字节数，是 rsect/s 的一半，因为每扇区大小为 512 字节。
- wkB/s：每秒写 K 字节数，是 wsect/s 的一半。
- avgrq-sz：平均每次设备 I/O 操作的数据大小 (扇区)，即 delta(rsect+wsect)/delta(rio+wio)。
- avgqu-sz：平均 I/O 队列长度，即 delta(aveq)/s/1000(因为 aveq 的单位为毫秒)。
- await：平均每次设备 I/O 操作的等待时间 (毫秒)，即 delta(ruse+wuse)/delta(rio+wio)。
- svctm：平均每次设备 I/O 操作的服务时间 (毫秒)，即 delta(use)/delta(rio+wio)。
- %util：一秒中有百分之多少的时间用于 I/O 操作，或者说一秒中有多少时间 I/O 队列是非空的，即 delta(use)/s/1000(因为 use 的单位为毫秒)。

6.3.3.2　IO 资源分析

比较重要的参数分析：

- %iowait： IO 占有 CPU 的利用率，当该值超过 5% 时，需引起关注。
- %util：磁盘 IO 使用率，主要观察其是否已经接近或者超过 100%。如果%util 接近 100%，说明 I/O 请求频繁，I/O 系统已经满负荷，该磁盘可能存在瓶颈。
- svctm：服务时间，主要说明磁盘本身的 IO 性能快慢。比如在 TPCC 测试中，一般盘阵服务时间通常在 0.25ms 左右。如果该值过大，则表明磁盘性能存在问题；此外，CPU/内存的负荷也会对该值产生一定影响。
- await：主要说明平均每次 IO 响应时间，一般小于 5ms。
  - 其中，svctm 通常应小于 await(因为等待时间会算入 svctm)。await 的大小一般取决于服务时间 (svctm) 以及 I/O 队列的长度 (avgqu-sz) 和 I/O 请求的发出模式。
  - 如果 svctm 比较接近 await，说明 I/O 几乎没有等待时间。
  - 如果 await 远大于 svctm，说明 I/O 队列太长，应用得到的响应时间变慢。
  - 如果响应时间超过了用户可以容许的范围，这时可以考虑更换更高性能的磁盘、调整内核 IO 调度的 elevator 算法、优化应用或者升级 CPU。
- avgqu-sz：可作为衡量系统 I/O 负荷的指标，但由于 avgqu-sz 是按照单位时间的平均值，所以不能反映瞬间的 I/O 峰值。若分析发现 avgqu-sz 持续偏高，则说明读写 IO 请求排队数量比较多。另外，要结合 avgrq-sz 的情况来判断 IO 量的情况：如果 avgrq-sz 比较大，则说明平均每次读写请求的数据量比较多；如果 avgqu-sz 和 avgrq-sz 均较大，则表明存在明显的 IO 性能问题。
- avgrq-sz：读写请求的数据量 (扇区数)，上面已经说明该值和 avgqu-sz 一起决定 IO 问题：如果 avgqu-sz 排队数量多，但请求的数据小的话，其实 IO 也会很小；如果请求数据大的话，IO 的数据才会高。

对于进一步分析 IO 的使用分布情况，建议采用 pidstat 等工具进行进一步的观察，以便快速定位 checkpoint、wal 日志等关键 I/O 分布情况。

6.3.3.3　I/O 瓶颈示例

以 TPCH Q18 为例，通过 iostat 工具，每 1 秒打印一次 IO 使用情况如下图。iowait 百分比已经达到 21.41%，IO 使用率达到 100%，svctm 达到了 11.9，因此可判断存在磁盘 IO 性能问题。

```
[kingbase@localhost bin]$ iostat -xkt 1
yyyy 年 MM 月 dd 日 21 时 14 分 11 秒
avg-cpu:  %user   %nice %system %iowait  %steal   %idle
           1.76    0.00    2.02   21.41    0.00   74.81

Device:         rrqm/s   wrqm/s     r/s     w/s    rkB/s    wkB/s avgrq-sz avgqu-sz   await r_await w_await
svctm  %util
sda              15.00     0.00   84.00    0.00 38496.00     0.00   916.57     8.46   91.65   91.65    0.00
11.90 100.00
dm-0              0.00     0.00    0.00    0.00     0.00     0.00     0.00     0.00    0.00    0.00    0.00
 0.00   0.00
dm-1              0.00     0.00   24.00    0.00    96.00     0.00     8.00     0.95   39.54   39.54    0.00
12.92  31.00
```

以 kbbench 高并发更新场景为例，在 kingbase 库上建立测试表：

```
CREATE TABLE t_update(sid INT PRIMARY KEY, info TEXT, val INT DEFAULT 0, crt_time TIMESTAMP);
CREATE TABLESPACE tbs1 LOCATION '/home/user';
ALTER INDEX t_update_pkey SET TABLESPACE tbs1;
--插入一亿条数据
INSERT INTO t_update SELECT generate_series(1, 100000000), md5(random()::TEXT), 0, NOW();
```

压测脚本：

```
\set id random(1, 100000000)
UPDATE t_update SET val=val+1, crt_time=NOW() WHERE sid=:id;
```

压测：

```
./kbbench -M prepared -n -r -f bigtable_update.sql -P 5 -c 4 -j 4 -T 300 -p 65530 -U kingbase kingbase
```

信息:

-p KingbaseES 端口号，-U 用户名

通过 iostat 工具，每 1 秒打印一次 IO 使用情况如下图。iowait 百分比已经达到 48.72%，IO 使用率达到 99.7%，svctm 达到了 3.00，因此可判断存在磁盘 IO 性能问题。

```
[kingbase@localhost bin]$ iostat -xkt 1
MM/dd/yyyy 09:34:36 AM
avg-cpu:  %user   %nice %system %iowait  %steal   %idle
           1.02    0.00    7.91   48.72    0.00   42.35

Device:         rrqm/s   wrqm/s     r/s     w/s    rkB/s    wkB/s avgrq-sz avgqu-sz   await r_await w_await
svctm  %util
sda               0.00     0.00  187.00  145.00  4172.00  2656.00    41.13     2.92    6.47    9.11    3.07
 3.00  99.70
```

6.3.4　网络资源分析

6.3.4.1　网络信息简介

网络资源监控的工具较多，大部分工具的监控内容相似，本节以 sar 为例做重点介绍。sar（System Activity Reporter）是系统活动情况报告的缩写。它是目前 Linux 上最为全面的系统性能分析工具之一，可以从多方面对系统的活动进行报告，包括：文件的读写情况、系统调用的使用情况、磁盘 I/O、CPU 效率、内存使用状况、进程活动及 IPC 有关的活动等。

1. 使用 `sar -n DEV` 监控网络传输情况：

```
[root@localhost ~]$ sar -n DEV
Linux 3.10.0-1160.el7.x86_64 (localhost.localdomain)    yyyy 年 MM 月 dd 日  _x86_64_        (64 CPU)
00 时 00 分 01 秒     IFACE   rxpck/s   txpck/s    rxkB/s    txkB/s   rxcmp/s   txcmp/s  rxmcst/s
00 时 10 分 01 秒 br-812aeda36dbb      0.00      0.00      0.00      0.00      0.00      0.00      0.00
00 时 10 分 01 秒        lo     14.04     14.04      2.09      2.09      0.00      0.00      0.00
00 时 10 分 01 秒 virbr0-nic      0.00      0.00      0.00      0.00      0.00      0.00      0.00
00 时 10 分 01 秒    virbr0      0.00      0.00      0.00      0.00      0.00      0.00      0.00
00 时 10 分 01 秒       em4      0.00      0.00      0.00      0.00      0.00      0.00      0.00
00 时 10 分 01 秒       em3      0.00      0.00      0.00      0.00      0.00      0.00      0.00
00 时 10 分 01 秒       em1   1583.71    970.87   1054.06    133.84      0.00      0.00     69.35
00 时 10 分 01 秒       em2      0.00      0.00      0.00      0.00      0.00      0.00      0.00
00 时 10 分 01 秒   docker0      0.00      0.00      0.00      0.00      0.00      0.00      0.00
```

字段说明：

- IFACE：网络设备名。
- rxpck/s：每秒钟接收到的包数目。
- txpck/s：每秒钟发送出去的包数目。
- rxkB/s：每秒钟接收到的字节数。

- txkB/s：每秒钟发送出去的字节数。
- rxcmp/s：每秒钟接收到的压缩包数目。
- txcmp/s：每秒钟发送出去的压缩包数目。
- rxmcst/s：每秒接收的多播（multicast）包的总数。

2. 使用 EDEV 关键字查看网络传输的失败情况：

```
[root@localhost ~]$ sar -n EDEV 2
Linux 3.10.0-862.el7.x86_64 (rdjc2)     yyyy 年 MM 月 dd 日  _x86_64_        (40 CPU)

10 时 13 分 55 秒     IFACE   rxerr/s   txerr/s    coll/s  rxdrop/s  txdrop/s  txcarr/s  rxfram/s  rxfifo/s
txfifo/s
10 时 13 分 57 秒    vmnet8      0.00      0.00      0.00      0.00      0.00      0.00      0.00      0.00
    0.00
10 时 13 分 57 秒        lo      0.00      0.00      0.00      0.00      0.00      0.00      0.00      0.00
    0.00
10 时 13 分 57 秒       em2      0.00      0.00      0.00      0.00      0.00      0.00      0.00      0.00
    0.00
10 时 13 分 57 秒       em3      0.00      0.00      0.00      0.00      0.00      0.00      0.00      0.00
    0.00
10 时 13 分 57 秒       em1      0.00      0.00      0.00      0.00      0.00      0.00      0.00      0.00
    0.00
10 时 13 分 57 秒       em4      0.00      0.00      0.00      0.00      0.00      0.00      0.00      0.00
    0.00
10 时 13 分 57 秒    vmnet1      0.00      0.00      0.00      0.00      0.00      0.00      0.00      0.00
    0.00
```

字段说明：

- rxerr/s：每秒钟接收到的损坏的包的数目。
- txerr/s：当发送包时，每秒钟发生的错误数。
- coll/s：当发送包时，每秒钟发生的冲撞（collisions）数（这个是在半双工模式下才有）。
- rxdrop/s：由于缓冲区满，网络设备接收端，每秒钟丢掉的网络包的数目。
- txdrop/s：由于缓冲区满，网络设备发送端，每秒钟丢掉的网络包的数目。
- txcarr/s：当发送数据包时，每秒钟载波错误发生的次数。
- rxfram/s：在接收数据包时，每秒钟发生的帧对齐错误的次数。
- rxfifo/s：在接收数据包时，每秒钟缓冲区溢出错误发生的次数。
- txfifo/s：在发送数据包时，每秒钟缓冲区溢出错误发生的次数。

6.3.4.2　网络资源分析

主要分析：

- 检查 rxbyt/s 和 txbyt/s 字段中的收发字节数，判断是否已达到网络带宽瓶颈。需要注意的是，网络的性能取决于源端和目的端以及中间设备（网线、路由器、交换机等）的整体表现。例如：源端和目的端都是千兆网卡，但是它们之间的交换机为百兆，则两端的网络传输上限只能是百兆。为了更准确的了解网络性能的真实表现，可以通过 scp 或者 iperf 来做测试。
- 查看是否有网络传输错误，如果存在则需要做更进一步分析。网络信息相关的工具主要包括：ping、nmon、iftop、sar、netstat、ethtool、ifconfig、mii-tool。

6.3.4.3　网络瓶颈示例

以 TPCC 为例，在 warehouses=30，terminals=300 环境下，通过 `sar -n DEV 2` 命令读取运行的平均网络参数。

在 10 MBps 网卡配置下，通过计算可发现收发速度已达到网卡性能极限：

```
Average:        IFACE   rxpck/s   txpck/s    rxkB/s    txkB/s   rxcmp/s   txcmp/s  rxmcst/s
Average:           lo    409.02    409.02    275.48    275.48      0.00      0.00      0.00
Average:    virbr0-nic      0.00      0.00      0.00      0.00      0.00      0.00      0.00
Average:       virbr0      0.00      0.00      0.00      0.00      0.00      0.00      0.00
Average:        ens33   5594.49   5762.38   1123.03   1225.24      0.00      0.00      0.00
```

在 100 MBps 网卡配置下，通过计算可发现远未达到网卡极限，网络瓶颈已消除：

```
Average:        IFACE   rxpck/s   txpck/s    rxkB/s    txkB/s   rxcmp/s   txcmp/s  rxmcst/s
Average:           lo   1190.17   1190.17    784.82    784.82      0.00      0.00      0.00
Average:    virbr0-nic      0.00      0.00      0.00      0.00      0.00      0.00      0.00
Average:       virbr0      0.00      0.00      0.00      0.00      0.00      0.00      0.00
Average:        ens33  13851.51  16553.08   2743.45   3173.08      0.00      0.00      0.00
```

第7章　数据库系统性能分析与优化

随着现代应用程序复杂性的提升，确保数据库系统的高效运行已成为一项关键任务。本章节不仅提供理论指导，还强调实践技能的重要性，旨在帮助用户掌握如何识别性能瓶颈、执行有效的查询分析以及实施优化策略以显著提升系统整体效率。

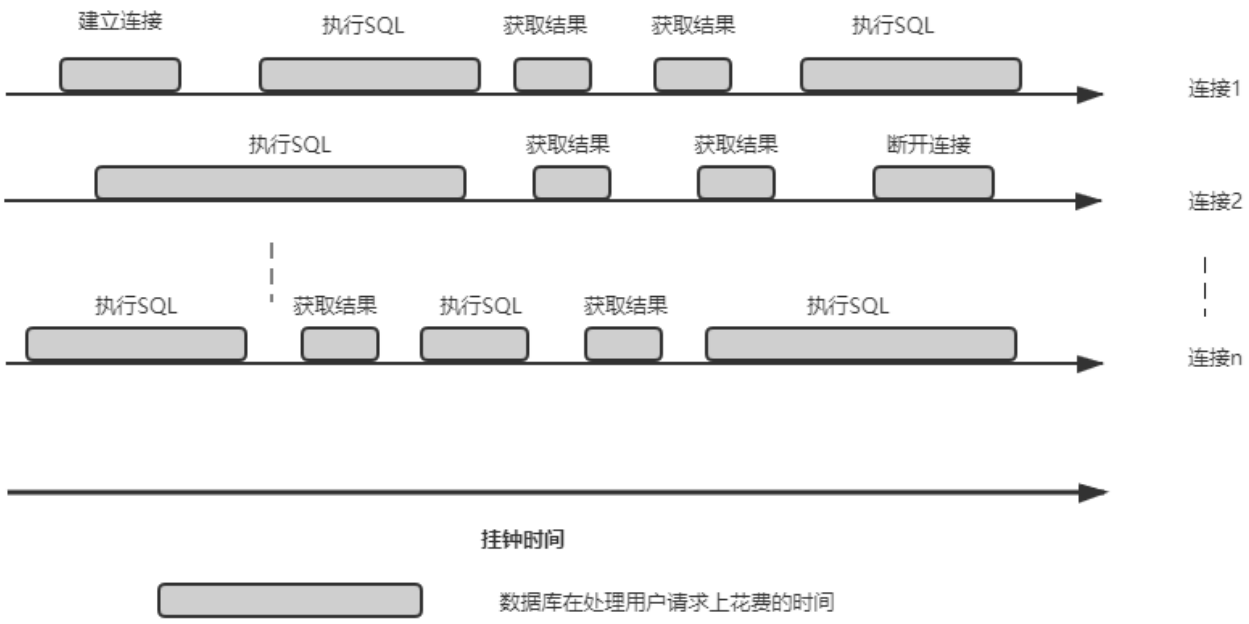
本章节包含以下内容

- 时间模型基础
- 性能分析概述
- 等待事件瓶颈分析与优化
- CPU 瓶颈分析与优化

7.1　时间模型基础

7.1.1　数据库时间分析

7.1.1.1　数据库时间模型概述



数据库时间（DB Time）指服务器响应客户端请求所消耗的总时长，具体包括客户端会话进程（Backend）执行 SQL 语句所需的时间以及连接建立和断开的时间之和。

数据库服务器在处理客户端请求的过程中，包括空闲等待（如等待客户端输入 SQL 请求的 ClientRead 等待事件）、非空闲等待（如执行文件 I/O 操作的 DataFileRead、DataFileWrite 等待事件），以及执行 SQL 语句花费的 CPU 时间。因此，数据库时间可分解为非空闲等待时间和 CPU 时间之和：

数据库时间 = 非空闲等待时间 + CPU 时间，即：DB Time = FG Wait + DB CPU

需要注意的是，KingbaseES 后台工作进程的等待时间和 CPU 时间并不计入上述数据库时间范围内。

7.1.1.2　数据库时间模型组成

数据库总时间可分解为非空闲等待时间和 CPU 时间，从整体上了解了数据库时间的主要构成部分后，有助于判断是 CPU 负载过高还是等待事件过多：

- DB Time：客户端会话执行 SQL 语句的总时间
- FG Wait：客户端会话非空闲等待时间
- DB CPU：DB Time - FG Wait Time（通过上述两者计算得出）

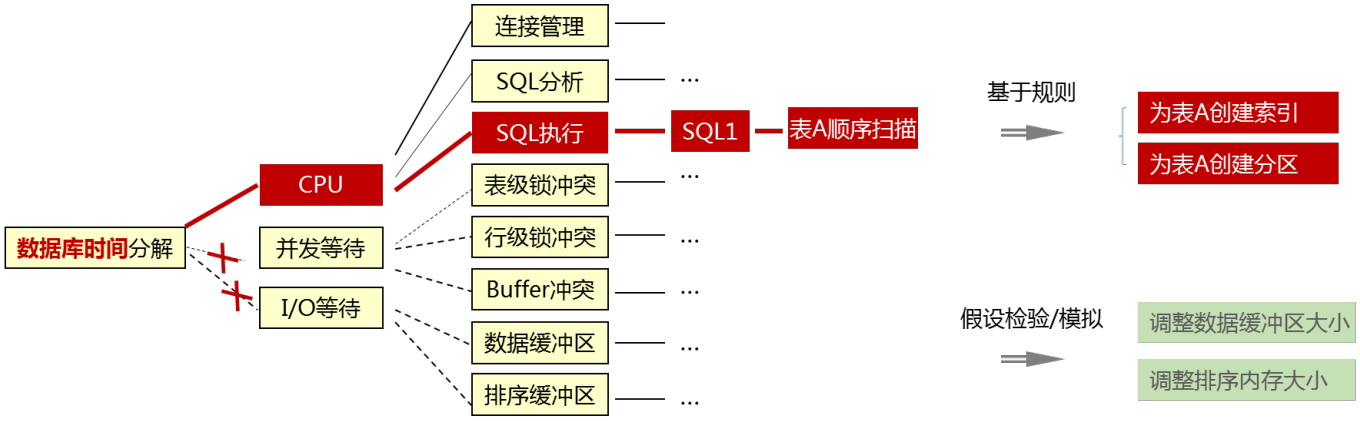
KingbaseES 提供了与数据库时间相关的动态性能视图，便于观察数据库时间的关键组成部分：

- sys_stat_dbtime：数据库时间总体分解
- sys_stat_wait：数据库时间按等待时间分解
- sys_stat_sql：数据库时间按 SQL 语句分解

数据库时间模型是 KingbaseES 为量化数据库性能指标、辅助性能诊断与优化而设计的一套体系，该模型能够从多个维度对数据库时间进行细分。结合数据库时间总值和各维度分解项的时间统计值，用统计项时间除以数据库总时间即可获得相应的百分比指标，这些百分比能够清晰地识别出数据库时间的主要消耗环节。

7.2　性能分析概述

数据库的运行状态对分析某些性能问题至关重要，为了全面监控数据库的运行状态，我们可以分析系统状态统计信息和日志信息，并从中发现潜在的性能相关问题。



与数据库性能相关的内容主要包括：

类型	工具
实时分析	sys_stat_activity（实时等待事件）sys_stat_dbtimesys_stat_statementssys_locks
累积式分析	sys_kwr（基本覆盖所有可优化点的分析）sys_kddmkbbadger
瞬时问题分析	sys_ksh

数据库内置了性能视图和日志系统，为性能诊断提供了有力支持。以下是进行性能分析的一般步骤：

1. 查看动态性能视图/KWR 中的等待时间指标，识别是否存在较多的等待事件。常见的等待事件类型有：I/O、网络、锁等待等。
2. 针对占比显著的等待事件（通常为 5% 以上），根据具体等待事件的含义，采取相应的优化措施。
3. 若未发现明显的等待事件，则主要关注 CPU 计算相关问题，从实例效率、慢 SQL 查询、硬件配置与系统架构等方面进行深入分析和优化。

4. 查看其它性能指标，进一步分析导致性能问题的根本原因。

7.3　等待事件瓶颈分析与优化

等待事件是数据库系统内部记录的一种统计信息。当发生等待事件时，表明数据库的后台进程需要在该事件完成之后才能继续执行后续操作。等待事件揭示了可能影响系统性能的各种问题症状，例如缓冲区不足、锁资源争用等。

针对等待事件的统计，目前数据库内部提供了四种工具。

工具名称	介绍	适用场景
sys_stat_activity	实时展示各活跃会话的当前等待事件	实时分析
sys_stat_sqlwait	累积式记录各 SQL 语句等待事件的时间	实时分析
sys_stat_wait	记录等待事件及在数据库时间中的百分比	实时分析
sys_kwr	以快照形式记录各等待事件的累积时间	基于快照做分析

等待事件的分类有：

- LightLock：轻量级锁。
- Lock：重量级锁。
- BufferPin：数据缓冲区等待。
- Activity：后台辅助进程活动等待。
- Client：客户端等待。
- Extension：扩展插件等待。
- IPC：进程间通讯等待。
- TimeOut：超时等待。
- I/O：I/O 等待。

7.3.1　I/O 类等待事件分析与优化

7.3.1.1　DataFileRead 和 DataFileWrite

KingbaseES 数据库在读取关系表或索引数据页时，若该数据页已存在于共享缓冲区中，则直接返回该数据页；若未命中共享缓冲区，则需要从存储设备中读取该数据页，此过程将触发 DataFileRead 等待事件。这种等待事件通常由 SELECT、UPDATE 和 DELETE 等 DML 操作触发。

以下为常见原因：

- 共享内存缓冲区配置过小：较小的共享缓冲区容量会导致数据页访问命中率下降，从而增加从存储设备读取数据的频率。
- 执行大规模顺序扫描操作：大量顺序扫描操作会显著消耗共享缓冲区资源，导致其快速满载并触发 DataFileRead 等待事件。
- 包含大结果集的 CREATE TABLE AS SELECT 和 CREATE INDEX 语句：在包含大结果集的关系表上运行这些 SQL 语句，会导致较多地数据页读取操作。
- 同时运行的多个 vacuum worker：针对频繁更新或删除的关系表，后台 vacuum 进程在回收表空间过程中会产生较多地数据页读取请求。
- 优化器自动采样：当处理大表连接时，优化器为了获得更精确的执行计划，会增加 ANALYZE 操作的执行频率。

以下是优化建议：

- 适当扩大共享缓冲区容量，增加可驻留的数据页数量。
- 数据库系统应选择适合其存储介质的 I/O 调度策略。常见的磁盘调度算法包括 CFQ、Deadline、NOOP 和 Anticipatory。针对机械硬盘，建议采用 Deadline 策略，这种配置在 TPCC 测试中表现优异；而对于固态硬盘，默认设置通常无需调整。调整方法示例：

```
echo deadline >/sys/block/sd_target_storage/queue/scheduler
```

- 将数据文件与日志文件分散至不同的物理存储设备或利用 RAID 设备，以充分利用多设备的并行 I/O 能力，实现 I/O 压力分担。
- 识别 DataFileRead 比例较高的 SQL 语句，并结合 EXPLAIN 命令评估其执行计划中的扫描方式是否包含顺序扫描，比如使用 sys_stat_sqlwait 视图。
- 识别顺序扫描频率高的关系表，分析涉及该表的 SQL 语句的执行计划是否合理性，比如使用 sys_stat_all_tables 视图。
- 分析执行时间较长的 SQL 语句中的 WHERE 子句所涉及列是否使用了索引，如果没有，请考虑为此列创建索引。
- 对于非常大的表，考虑使用分区表。这种方案能够降低维护成本，数据库仅需针对特定分区进行操作和维护。

调优用例：DataFileRead & DataFileWrite

1. 配置参数：

```
commit_delay = 10
commit_siblings = 16
synchronous_commit = off
```

上述参数可见配置管理手册的预写式日志章节。

2. TPCC 配置：

```
warehouses=10
terminals=10
runMins=1
```

3. 调优前的 TPCC 测试结果：

```
Measured tpmC (NewOrders) = 15183.24
Measured tpmTOTAL = 33674.97
Session Start     = yyyy-MM-dd HH:mm:ss
Session End       = yyyy-MM-dd HH:mm:ss
Transaction Count = 33720
```

4. 调优前，DataFileRead 等待事件的数据库占比很高，并且存在少量的 DataFileWrite 等待事件。

等待事件	等待次数	总时间(s)	平均时间(ms)	% 数据库时间	事件类型
DB CPU	-	37.36	-	7.09	-
DataFileRead	46509	420.62	9.04	79.87	IO
transactionid	676	27.34	40.44	5.19	Lock
WALWriteLock	1607	27.13	16.89	5.15	LightLock
WALSync	6605	10.76	1.63	2.04	IO
WALWrite	6642	1.37	0.21	0.26	IO
buffer_io	47	1.26	26.85	0.24	LightLock
DataFileExtend	2259	0.55	0.24	0.10	IO
DataFileWrite	12938	0.22	0.02	0.04	IO
wal_insert	153	0.01	0.10	0.00	LightLock
DataFileImmediateSync	12	0.01	1.17	0.00	IO

分析和优化：DataFileRead、DataFileWrite 等待事件占数据库时间的百分比高，说明数据页读取和写入的时候，发生的磁盘 I/O 比较高，检查当前共享缓冲区 shared_buffers 大小：

```
kingbase=# show shared_buffers;
 shared_buffers
----------------
 128MB
(1 行记录)
```

执行命令\l+ 检查数据库的占用空间：

```
kingbase=# \l+
                                                                    数据库列表
   名称    |  拥有者  | 字元编码 |  校对规则  |   Ctype    | ICU 排序 |       存取权限        | 大小
-----------+----------+----------+------------+------------+----------+-----------------------+-------
 kingbase  | kingbase | UTF8     | zh_CN.utf8 | zh_CN.utf8 |          |                       | 11 GB
 security  | kingbase | UTF8     | zh_CN.utf8 | zh_CN.utf8 |          |                       | 17 MB
 template0 | kingbase | UTF8     | zh_CN.utf8 | zh_CN.utf8 |          | =c/kingbase          +| 17 MB
           |          |          |            |            |          | kingbase=CTc/kingbase |
 template1 | kingbase | UTF8     | zh_CN.utf8 | zh_CN.utf8 |          | =c/kingbase          +| 17 MB
           |          |          |            |            |          | kingbase=CTc/kingbase |
 test      | kingbase | UTF8     | zh_CN.utf8 | zh_CN.utf8 |          |                       | 17 MB
(5 行记录)
```

当前所使用的 kingbase 库的磁盘占用较大，共享缓存明显偏小，我们增大 shared_buffers，修改 kingbase.conf 并重启服务器。

```
shared_buffers = 3GB
```

5. 调优后的 TPCC 测试结果：

```
Measured tpmC (NewOrders) = 16601.83
Measured tpmTOTAL = 36780.18
Session Start     = yyyy-MM-dd 07:15:19
Session End       = yyyy-MM-dd 07:16:19
Transaction Count = 36846
```

6. 优化后，DataFileRead 等待事件的数据库时间百分比相比调优前取得大幅下降，DataFileWrite 等待事件在子报告中已不再出现。

等待事件	等待次数	总时间(s)	平均时间(ms)	% 数据库时间	事件类型
DB CPU	-	83.01	-	46.96	-
DataFileRead	14416	49.44	3.43	27.97	IO
transactionid	4249	35.18	8.28	19.90	Lock
tuple	217	4.12	18.99	2.33	Lock
DataFileImmediateSync	12	4.04	336.55	2.28	IO
DataFileExtend	4409	0.57	0.13	0.33	IO
extend	12	0.37	30.63	0.21	Lock
wal_insert	224	0.02	0.11	0.01	LightLock
clog_ctl	76	0.01	0.07	0.00	LightLock
lock_manager	36	0.00	0.13	0.00	LightLock
buffer_content	84	0.00	0.05	0.00	LightLock

7.3.1.2　BufFileRead 和 BufFileWrite

BufFileRead 和 BufFileWrite 等待事件通常发生在执行排序、哈希计算等操作或创建索引等维护性工作期间，由于工作内存或维护内存不足，系统将临时数据写入并读取磁盘文件，从而引发 I/O 操作。

工作内存大小受 work_mem 参数控制，默认值为 4MB。适当扩大该参数可以增加内部排序和哈希计算的可用内存，进而优化系统性能。然而，由于每个客户端会话进程都有各自的工作内存；因此，在高并发场景下，过大的 work_mem 值可能导致数据库实例消耗过多内存资源，容易引发内存不足的严重问题。

以下为常见原因：

- work_mem 参数设置过小：ORDER BY 子句、GROUP BY 子句、Hash 连接、DISTINCT、窗口函数和 CREATE TABLE AS SELECT 语句等操作会使用工作内存。
- maintenance_work_mem 参数设置过小：CREATE INDEX 和 CLUSTER 语句会使用专用的维护工作内存。

以下是优化建议：

- 合理配置 work_mem：适当增大该参数值有助于减少临时文件的读写操作，从而提升性能。但是，过大的 work_mem 值可能导致数据库内存资源过度占用。
- 调整 maintenance_work_mem：通常维护性操作仅在少数几个客户端连接上执行，因此可以根据实际需求适当增加该参数值以优化性能。

- 控制客户端连接数量：由于每个活动连接都会占用 work_mem 设定的工作内存空间，过多的并发连接可能导致内存消耗过大，建议使用连接池机制限制同时活跃的客户端连接数量。

调优用例 BufFileRead & BufFileWrite：

1. KingbaseES 配置参数：

```
max_parallel_workers = 0
synchronous_commit = off
commit_delay = 10
commit_siblings = 16
shared_buffers = 3GB    # 扩大共享缓存区
```

上述参数可见配置管理手册的资源消耗、预写式日志章节。

2. 准备测试数据：

```
DROP TABLE IF EXISTS t1;
DROP TABLE IF EXISTS t2;
CREATE TABLE t1(id int, name text);
CREATE TABLE t2(id int, name text);
INSERT INTO t1 SELECT generate_series(1, 10000000), generate_series(1, 10000) || repeat('a', 20);
INSERT INTO t2 SELECT generate_series(1, 1000000),  generate_series(1, 10000) || repeat('a', 20);
```

创建名为 select.sql 的 SQL 文件，内容如下：

```
SELECT t1.id, t2.name FROM t1, t2 WHERE t1.id = t2.id ORDER BY 2 DESC, 1 DESC;
```

3. 执行 kbbench 负载测试，参数为 4 个并发线程和 1 分钟运行时间：

```
./kbbench -M prepared -n -r -f select.sql -P 5 -c 4 -j 4 -T 60 -p 65530 -U kingbase kingbase
```

4. 优化前，kbbench 的 TPS：

```
scaling factor: 1
query mode: prepared
number of clients: 4
number of threads: 4
duration: 60 s
number of transactions actually processed: 24
latency average = 12528.160 ms
latency stddev = 7950.497 ms
tps = 0.286422 (including connections establishing)
tps = 0.286492 (excluding connections establishing)
statement latencies in milliseconds:
     12528.160  select t1.id, t2.name from t1, t2 where t1.id = t2.id order by 2 desc, 1 desc;
```

5. 优化前，BufFileRead 和 BufFileWrite 等待事件的数据库时间百分比相对较高。

等待事件	等待次数	总时间(s)	平均时间(ms)	% 数据库时间	事件类型
DB CPU	-	207.02	-	68.33	-
BufferFileRead	488468	71.36	0.15	23.56	IO
BufferFileWrite	512367	11.09	0.02	3.66	IO
MessageQueueReceive	9940	7.41	0.75	2.45	IPC
ClientWrite	3089	4.81	1.56	1.59	Client
parallel_hash_join	52	0.50	9.58	0.16	LightLock
ParallelFinish	26	0.31	12.06	0.10	IPC
Hash/Build/HashingOuter	14	0.23	16.08	0.07	IPC
Hash/Build/HashingInner	11	0.16	14.76	0.05	IPC
parallel_query_dsa	5	0.03	5.88	0.01	LightLock
DataFileImmediateSync	12	0.01	1.04	0.00	IO

6. 分析和优化：BufFileRead 和 BufFileWrite 等待事件的数据库时间占比过高，表明业务 SQL 执行过程中存在排序操作的工作内存不足的问题，此时系统会使用临时文件进行排序操作。work_mem 默认为 4MB，分析 SQL 语句的执行计划，排序时使用的磁盘空间远大于 4MB，磁盘 I/O 较高，导致性能下降。

```
EXPLAIN ANALYZE SELECT t1.id, t2.name FROM t1, t2 WHERE t1.id = t2.id ORDER BY 2 DESC, 1 DESC;
                                                                    QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------
 Gather Merge  (cost=238002.83..335231.92 rows=833334 width=25) (actual time=9384.754..9845.130 rows=1000000
loops=1)
   Workers Planned: 2
   Workers Launched: 0
   ->  Sort  (cost=237002.81..238044.48 rows=416667 width=25) (actual time=9383.981..9699.567 rows=1000000
loops=1)
         Sort Key: t2.name DESC, t1.id DESC
         Sort Method: external merge  Disk: 34320kB
         ->  Parallel Hash Join  (cost=19632.00..188138.50 rows=416667 width=25) (actual time=3484.091..
8439.156 rows=1000000 loops=1)
               Hash Cond: (t1.id = t2.id)
               ->  Parallel Seq Scan on t1  (cost=0.00..115742.19 rows=4166719 width=4) (actual time=0.010..
1385.502 rows=10000000 loops=1)
               ->  Parallel Hash  (cost=11574.67..11574.67 rows=416667 width=25) (actual time=366.083..366.
084 rows=1000000 loops=1)
                     Buckets: 65536  Batches: 32  Memory Usage: 2496kB
                     ->  Parallel Seq Scan on t2  (cost=0.00..11574.67 rows=416667 width=25) (actual time=0.
098..153.355 rows=1000000 loops=1)
 Planning Time: 0.267 ms
 Execution Time: 9890.784 ms
(14 rows)
```

将 work_mem 扩大到合适的值，比如 512MB，然后重新执行测试 kbbench 测试。

```
kingbase=# show work_mem;
 work_mem
----------
 512MB
(1 行记录)
```

7. 优化后，kbbench 的 TPS 相较于优化前有显著提升：

```
scaling factor: 1
query mode: prepared
number of clients: 4
number of threads: 4
duration: 60 s
number of transactions actually processed: 36
latency average = 7186.711 ms
latency stddev = 2618.729 ms
tps = 0.546710 (including connections establishing)
tps = 0.546867 (excluding connections establishing)
statement latencies in milliseconds:
      7186.711  select t1.id, t2.name from t1, t2 where t1.id = t2.id order by 2 desc, 1 desc;
```

8. 优化后：BufFileRead 和 BufFileWrite 等待事件均不再出现。

等待事件	等待次数	总时间(s)	平均时间(ms)	% 数据库时间	事件类型
DB CPU	-	233.42	-	90.06	-
MessageQueueReceive	12756	10.17	0.80	3.92	IPC
ClientWrite	4287	7.55	1.76	2.91	Client
DataFileRead	28612	2.67	0.09	1.03	IO
parallel_hash_join	349	2.64	7.56	1.02	LightLock
buffer_io	238	2.27	9.54	0.88	LightLock
Hash/Build/HashingOuter	15	0.23	15.31	0.09	IPC
ParallelFinish	17	0.09	5.04	0.03	IPC
Hash/Build/HashingInner	14	0.06	4.25	0.02	IPC
BgWorkerShutdown	11	0.04	3.66	0.02	IPC
buffer_mapping	6	0.02	3.33	0.01	LightLock

7.3.2　LightLock 类等待事件分析与优化

7.3.2.1　buffer_content

当某个会话等待读取或写入内存中的某个数据页面，而另一个会话正锁定该页面以进行写入时，会发生 buffer_content 等待事件。

数据页的读写操作通常通过共享缓冲区来实现。具体而言，在执行读取操作前，系统会对目标数据页附加共享锁；而在进行写入操作之前，则需要对该数据页施加排他锁。当数据页处于共享锁定状态时，其他会话仍可执行对其的读取操作；然而，一旦施加了排他锁，所有其他的读写请求都将被阻塞，直到该锁被释放为止。

以下为常见原因：

- 大量并发会话同时尝试更新相同的缓冲区数据页易触发 buffer_content 等待事件，这种现象在涉及大量索引的关系表环境中尤为明显。
- 当请求访问的数据页未位于共享缓冲区中时，已持有排他锁的会话需执行磁盘 I/O 将所需数据页加载到内存中，这一过程需要较长时间，导致其他试图访问该数据页的会话出现更长时间的等待。
- 大量外键约束会在读取目标数据页时触发额外的锁定机制。具体而言，在验证这些外键约束时，系统需要检查相关引用的数据页，从而增加了额外的数据页锁定操作。

以下是优化建议：

- 共享缓冲区大小配置合理，尽量使工作负载相关数据页驻留在共享缓冲区内
- 对数据量大的表实施分区策略，使高访问频率的数据页驻留在共享缓冲区内
- 避免定义不必要的外键约束，减少对非必要数据页的访问开销
- 移除无用的索引

7.3.2.2　lock_manager

lock_manager 等待事件表示正等待增加或者检查用于后端的锁；或者在并行查询时，等待加入或者退出一个锁定组。为了在无法使用快速路径锁时分配、检查和取消分配锁，KingbaseES 需要对共享锁的内存区域进行维护，此时会产生 lock_manager 等待事件。

以下为常见原因：

- 并发活动会话正在执行不使用快速路径锁的查询，并且会话数量超过了最大 CPU 数量。
- 大量并发会话正在访问过度分区的表，并且每个分区都有多个索引。
- 数据库系统出现连接风暴。默认情况下，当数据库性能下降时，某些应用程序或连接池软件会自动增加连接数，此时会进一步恶化性能问题。
- 大量会话在不修剪分区的情况下查询父级表。
- DDL、DML 或维护命令锁定了经常访问或修改的繁忙关系或元组。

以下是优化建议：

- 删除不必要的索引：数据库中可能会存在未被利用或使用频率较低的索引。定期检查和清理这些无用的索引，有助于提升整体查询性能。
- 优化查询以实现快速路径锁定：可通过查询 sys_locks 表中的 fastpath 列确认的查询是否启用了快速路径锁定。若查询没有使用快速路径锁定，请尝试将每个查询的关系数减少到 16 个以下。
- 使用连接池程序：当活动连接的总数超过系统最大 CPU 数量时，系统 CPU 资源不足，此时应考虑使用或优化连接池。
- 优化其他等待事件：当 lock_manager 事件长期处于等待事件列表中的前两位时，可检查列表中是否同时存在 Relation、transactionid 和 tuple 相关的 Lock 类等待事件，若这些事件出现在等待事件列表的前列，请优先优化这些等待事件，这些等待事件可能是 lock_manager 等待事件的驱动因素。

7.3.2.3　wal_insert

WAL 日志是数据库恢复机制的核心组件之一，其主要功能是保障事务的持久性和数据的一致性。当 WAL 缓冲区空间不足或需要将 WAL 日志同步写入磁盘时，可能会触发 wal_insert 等待事件，该现象在 WAL 缓冲区容量较小时尤为显著。

以下为常见原因：

- WAL 缓冲区满：当 WAL 缓冲区已达到容量上限时，新的记录无法立即插入到缓冲区中。此时，数据库引擎会暂停处理新事务，直到现有记录被成功刷新至磁盘。
- 同步写入延迟：系统支持两种配置模式，全同步（即每次事务提交均立即将所有更改写入磁盘）或非全同步（允许部分更改暂时保留在缓存中）。若采用全同步模式，每次 WAL 记录的插入操作都需要等待 I/O 操作完成，从而导致额外的延迟。

以下是优化建议：

- 增加 WAL 缓冲区大小：调整配置参数 wal_buffers 的大小。
- 优化磁盘 I/O 性能：确保数据库服务器具备充足的存储带宽和低延迟特性。建议使用更快、更可靠的存储介质（如 SSD），以有效降低 I/O 等待时间。
- 调整同步模式：若全同步写入导致大量 wal_insert 等待事件的产生，可以将 WAL 日志同步级别从 fsync = on 调整为 fsync = off 或其他更宽松的设置（如使用异步 I/O）。由于同步级别的降低会导致数据安全性的下降，因此在高可用环境中应慎重调整同步级别。

优化用例：wal_insert

1. KingbaseES 配置参数：

```
shared_buffers = 3GB    # 扩大共享缓存区
wal_buffers = 32kB
```

上述参数可见配置管理手册的资源消耗、预写式日志章节。

2. TPCC 配置参数：

```
warehouses=10
terminals=10
runMins=1
```

3. 优化前的 TPCC 测试结果：

```
Measured tpmC (NewOrders) = 16446.08
Measured tpmTOTAL = 36624.88
Session Start     = yyyy-MM-dd 09:03:44
Session End       = yyyy-MM-dd 09:04:44
Transaction Count = 36666
```

4. 优化前，报告中存在 wal_insert 等待事件。

等待事件	等待次数	总时间(s)	平均时间(ms)	% 数据库时间	事件类型
DB CPU	-	125.70	-	55.28	-
WALWriteLock	19108	34.55	1.81	15.20	LightLock
transactionid	4008	29.43	7.34	12.94	Lock
DataFileRead	59980	15.61	0.26	6.86	IO
WALSync	32480	15.33	0.47	6.74	IO
wal_insert	4168	5.22	1.25	2.30	LightLock
WALWrite	56510	1.11	0.02	0.49	IO
buffer_content	118	0.23	1.96	0.10	LightLock
DataFileExtend	5101	0.13	0.02	0.06	IO
WALBufMappingLock	1269	0.05	0.04	0.02	LightLock
DataFileImmediateSync	12	0.01	1.10	0.01	IO

5. 分析和优化，WALWriteLock 和 wal_insert 等待事件的数据库时间占比高，可以适当扩大 WAL 日志缓冲区容量，使 WAL 记录能够充分驻留在缓冲区，并以批处理的方式一次性写入磁盘文件。修改 kingbase.conf，重启服务器：

```
shared_buffers = 3GB    # 扩大共享缓存区
wal_buffers = 32MB
```

6. 优化后的 TPCC 测试结果相较于优化前取得提升。

```
Measured tpmC (NewOrders) = 17006.45
Measured tpmTOTAL = 37525.94
Session Start     = yyyy-MM-dd 09:11:59
Session End       = yyyy-MM-dd 09:12:59
Transaction Count = 37610
```

7. 优化后，wal_insert 等待事件的数据库时间百分比低于优化前。

等待事件	等待次数	总时间(s)	平均时间(ms)	% 数据库时间	事件类型
DB CPU	-	139.66	-	69.65	-
transactionid	3605	28.03	7.78	13.98	Lock
WALSync	33112	12.87	0.39	6.42	IO
WALWriteLock	8136	9.67	1.19	4.82	LightLock
DataFileRead	69626	7.39	0.11	3.69	IO
tuple	165	1.13	6.85	0.56	Lock
WALWrite	33112	1.00	0.03	0.50	IO
DataFileExtend	11668	0.45	0.04	0.23	IO
wal_insert	1594	0.23	0.14	0.11	LightLock
extend	19	0.03	1.56	0.01	Lock
DataFilePrefetch	5248	0.02	0.00	0.01	IO

7.3.2.4　WALWriteLock

WALWriteLock 等待事件发生在等待 WAL 记录由缓冲区写入到磁盘，或者 WAL 段切换期间。

当一个事务尝试写入 WAL 记录时，必须先获取 WALWriteLock 锁以保证该操作的原子性，并防止其他事务对同一 WAL 记录进行并发修改。这种锁机制能够有效避免数据不一致状态的发生。

如果当前事务需要插入新的 WAL 记录，而另一个事务已经持有 WALWriteLock 锁，则当前事务将被阻塞并进入 WALWriteLock 等待状态。在写入 WAL 记录较为频繁的情况下，如果同时存在大量事务请求，可能会导致 WALWriteLock 等待事件数量较高。

以下为常见原因：

- 事务对表执行大量修改操作时触发了大量 WAL 日志写入。
- 执行了大量小的事务，每个提交动作本身都会触发 WAL 日志写入。

以下是优化建议：

- 尽量减少频繁的小型事务，并考虑将它们合并或分批处理以降低锁竞争。
- 通过增加数据库可用的 WAL 缓冲区容量来降低磁盘写入频率，缓解锁竞争。
- 使用分组提交方式减少写入 WAL 日志的次数，以增加单位时间内事务的提交量：

```
commit_delay = 10
commit_siblings = 16
```

- 通过异步提交和关闭整页写的方式降低写 WAL 日志的数据量：

```
synchronous_commit=off
full_page_writes=off
```

注意:

1. 上述参数可见配置管理手册的预写式日志章节。
2. 使用异步提交存在风险：异步提交（synchronous_commit=off）可以在提交事务的时候不等待 WAL 日志写入磁盘就返回，存在丢失事务的风险，需谨慎使用。
3. 关闭整页写存在风险：禁用整页写入（full_page_writes=off）后，在系统崩溃时，可能会导致数据丢失。若服务器硬件能够降低部分页面写入的风险（例如配置了电池供电的磁盘控制器或使用支持原子操作的文件系统），并且在充分评估后认为关闭此参数所引发的数据丢失风险处于可接受范围内，则可以考虑禁用该功能。

优化用例：WALWriteLock

1. KingbaseES 配置参数：无
2. TPCC 配置参数：

```
warehouses=10
terminals=10
runMins=1
```

3. 优化前的 TPCC 测试结果：

```
Measured tpmC (NewOrders) = 3389.41
Measured tpmTOTAL = 7497.88
Session Start     = yyyy-MM-dd 07:05:14
Session End       = yyyy-MM-dd 07:06:15
Transaction Count = 7538
```

4. 优化前，报告中存在 WALWriteLock 等待事件。

等待事件	等待次数	总时间(s)	平均时间(ms)	% 数据库时间	事件类型
DB CPU	-	37.36	-	7.09	-
DataFileRead	46509	420.62	9.04	79.87	IO
transactionid	676	27.34	40.44	5.19	Lock
WALWriteLock	1607	27.13	16.89	5.15	LightLock
WALSync	6605	10.76	1.63	2.04	IO
WALWrite	6642	1.37	0.21	0.26	IO
buffer_io	47	1.26	26.85	0.24	LightLock
DataFileExtend	2259	0.55	0.24	0.10	IO
DataFileWrite	12938	0.22	0.02	0.04	IO
wal_insert	153	0.01	0.10	0.00	LightLock
DataFileImmediateSync	12	0.01	1.17	0.00	IO

5. 分析和优化：KingbaseES 默认采用同步提交 WAL 日志的方式，即每次事务提交的时，需等待 WAL 日志写入磁盘完成。可以采用异步提交的方式，即 WAL 日志写请求提交后事务立刻返回，之后继续执行其它任务。并且可以通过分组提交的方式合并一些 WAL 日志的提交，从而降低磁盘 I/O 操作频率。修改 kingbase.conf，重启服务器：

```
commit_delay = 10
commit_siblings = 16
synchronous_commit = off
```

6. 优化后的 TPCC 测试结果相较于优化前取得巨大提升。

```
Measured tpmC (NewOrders) = 15183.24
Measured tpmTOTAL = 33674.97
Session Start     = yyyy-MM-dd 07:10:55
Session End       = yyyy-MM-dd 07:11:55
Transaction Count = 33720
```

7. 优化后，WALWriteLock 等待事件的数据库时间百分比低于优化前。

等待事件	等待次数	总时间(s)	平均时间(ms)	% 数据库时间	事件类型
DB CPU	-	112.23	-	43.39	-
DataFileRead	158656	98.62	0.62	38.13	IO
transactionid	4044	37.82	9.35	14.62	Lock
WALWriteLock	18	3.79	210.64	1.47	LightLock
tuple	138	2.91	21.10	1.13	Lock
DataFileWrite	80189	1.87	0.02	0.72	IO
wal_insert	405	0.92	2.28	0.36	LightLock
DataFileExtend	6672	0.34	0.05	0.13	IO
extend	10	0.09	8.51	0.03	Lock
buffer_content	206	0.04	0.18	0.01	LightLock
DataFileImmediateSync	12	0.01	1.06	0.00	IO

7.3.3　Lock 类等待事件分析与优化

7.3.3.1　extend

Lock 类 extend 等待事件表示正等待扩展一个关系。

当后端进程试图扩展一个被另一个进程锁定的关系时，若此时有其他进程也尝试进行相同的扩展操作，则会产生 extend 等待事件。由于每次只有一个进程能够执行关系的扩展操作，因此系统会产生此等待事件，此事件通常由 INSERT、COPY 和 UPDATE 等操作触发。

以下为常见原因：

- 并发插入或更新激增：对同一个表执行插入或更新的操作的并发会话数会大幅增加。
- 网络带宽不足：数据库实例的网络带宽无法满足当前工作负载的需求，引发存储延迟，从而增加了 extend 等待事件的发生频率。

以下是优化建议：

- 减少对同一个表的并发插入和更新操作：通过合理控制并发度或采用适当的锁机制来降低竞争。
- 提高网络带宽：升级数据库实例的网络配置以确保足够的带宽资源，减少通信延迟。

7.3.3.2　Relation

Lock 类的 Relation 等待事件表示正等待获取一个关系上的锁。

当查询等待获取表或视图上已被另一个事务获取的锁时，系统会产生 Relation 事件。

大多数 SQL 语句都会隐式使用锁来控制对表中数据的并发访问。除隐式锁定外，还可以显式使用 LOCK 命令来管理这些锁操作。由于不同的锁定模式之间彼此可能不兼容，当多个事务尝试同时访问同一数据库对象时，可能会引发锁竞争并产生 Relation 事件。

以下是常见原因：

- 并发会话数的增加：当使用冲突锁定模式（如排他锁或共享更新锁）操作同一张表时，并发会话数的增加会提高出现锁冲突的概率。

- 维护操作：VACUUM 和 ANALYZE 等数据库维护命令可能会导致 Relation 等待事件的增加。具体来说，VACUUM FULL 操作会获取 ACCESS EXCLUSIVE 锁，ANALYZE 操作会获取 SHARE UPDATE EXCLUSIVE 锁，这两种锁都可能产生 Relation 等待事件。可能对正在运行的查询或事务造成阻塞。应用层面的数据维护操作（例如刷新物化视图）也可能增加 Relation 等待事件的发生频率。
- 读写器实例之间的锁冲突：写入器和读取器持有的关系锁之间可能存在锁冲突。当前系统仅将 ACCESS EXCLUSIVE 关系锁复制到读取器实例中，但 ACCESS EXCLUSIVE 关系锁与读取器持有的任何 ACCESS SHARE 关系锁冲突，从而导致读取器上的 Relation 等待事件显著增加。

以下是优化建议：

- 减少阻塞 SQL 语句的影响：对于某些 SQL 语句（如 SELECT 和 LOCK），可以使用 NOWAIT 选项，如果无法立即获取锁，则 NOWAIT 选项会取消请求锁定的查询。此外，可以在会话级别设置 lock_timeout 参数限制 SQL 语句等待获取关系锁的时间。若未在指定时间范围内获取关系锁，则取消请求锁定的事务。
- 减少维护操作的影响：比如在非高峰时段手动执行维护操作，减少 Autovacuum 任务导致的 Relation 等待事件。
- 检查读取器锁：定期监控和分析读取器上的锁状态。

7.3.3.3 transactionid

Lock 类 transactionid 等待事件表示正等待一个事务结束。

当一个事务尝试获取已经被并发运行的其他事务持有的行级锁时，会触发 transactionid 等待事件，此时尝试获取行级锁的事务将无法继续执行，直到已持有锁的事务在 COMMIT 或 ROLLBACK 语句完成后才可继续执行。

当此 transactionid 事件出现频率超过正常值时，通常是 UPDATE、SELECT …FOR UPDATE 或 SELECT … FOR KEY SHARE 语句引起的。

以下为常见原因：

- 高并发性：KingbaseES 采用精细的行级锁定机制。在高并发环境下，多个事务可能对同一行记录竞争锁资源。
- 未提交事务的影响：空闲但持有锁的事务会阻止其他事务访问被锁定的数据行。
- 长时间运行的事务：长时间未完成的事务会导致其持有的锁被长期占用，从而阻塞其他事务的执行。

以下是优化建议：

- 应对高并发场景：减少系统中的并发连接数量，例如控制活跃会话数上限，或采用连接池技术；同时优化数据模型和访问模式以降低 UPDATE 和 SELECT …FOR UPDATE 语句的锁争用情况。
- 处理空闲事务：尽可能开启自动提交，确保应用程序正确处理 COMMIT、ROLLBACK 或 END 操作。
- 应对长时间运行的事务：在长事务场景中应尽量避免长时间锁定行记录；同时可以通过设置合适的超时策略来限制查询执行时间。

7.3.3.4　tuple

Lock 类的 tuple 等待事件表示一个后端正在等待获取元组上的锁，而另一个后端在同一元组上持有互斥锁。

以下是常见原因：

- 大量并发会话执行 UPDATE 或 DELETE 语句以尝试获取同一元组的锁。
- 高度并发的会话使用 FOR UPDATE 或 FOR NO KEY UPDATE 锁定模式运行 SELECT 语句。
- 应用或连接池因各种因素开启更多会话，这些新会话试图修改相同的行，导致数据库负载急剧增加。

以下是优化建议：

- 检查应用程序的逻辑状态：检查会话是否长时间处于’idle in transaction’状态。如果是，请考虑使用 sys_terminate_backend 函数终止该会话。
- 分析阻塞会话：tuple 等待事件发生时，分析锁之间的互相依存关系以确定阻塞源会话及其被阻塞的目标会话。
- 减少系统并发程度：通过优化资源访问模式来降低整体并发水平。
- 优化表结构与数据组织方式：通过对表结构进行合理调整，改善数据分布特性，从而降低高并发环境下的竞争热点。
- 改进应用业务逻辑：通过调整业务处理流程，减少对行级锁的竞争强度。
- 合理运用行锁机制和重新设计查询：通过对访问模式的分析，优化查询语句以提高锁资源的利用效率。
- 采用具有重试机制的 NOWAIT 子句：在进行锁定请求时，配置适当的超时参数和重试策略，以避免多余的等待时间。
- 引入优化的并发控制策略：评估并采用基于乐观锁或混合锁机制的并发控制方案，以提高系统整体性能。
- 调整数据库事务隔离级别：根据实际业务需求，适当降低或升高事务隔离级别，以在数据一致性与系统吞吐量之间取得平衡。

调优用例：transactionid & tuple

1. KingbaseES 配置参数：

```
synchronous_commit = off
commit_delay = 10
commit_siblings = 16
shared_buffers = 3GB    # 扩大共享缓存区
```

上述参数可见配置管理手册的预写式日志、资源消耗章节。

2. TPCC 配置参数：

```
warehouses=2
terminals=10
runMins=1
```

3. 优化前的 TPCC 测试结果：

```
Measured tpmC (NewOrders) = 14816.25
Measured tpmTOTAL = 33103.65
Session Start     = yyyy-MM-dd 07:47:18
Session End       = yyyy-MM-dd 07:48:18
Transaction Count = 33149
```

4. 优化前，transactionid 等待事件占数据库时间的百分比较高，并且数据库时间中存在一定百分比的 tuple 等待事件。

等待事件	等待次数	总时间(s)	平均时间(ms)	% 数据库时间	事件类型
DB CPU	-	119.63	-	43.72	-
transactionid	14890	114.23	7.67	41.75	Lock
DataFileRead	5863	22.22	3.79	8.12	IO
tuple	2115	17.30	8.18	6.32	Lock
DataFileExtend	5259	0.15	0.03	0.05	IO
wal_insert	390	0.04	0.10	0.01	LightLock
lock_manager	207	0.02	0.11	0.01	LightLock
buffer_content	395	0.02	0.04	0.01	LightLock
DataFileImmediateSync	12	0.01	1.05	0.00	IO
clog_ctl	79	0.00	0.06	0.00	LightLock
extend	8	0.00	0.34	0.00	Lock

5. 分析和优化：在 10 个 TPCC 客户端并发访问 2 个数据仓库，引起 TPCC 测试表（包括关系表、元组等对象锁）的锁竞争显著增加。为缓解这一问题，可适当控制 TPCC 客户端的并发数量，使其与数据库仓库的数量保持接近 1:1 的比例，从而有效降低关系表和元组上的锁冲突。

```
warehouses=10
terminals=10
runMins=1
```

6. 优化后的 TPCC 测试结果相较于优化前取得较大提升。

```
Measured tpmC (NewOrders) = 19510.44
Measured tpmTOTAL = 42774.5
Session Start     = yyyy-MM-dd 07:50:06
Session End       = yyyy-MM-dd 07:51:06
Transaction Count = 42812
```

7. 优化后，transactionid 等待事件的数据库时间百分比显著下降，tuple 等待事件在子报告中不再出现。

等待事件	等待次数	总时间(s)	平均时间(ms)	% 数据库时间	事件类型
DB CPU	-	138.17	-	91.25	-
transactionid	1097	7.55	6.89	4.99	Lock
DataFileRead	9834	5.18	0.53	3.42	IO
DataFileExtend	7175	0.26	0.04	0.17	IO
DataFileImmediateSync	12	0.11	8.78	0.07	IO
wal_insert	527	0.09	0.16	0.06	LightLock
extend	18	0.04	2.46	0.03	Lock
clog_ctl	116	0.01	0.06	0.00	LightLock
buffer_content	31	0.00	0.08	0.00	LightLock
DataFilePrefetch	142	0.00	0.02	0.00	IO
lock_manager	13	0.00	0.07	0.00	LightLock

7.3.4　Client 类等待事件分析与优化

7.3.4.1　ClientRead

Client 类的 ClientRead 等待事件表示系统正在等待从客户端读取数据。

当 KingbaseES 等待从客户端接收数据时，会产生 ClientRead 事件。

以下为常见原因：

- 网络延迟增加：较高的网络延迟会增加数据库集群从客户端接收数据所需的时间。
- 客户端负载过高：客户端负载的增加可能会导致客户端向数据库集群传输数据的过程中产生时间延迟。
- 过多的网络往返次数：大量网络往返操作可能会影响数据从客户端到数据库集群的传输效率。
- 大型复制操作：向数据库集群发送大量数据会占用较多时间，影响客户端与服务器之间的数据传输速度。
- 空闲客户端连接：处于事务状态的客户端连接长期处于空闲状态，正在等待客户端发送更多数据或新的命令。

以下是优化建议：

- 检查和优化网络连接：使用专业工具诊断并降低网络延迟，确保网络连接稳定。使用更高效的网络配置，例如通过增加带宽或减少网络跳数等方式改善网络性能。
- 监控客户端性能：实时监控客户端的 CPU 和内存使用情况，确保其具备足够的资源支持数据传输需求，例如通过优化客户端应用程序的负载均衡策略缓解高负载问题。
- 批量处理数据：对于需要传输大量数据的情况（例如大型结果集），可考虑采用分批读取或优化查询的方式减轻客户端和服务器之间的压力。
- 利用连接池和缓存机制：运用连接池管理客户端与数据库之间的连接，以减少每次请求的建立时间，同时可以结合缓存技术降低频繁的数据传输需求。
- 优化查询语句：确保发送至数据库的 SQL 语句高效简洁，避免执行耗时操作如全表扫描等。
- 使用性能监控工具：使用动态性能视图、KWR 报告、kbbadger或其他性能监控工具，实时跟踪 ClientRead 事件的发生频率及其根本原因，采并根据监控结果采取相应的优化措施。

7.3.4.2 ClientWrite

Client 类 ClientWrite 等待事件表示系统正在等待向客户端写入数据。

当 KingbaseES 等待将数据写入客户端时，会发生 ClientWrite 事件。

KingbaseES 数据库集群与客户端之间的网络吞吐量下降可能导致此事件的发生。同时，客户端的 CPU 压力大以及网络饱和也可能引起此事件频繁发生；具体而言，CPU 压力大指 CPU 被充分占用并且存在任务等待处理；网络饱和指数据库与客户端之间网络传输的数据量超过网络的传输能力。

以下为常见原因：

- 网络延迟增加：KingbaseES 与客户端之间的网络延迟升高，延长了客户端接收数据所需的时间。
- 客户端负载过高：客户端面临 CPU 压力或网络饱和问题，延缓了从 KingbaseES 接收数据的速度。
- 大量数据传输至客户端：KingbaseES 向客户端发送了大量的数据，客户端无法以与服务器发送速度相同的速度接收这些数据，例如大型表的复制操作可能会导致 ClientWrite 事件显著增加。

以下是优化建议：

- 优化网络性能：将客户端部署在与 KingbaseES 相同的子网中。
- 减少传输数据量：调整应用程序以降低 KingbaseES 发送至客户端的数据量，缓解客户端 CPU 和网络资源的压力。
- 扩展客户端资源：若客户端性能成为瓶颈，应增加客户端的硬件资源。

对于网络层性能问题，一般应该考虑减少网络的传输，如：

- 优化应用逻辑：通过修改 SQL 语句或使用 LIMIT 分页等方式减少返回的数据行数（需根据实际应用场景选择合适的策略）。
- 配置批量返回数据：在接口端设置 defaultRowFetchSize 以实现批量数据返回。需要注意的是，如果应用程序确实需要较多返回数据，采用批量返回模式可能增加额外开销。此外，在自动提交模式下使用批量返回功能时，需进行相应的服务器端配置（具体参见网络优化相关参数说明）。
- 错峰执行高带宽操作：将占用较多网络资源的语句或操作安排在不同的时间窗口执行。

调优用例：ClientWrite

1. KingbaseES 配置参数：

```
synchronous_commit = off
commit_delay = 10
commit_siblings = 16
shared_buffers = 3GB
```

上述参数可见配置管理手册的预写式日志、资源消耗章节。

2. 准备数据：

在 kingbase 库下创建表：

```
DROP TABLE t1;
CREATE TABLE t1(id int, name text);
INSERT INTO t1 SELECT generate_series(1, 400000), repeat('a', 400);
```

创建名为 select.sql 的 SQL 文件，内容如下：

```
SELECT * FROM t1;
```

执行 kbbench 负载测试，参数为 4 个并发线程和 1 分钟运行时间：

```
./kbbench -M prepared -n -r -f select.sql -P 5 -c 4 -j 4 -T 60 -p 65530 -U kingbase kingbase
```

信息:

-p 端口号，-U 用户名

---

3. 优化前的 TPS：

```
scaling factor: 1
query mode: prepared
number of clients: 4
number of threads: 4
duration: 60 s
number of transactions actually processed: 62
latency average = 3973.085 ms
latency stddev = 1148.465 ms
tps = 1.001276 (including connections establishing)
tps = 1.001626 (excluding connections establishing)
statement latencies in milliseconds:
      3973.085  Select * from t1;
```

4. 优化前，ClientWrite 等待事件占数据库时间的百分比非常高：

等待事件	等待次数	总时间(s)	平均时间(ms)	% 数据库时间	事件类型
DB CPU	-	22.80	-	9.43	-
ClientWrite	59665	217.97	3.65	90.12	Client
DataFileRead	22800	1.07	0.05	0.44	IO
DataFileImmediateSync	12	0.01	1.15	0.01	IO
DataFileExtend	87	0.00	0.01	0.00	IO
WALSync	1	0.00	1.05	0.00	IO
WALWrite	1	0.00	0.36	0.00	IO
RelationMapRead	1	0.00	0.02	0.00	IO

5. 分析和优化：ClientWrite 等待事件占比高，主要是由于业务查询语句返回了全部的结果集，导致 KingbaseES 服务器在传输大结果集到客户端时开销较大。为解决此问题，可以提高网络带宽，也可以限制结果集大小（例如仅返回前 100 条记录：`SELECT * FROM t1 LIMIT 100;`，依赖于具体业务场景，此处仅为示例说明）。

6. 优化后的 TPS 相较于优化前有巨大提升。

```
scaling factor: 1
query mode: prepared
number of clients: 4
number of threads: 4
duration: 60 s
number of transactions actually processed: 1583063
latency average = 0.151 ms
latency stddev = 0.166 ms
tps = 26382.209334 (including connections establishing)
tps = 26393.508140 (excluding connections establishing)
statement latencies in milliseconds:
         0.150  Select * from t1 limit 100;
```

7. 优化后：ClientWrite 等待事件在子报告中不再出现。

等待事件	等待次数	总时间(s)	平均时间(ms)	% 数据库时间	事件类型
DB CPU	-	144.82	-	99.68	-
DataFileImmediateSync	12	0.46	38.70	0.32	IO
DataFileExtend	44	0.00	0.03	0.00	IO
DataFileRead	15	0.00	0.03	0.00	IO
WALSync	1	0.00	0.39	0.00	IO
lock_manager	2	0.00	0.12	0.00	LightLock
WALWrite	1	0.00	0.07	0.00	IO
RelationMapRead	2	0.00	0.02	0.00	IO

7.4　CPU 瓶颈分析与优化

当数据库没有明显等待事件时，表明限制数据库性能提升的主要因素在于 CPU 计算速度。

可考虑以下优化方向：

1. 实例效率分析与优化：通过调整实例层面的参数设置，能够实现更广泛的性能改进，建议优先采用此类优化措施。
2. SQL 分析与优化：按照执行时间从长到短的顺序，对 TOP SQL 语句进行逐条分析和优化，是提高数据库性能的一种常见且有效的方法。
3. 硬件与架构层面的分析与优化：当软件层面的优化无法满足需求时，可考虑通过升级硬件或调整系统架构实现更高的性能提升。

7.4.1　实例效率分析与优化

- 使用执行计划缓存：对于大量短语句来说，词法、语法和语义解析阶段以及生成执行计划阶段所占用的时间比例较高。启用执行计划缓存功能可以有效降低数据库的响应时间。目前 KingbaseES 支持扩展协议的本地执行

计划缓存和普通协议的全局执行计划缓存，这两种方式均可显著提升系统性能。

- 绑核：将服务器进程 kingbase 绑定到固定的 CPU 核号上运行，可以提高 CPU 缓存命中率，改善整体性能指标。具体操作方法可参考 bindcpulist 参数设置。

7.4.2　SQL 分析与优化

- 收集统计信息：准确的统计信息是生成合理执行计划的基础，统计信息不准可能误导优化器选择不合理的执行策略。
- 创建索引：构建高效的索引能够显著减少查询响应时间。
- 控制执行计划：针对复杂的 SQL 语句，由于优化器的局限性或估算偏差，可能导致生成的执行计划不是最优，此时可通过使用 HINT 或调整相关参数等方式人为干预以确保执行策略的合理性。
- 使用并行：在 CPU 资源充足的环境下，启用并行查询机制可以有效提升复杂查询的执行效率
- 改写语句：采用更高效的编码方式改写 SQL 语句，例如将 UNION 改为 UNION ALL、将过滤条件下推至基表以减少不必要的数据操作。

7.4.3　架构与硬件分析与优化

- 升级架构：如通过实施读写分离方案提高高并发读场景下的系统吞吐量。
- 升级硬件，通过升级硬件配置，如增强 CPU 性能，可以有效提升系统的纵向扩展能力。

版权声明

中电科金仓（北京）科技股份有限公司（简称：电科金仓）版权所有，并保留对本手册及本声明的一切权利。

未得到电科金仓的书面许可，任何人不得以任何方式或形式对本手册内的任何部分进行复制、摘录、备份、修改、传播、翻译成其他语言、将其全部或部分用于商业用途。

免责声明

本手册内容依据现有信息制作，由于产品版本升级或其他原因，其内容有可能变更。电科金仓保留在没有任何通知或者提示的情况下对手册内容进行修改的权利。

本手册仅作为使用指导，电科金仓在编写本手册时已尽力保证其内容准确可靠，但并不确保手册内容完全没有错误或遗漏，本手册中的所有信息也不构成任何明示或暗示的担保。

技术支持

- 电科金仓官方网站：http://www.kingbase.com.cn/
- 电科金仓文档中心：http://help.kingbase.com.cn/
- 全国服务热线：400-601-1188
- 电科金仓技术支持与反馈信箱：support@kingbase.com.cn

服务周期承诺

由于市场需求在不断变化，技术创新和发展的进程不断加剧，产品的版本更迭不可避免。电科金仓对于产品版本生命周期的有效管理，有助于您提前规划项目，更好地从产品服务终止上过渡。

表 1: KingbaseES 产品生命周期里程碑

关键里程碑点	定义
产品发布日期	产品正式发布版本，即 GA（general availability）版本的发布日期。
停止销售日期	正式停止销售的日期，版本停止接受订单日。该日之后，产品将不再销售。
停止功能升级日期	在该日期之后，不再提供新特性和新硬件支持。但依旧提供错误修复、安全修复、功能维护等服务。
停止功能维护日期	在该日期之后，不再维护功能，修复问题。但依旧提供安全修复等服务
停止安全维护日期	在该日期之后，不再发布补丁版本修复中高风险漏洞，仅提供有限的支持。
产品服务终止日期	停止提供产品服务和支持的日期。包括软件维护版本，缺陷修复，以及针对该产品的所有服务支持（包括服务热线和远程/现场支持）。

服务周期策略

金仓数据库管理系统 KingbaseES 产品确保以下的服务周期：

1）产品自发布之日起至产品停止功能升级（包含新特性、新硬件支持）之日止不少于 5 年。

2）产品停止功能升级之日起至产品停止功能维护（主要包括问题修复）之日止不少于 4 年。

3）产品功能维护停止之日起至产品停止安全维护（包括中高风险漏洞修复）之日止不少于 2 年。

针对金仓数据库管理系统 KingbaseES V9 产品（发布日期：2022 年 7 月 15 日），其具体服务周期安排如下：

- 产品销售日期：以产品销售合同所约定日期为准。
- 停止升级日期：自产品销售日期起计算，满 5 年之日止。
- 服务终止日期：自产品销售日期起计算，满 11 年之日止。

服务终止策略

金仓数据库管理系统 KingbaseES 产品确保在销售后，至少提供 6 年的服务支持。

备注:

电科金仓将会综合各方因素来确定产品服务终止日期。并将在实际产品服务终止日期之前至少 90 天，通过公开方式宣布产品服务终止日期。